

HERMES Dossiê técnico para revisão

Rodada R200 · versão 2.00 · XAUUSD M30 · 13/09/2026

Conclusão: a Referência permanece como base de pesquisa. Os dois complementos de pivô aumentaram a atividade, reduziram o lucro líquido e tiveram mais meses completos negativos. O caso Início reduziu o drawdown relativo, mas registrou uma queda máxima em dólares maior. Nenhum dos três atingiu a meta de todos os meses positivos.

Caso	Lucro líquido USD	Trades	Fator de lucro	DD relativo	Meses negativos ¹
1 Referência	204.912,54	261	2,01	42,76%	17
2 Continuidade	191.523,99	299	1,80	43,58%	22
3 Início	169.116,06	342	1,59	36,85%	18

¹ Janeiro/2022 a agosto/2026: 56 meses completos. Setembro/2026 é parcial e está incluído nos lucros totais. Contando esse mês parcial, o caso Início tem 19 meses negativos, contra 18 na janela completa.

Capital inicial reportado: **US\$ 10.000** por caso. Lote fixo **1,00**, alvo nominal **5R**, uma posição por vez. Custos de swap estão incluídos; comissões e taxas foram reportadas como zero. Os testes foram executados pelo usuário e os CSVs foram auditados aqui.

O que este documento entrega

Comparativo auditado; tabelas de todos os meses; resultados anuais em dólar e comparação em reais com Selic; gráficos; regras dos três perfis; auditoria técnica do código; limitações dos dados; roteiro de revisão para o Claude. O anexo final reproduz o fonte completo da versão entregue.

O código e os resultados estão associados à mesma rodada, mas os CSVs não incluem o hash do executável EX5. O hash do fonte e a reprodução da referência dão rastreabilidade, sem provar identidade binária. Trata-se de auditoria técnica interna, sem certificação externa.

01 · O efeito das entradas novas

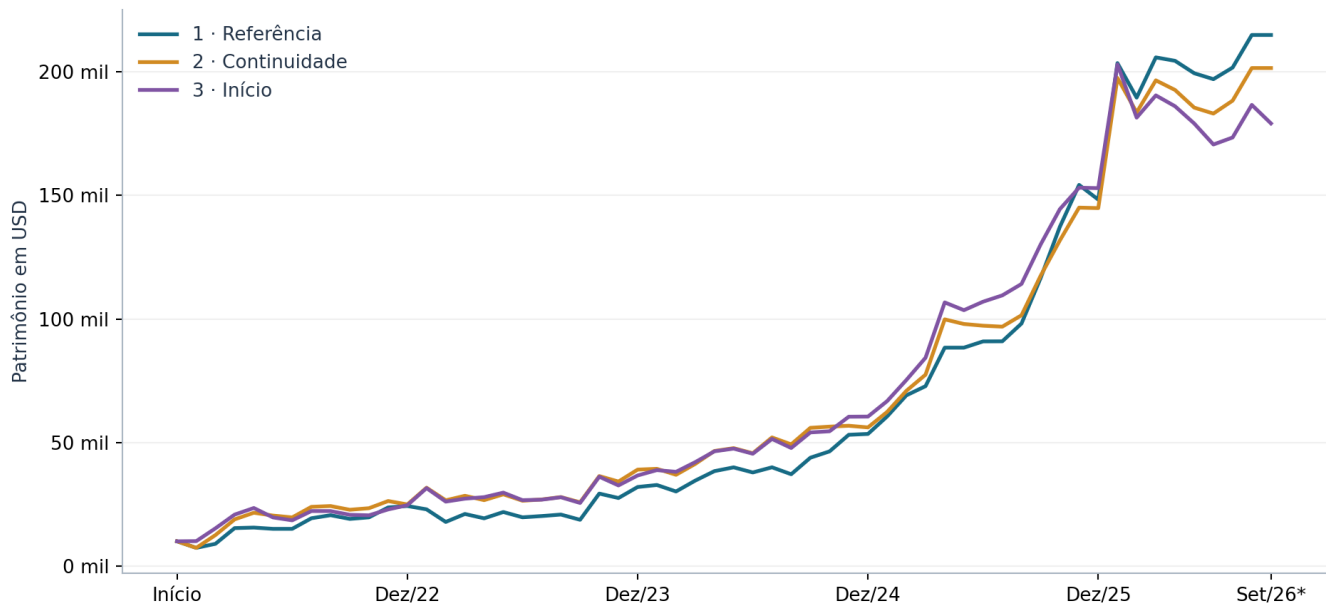
Métrica	1 Referência	2 Continuidade	3 Início
Lucro USD	204.912,54	191.523,99	169.116,06
Diferença vs. referência	0,00	-13.388,55	-35.796,48
Variação do lucro	0,00%	-6,53%	-17,47%
Acerto	26,05%	25,42%	24,56%
Entradas por PIVOT	0	58	111
Entradas por BASE (diferença)	261	241	231
Swap USD	-17.865,66	-19.737,81	-21.182,04
DD máximo em dinheiro USD	29.778,30	34.410,58	55.534,40
Positivos / negativos / zeros ¹	37 / 17 / 2	34 / 22 / 0	38 / 18 / 0
Setembro/2026 parcial USD	0,00	0,00	-7.502,29

¹ Meses completos. A classificação usa mudança de patrimônio (equity), incluindo posições abertas na virada. Saldo realizado aparece nos CSVs de origem e é conciliado separadamente.

O caso 2 executou 58 entradas por pivô, mas teve apenas 38 operações a mais no total; as entradas BASE passaram de 261 para 241. No caso 3, foram 111 entradas por pivô e 81 operações líquidas a mais; BASE caiu para 231. A nova ocupação altera oportunidades seguintes. Sem os ciclos individuais R200, não é possível atribuir a diferença de lucro exclusivamente aos trades PIVOT nem identificar exatamente quais entradas BASE foram substituídas.

Drawdown relativo e drawdown máximo em dinheiro podem ocorrer em instantes diferentes. O caso 3 teve DD relativo menor, mas DD máximo em dinheiro de US\$ 55.534,40. Isso impede concluir que ele melhorou todos os aspectos de risco.

02 · Evolução do patrimônio



Fonte: equity_end dos CSVs mensais, com capital inicial. Linha une observações de fim de mês; não reconstrói o caminho intradiário. * Setembro/2026 parcial.

Leitura do gráfico

O gráfico permite comparar a trajetória de capital no mesmo calendário. O valor final menor das versões de pivô é compatível com a conciliação dos resultados mensais. O crescimento percentual depende também do capital acumulado no início de cada período; não significa aumento automático de lote, que permanece fixo.

As curvas de mês fechado não mostram todos os picos e vales do teste. Para risco, usar também o drawdown intrateste reportado pelo MT5. A qualidade e cobertura dos ticks, latência, corretora e especificações completas precisam ser conferidas no relatório nativo de cada caso.

Protocolo de comparação

Janela planejada: 01/01/2022 até 12/09/2026 exclusivo. Avaliações reportadas: 02/01/2022 às 23:05 até 11/09/2026 às 20:30, horário do servidor. Cada caso reportou 55.498 barras avaliadas. O período foi usado repetidamente para pesquisa e seleção; não é uma amostra inédita de validação.

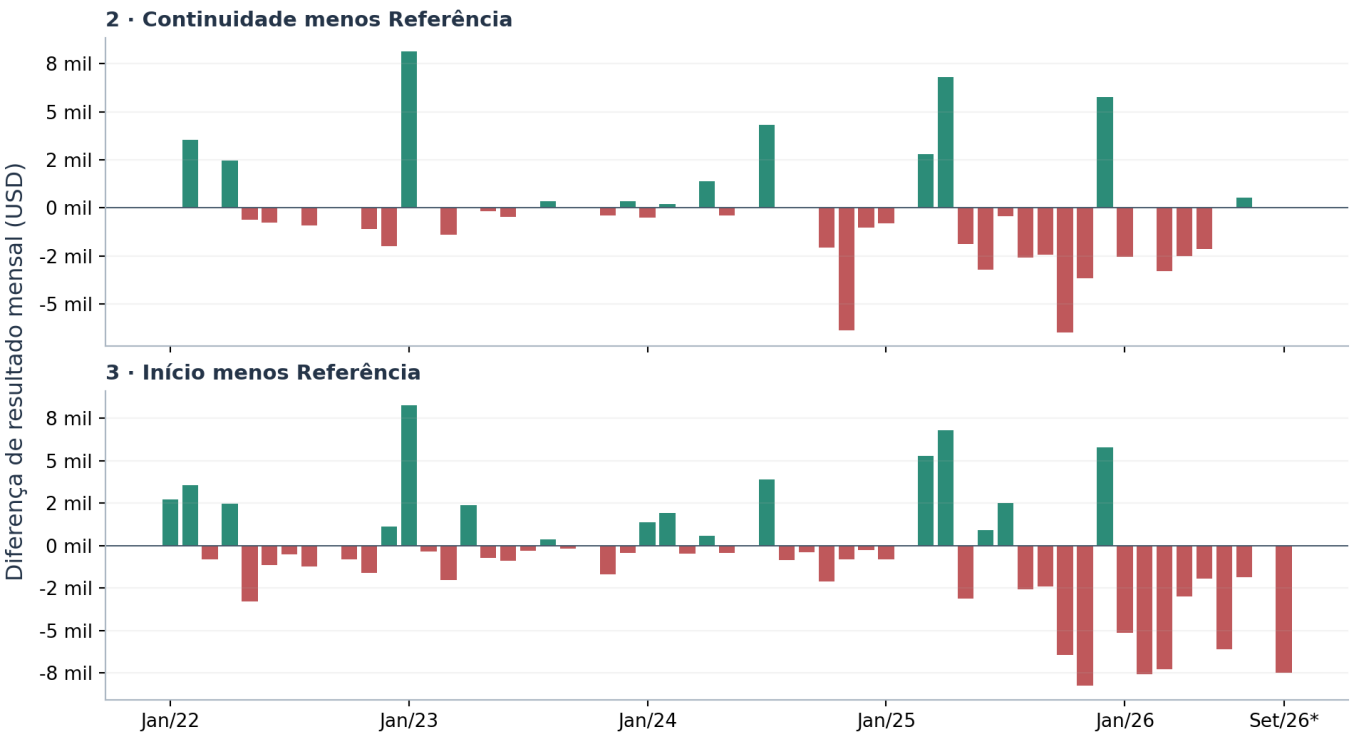
03 · Consistência mês a mês

1 · Referência retorno mensal (%)												
2022	-26,5	22,3	70,7	1,4	-3,3	0,0	28,5	6,3	-7,3	3,3	20,4	2,5
2023	-5,6	-22,2	18,1	-8,4	13,2	-9,8	2,6	3,0	-10,2	56,7	-6,1	16,2
2024	2,5	-8,0	14,6	11,1	4,0	-5,2	5,5	-7,0	18,0	5,8	14,5	0,6
2025	13,3	14,1	5,4	21,4	0,0	2,9	0,0	7,9	18,8	17,8	12,3	-3,9
2026	37,3	-6,9	8,6	-0,7	-2,5	-1,2	2,4	6,5	0,0			
	Jan	Fev	Mar	Abr	Mai	Jun	Jul	Ago	Set*	Out	Nov	Dez
2 · Continuidade retorno mensal (%)												
2022	-26,5	70,5	50,8	14,1	-5,3	-3,7	21,8	1,3	-6,2	2,8	12,4	-5,3
2023	27,2	-16,1	6,9	-6,2	8,9	-9,0	1,9	3,6	-7,6	41,1	-6,1	14,1
2024	0,8	-6,1	11,9	12,7	2,5	-4,3	14,0	-5,4	13,6	0,9	0,6	-1,2
2025	11,2	13,7	9,2	28,9	-1,9	-0,7	-0,4	4,8	15,8	12,1	10,1	-0,1
2026	36,4	-7,1	7,1	-2,0	-3,7	-1,3	2,9	7,0	0,0			
	Jan	Fev	Mar	Abr	Mai	Jun	Jul	Ago	Set*	Out	Nov	Dez
3 · Início retorno mensal (%)												
2022	0,7	51,4	36,4	12,8	-16,2	-6,0	20,3	-0,1	-6,7	-1,0	11,6	7,4
2023	28,1	-17,3	4,6	2,2	6,6	-10,3	0,8	3,6	-8,3	41,6	-9,7	12,4
2024	5,9	-1,8	10,3	10,4	2,3	-4,4	13,2	-7,1	13,2	0,9	10,8	0,1
2025	10,4	12,8	12,0	26,6	-2,9	3,3	2,4	4,2	14,0	10,9	6,0	-0,1
2026	32,8	-10,6	5,0	-2,3	-3,7	-4,7	1,7	7,6	-4,0			
	Jan	Fev	Mar	Abr	Mai	Jun	Jul	Ago	Set*	Out	Nov	Dez

Retorno = (equity_end / equity_start - 1) × 100. Cinza indica mês não observado. * Somente setembro de 2026 é parcial; meses de setembro dos anos anteriores são completos. Cores saturam em ±35%; os números mostram o retorno efetivo.

A ausência de operação não equivale a lucro. A Referência teve dois meses completos zerados; ambas as versões extras tiveram atividade suficiente para eliminar esses zeros, mas não eliminaram os meses negativos.

04 · Onde houve melhora ou piora



Diferença de mudança mensal de equity de cada versão menos a Referência. Barras verdes: resultado maior; vermelhas: menor. A diferença não identifica o lucro individual de um tipo de entrada.

2 · Continuidade

- Meses antes negativos que ficaram positivos: 2023-01.
- Meses antes positivos que ficaram negativos: 2022-12, 2024-12, 2025-06, 2025-07.
- Meses antes zerados que ficaram negativos: 2022-06, 2025-05.

3 · Início

- Meses antes negativos que ficaram positivos: 2022-01, 2023-01, 2023-04.
- Meses antes positivos que ficaram negativos: 2022-08, 2022-10.
- Meses antes zerados que ficaram negativos: 2022-06, 2025-05.

Comparação restrita aos mesmos 56 meses completos. Mudanças de um mês também podem decorrer de posições iniciadas no mês anterior. É necessário analisar ciclos e marcação das posições para explicar cada transição.

05 · Resultados anuais e Selic

Ano	Caso	Lucro USD	Retorno USD	Retorno BRL	Selic BRL
2022	1	14.297,77	142,98%	127,18%	12,39%
2022	2	14.922,19	149,22%	133,02%	12,39%
2022	3	14.614,58	146,15%	130,14%	12,39%
2023	1	7.688,73	31,64%	22,15%	13,04%
2023	2	14.096,05	56,56%	45,27%	13,04%
2023	3	12.040,67	48,92%	38,17%	13,04%
2024	1	21.469,76	67,12%	113,76%	10,88%
2024	2	17.081,81	43,78%	83,90%	10,88%
2024	3	23.825,25	65,00%	111,04%	10,88%
2025	1	94.825,68	177,39%	146,48%	14,32%
2025	2	88.748,82	158,20%	129,43%	14,32%
2025	3	92.445,06	152,85%	124,68%	14,32%
2026*	1	66.630,60	44,94%	34,12%	9,78%
2026*	2	56.675,12	39,13%	28,75%	9,78%
2026*	3	26.190,50	17,13%	8,39%	9,78%

Casos: 1 Referência, 2 Continuidade, 3 Início. * 2026 até 11/09/2026, sem anualizar o retorno parcial.

Retorno anual USD = $\text{patrimônio final} / \text{patrimônio inicial do ano} - 1$. Retorno BRL = $(\text{patrimônio final USD} \times \text{câmbio final}) / (\text{patrimônio inicial USD} \times \text{câmbio inicial}) - 1$. A Selic foi acumulada no mesmo intervalo. A comparação em BRL evita confrontar diretamente moedas diferentes.

Benchmark reutilizado dos arquivos históricos já arquivados: [SGS do Banco Central](#), série 11 (Selic diária) e série 1 (dólar). A origem, datas e taxas estão em [Benchmarks_Anuais.json](#). Não inclui impostos, spread cambial ou tarifas de remessa; tampouco iguala o risco do robô ao da Selic.

06 · Todos os meses: 2022 e 2023

Resultado mensal por mudança de patrimônio. USD e percentual sobre o patrimônio no início de cada mês.

Mês	Caso 1 USD	Caso 1 %	Caso 2 USD	Caso 2 %	Caso 3 USD	Caso 3 %
2022-01	-2.651,76	-26,52%	-2.651,76	-26,52%	74,36	0,74%
2022-02	1.639,75	22,31%	5.180,79	70,50%	5.180,79	51,43%
2022-03	6.358,59	70,75%	6.358,59	50,75%	5.557,23	36,43%
2022-04	216,27	1,41%	2.670,98	14,14%	2.669,69	12,83%
2022-05	-513,80	-3,30%	-1.136,60	-5,27%	-3.815,49	-16,25%
2022-06	0,00	0,00%	-754,29	-3,69%	-1.172,79	-5,96%
2022-07	4.287,42	28,49%	4.287,42	21,80%	3.755,22	20,31%
2022-08	1.227,39	6,35%	312,80	1,31%	-19,70	-0,09%
2022-09	-1.496,09	-7,28%	-1.496,09	-6,16%	-1.496,09	-6,73%
2022-10	627,00	3,29%	627,00	2,75%	-210,50	-1,02%
2022-11	4.009,67	20,36%	2.907,67	12,43%	2.386,97	11,63%
2022-12	593,33	2,50%	-1.384,32	-5,26%	1.704,89	7,44%
2023-01	-1.354,49	-5,57%	6.775,34	27,19%	6.909,35	28,07%
2023-02	-5.100,75	-22,23%	-5.100,75	-16,09%	-5.453,75	-17,30%
2023-03	3.224,71	18,07%	1.827,61	6,87%	1.191,51	4,57%
2023-04	-1.762,69	-8,37%	-1.762,69	-6,20%	611,71	2,24%
2023-05	2.549,41	13,21%	2.365,41	8,87%	1.828,51	6,56%
2023-06	-2.135,66	-9,77%	-2.601,16	-8,96%	-3.047,76	-10,26%
2023-07	506,35	2,57%	506,35	1,92%	206,65	0,78%
2023-08	607,75	3,00%	975,25	3,62%	975,25	3,63%
2023-09	-2.122,12	-10,19%	-2.122,12	-7,60%	-2.309,92	-8,30%
2023-10	10.607,79	56,69%	10.607,79	41,14%	10.607,79	41,56%
2023-11	-1.798,57	-6,13%	-2.205,27	-6,06%	-3.516,46	-9,73%
2023-12	4.467,00	16,23%	4.830,29	14,13%	4.037,79	12,38%

07 · Todos os meses: 2024 e 2025

Resultado mensal por mudança de patrimônio. USD e percentual sobre o patrimônio no início de cada mês.

Mês	Caso 1 USD	Caso 1 %	Caso 2 USD	Caso 2 %	Caso 3 USD	Caso 3 %
2024-01	808,02	2,53%	321,72	0,82%	2.154,92	5,88%
2024-02	-2.617,80	-7,98%	-2.410,80	-6,13%	-685,79	-1,77%
2024-03	4.394,79	14,56%	4.394,79	11,90%	3.920,09	10,28%
2024-04	3.832,39	11,09%	5.236,79	12,67%	4.386,39	10,43%
2024-05	1.535,46	4,00%	1.147,48	2,46%	1.085,39	2,34%
2024-06	-2.072,58	-5,19%	-2.072,58	-4,34%	-2.072,58	-4,36%
2024-07	2.086,34	5,51%	6.404,98	14,04%	5.991,09	13,18%
2024-08	-2.805,67	-7,02%	-2.805,67	-5,39%	-3.667,37	-7,13%
2024-09	6.691,10	18,01%	6.691,10	13,59%	6.307,60	13,20%
2024-10	2.563,71	5,85%	484,42	0,87%	459,91	0,85%
2024-11	6.715,86	14,47%	355,33	0,63%	5.891,54	10,80%
2024-12	338,14	0,64%	-665,75	-1,17%	54,06	0,09%
2025-01	7.087,36	13,26%	6.270,88	11,18%	6.270,88	10,37%
2025-02	8.522,66	14,08%	8.522,66	13,66%	8.522,66	12,77%
2025-03	3.725,86	5,39%	6.544,01	9,23%	9.019,72	11,98%
2025-04	15.583,59	21,41%	22.386,20	28,91%	22.386,20	26,56%
2025-05	0,00	0,00%	-1.874,29	-1,88%	-3.137,38	-2,94%
2025-06	2.535,72	2,87%	-690,07	-0,70%	3.438,02	3,32%
2025-07	33,83	0,04%	-387,27	-0,40%	2.550,86	2,38%
2025-08	7.190,06	7,91%	4.612,59	4,76%	4.612,59	4,21%
2025-09	18.456,40	18,81%	16.036,00	15,80%	16.036,00	14,05%
2025-10	20.713,57	17,77%	14.252,02	12,13%	14.252,02	10,95%
2025-11	16.929,38	12,33%	13.263,44	10,07%	8.680,84	6,01%
2025-12	-5.952,75	-3,86%	-187,35	-0,13%	-187,35	-0,12%

08 · Todos os meses: 2026 parcial

Resultado mensal por mudança de patrimônio. USD e percentual sobre o patrimônio no início de cada mês.

Mês	Caso 1 USD	Caso 1 %	Caso 2 USD	Caso 2 %	Caso 3 USD	Caso 3 %
2026-01	55.237,81	37,25%	52.678,71	36,37%	50.091,01	32,76%
2026-02	-13.956,34	-6,86%	-13.956,34	-7,07%	-21.548,90	-10,61%
2026-03	16.272,21	8,58%	12.979,01	7,07%	8.997,01	4,96%
2026-04	-1.378,72	-0,67%	-3.890,41	-1,98%	-4.392,61	-2,31%
2026-05	-5.026,75	-2,46%	-7.147,34	-3,71%	-6.975,82	-3,75%
2026-06	-2.400,80	-1,20%	-2.400,80	-1,29%	-8.506,60	-4,75%
2026-07	4.690,21	2,38%	5.219,31	2,85%	2.835,72	1,66%
2026-08	13.192,98	6,54%	13.192,98	7,01%	13.192,98	7,61%
2026-09*	0,00	0,00%	0,00	0,00%	-7.502,29	-4,02%

* Setembro termina no último tick recebido em 11/09/2026. Não tratá-lo como mês completo. Os resultados anuais de 2026 e o total do teste incluem esse intervalo.

Saldo realizado e patrimônio

O dossiê mede a consistência usando patrimônio: saldo mais resultado aberto. As tabelas de origem também registram booked_net e cycle_net_by_exit, úteis para explicar fechamentos, mas esses conceitos não devem substituir a mudança de equity na avaliação mensal. Uma transferência de lucro aberto para saldo ao fechar uma operação não é uma perda.

09 · Estratégias efetivamente utilizadas

A estratégia é uma implementação de pesquisa própria com médias, ADX e pivôs. Referências anteriores a Linda Raschke ou Larry Williams não certificam que suas metodologias foram reproduzidas integralmente. O código anexado é a definição operacional.

Componente	Implementação da rodada
Mercado e direção	XAUUSD M30; somente compras; uma posição por vez.
Médias	EMA21; SMA50; SMA200, sobre fechamento.
Força/direção	iADX do MT5, período 14; mínimo 20; $+DI > -DI$. Não é chamada a iADXWilder.
Volatilidade	ATR14; distância mínima do ask à EMA21 de 0,50 ATR.
Oscilador extra	SMA3 – SMA10; crescente em relação à barra anterior. Sinal SMA16 calculado; não é um cruzamento obrigatório. Nenhuma chamada a iMACD.
Stop	Menor mínima do candle fechado de sinal e dos dois anteriores –0,20 ATR. Aceito somente entre 1 e 2,5 ATR da entrada cotada.
Alvo e lote	5R sobre risco inicial cotado; lote fixo 1,00. Sem BE, parcial, médio, pirâmide ou reinvestimento.

Caso 1 · Referência

Mantém as regras originais do Hermes R190:2. O oscilador 3-10 não é um filtro obrigatório desta entrada original. O detalhamento exato das condições de tendência, toque anterior e retomada está na auditoria do código a seguir; as funções executáveis permanecem no anexo.

Casos 2 e 3 · Entrada adicional

Fundo F1, topo T, fundo mais alto F2; $F1 < F2 < T$. Cada extremo exige duas barras à esquerda e duas à direita. O gatilho exige rompimento de T no fechamento, com referência já conhecida antes da abertura do candle de rompimento. Perda de F1 invalida a estrutura, inclusive na vela do rompimento. F1 não substitui o stop estrutural de três candles.

Ambos os extras exigem candle de alta, fechamento acima da EMA21, $EMA21 > SMA50$, SMA50 subindo, $ADX14 \geq 20$, $+DI > -DI$, 3-10 crescente e distância mínima de 0,5 ATR. O caso 2 também exige fechamento acima da SMA200 e SMA200 subindo. O caso 3 retira somente essas duas condições da entrada extra.

10 · Auditoria técnica do código

Auditoria independente do código - Hermes Pivôs 2.00

Escopo: inspeção estática de todos os módulos do fonte e cruzamento com os exports agregados da rodada R200 recebidos em 13/09/2026. É uma revisão técnica do projeto, não certificação externa de rentabilidade. Não houve alteração do EA, compilação nativa, novo backtest ou ajuste de parâmetros nesta auditoria.

Fonte único auditado: Hermes_Pivos_Lab_200/Hermes_Pivos_Lab_200.mq5.

SHA-256: 69a4e02bfaaea2619aad1dea364e8e5c0d03c768ef46f5eeddd2ca7f1235ec9f.

As referências de linhas abaixo usam os módulos de src, que compõem o arquivo único. O mapa de hashes acompanha o JSON desta auditoria.

Parecer

Não identifiquei erro crítico demonstrado de antecipação de preços futuros no detector ativo de pivôs. O encadeamento de confirmação e a separação entre gatilho fechado e entrada na vela seguinte são coerentes com o protocolo. Isso não prova ausência de qualquer bug: a evidência disponível aqui é a leitura do código, os testes portáteis previamente registrados e os exports do usuário. O executável EX5 efetivamente utilizado não foi recebido com hash.

Os três casos declaram `valid_run=1`, reconciliações zeradas e nenhum erro interno de integridade. O controle reproduziu US\$ 204.912,54 e 261 operações. As variantes com entradas extras aumentaram operações, mas não melhoraram a consistência mensal: nos 56 meses completos (janeiro/2022 a agosto/2026), foram 17, 22 e 18 meses negativos nos casos 1, 2 e 3. Setembro/2026 é parcial e acrescenta um mês negativo observado ao caso 3; por isso o CSV registra 19 nesse caso. O caso 3 teve menor drawdown relativo, porém menor lucro; nenhum caso venceu todos os critérios.

Especificação exata das estratégias

Convenção: [1] é a última vela fechada de M30; [2], a anterior; [6], cinco barras observadas antes de [1]. Comparações de média/preço são estritas, salvo o limiar $ADX \geq 20$ e a distância mínima com tolerância numérica. Não é necessário $SMA50 > SMA200$.

Componente	Implementação real	Utilização
EMA21	iMA, 21, MODE_EMA, PRICE_CLOSE	Tendência, retomada e distância
SMA50 e SMA200	iMA, 50/200, MODE_SMA, PRICE_CLOSE	Tendência e inclinação [1] > [6]
ADX14	iADX, buffers 0/1/2	Força ≥ 20 e direção +DI > -DI
ATR14	iATR, 14	Distância, stop e normalização
Oscilador 3-10	SMA3(Close) – SMA10(Close)	Só filtra a entrada extra; valor atual > anterior
Linha de sinal	SMA16 do oscilador 3-10	Calculada e exportada; não exigida na entrada R200
Pivô ativo	Extremo estrito, duas barras de cada lado	Sequência causal F1-T-F2 e rompimento de T

O código usa `iADX`, não `iADXWilder`; a MetaQuotes documenta essas funções separadamente. Uma reprodução deve respeitar a implementação escolhida, sem trocar automaticamente o suavizador. [Documentação iADX](#), [documentação iADXWilder](#).

A associação histórica do projeto com Linda Raschke não autentica este robô como reprodução de uma estratégia original dela. O fonte comprova a fórmula SMA3 – SMA10, sinal SMA16 e a regra de crescimento. Não há chamada a iMACD, requisito de cruzar zero, requisito de cruzar o sinal ou inferência de IA durante a execução. São regras determinísticas.

Caso 1 - HERMES_REFERENCIA

Compra quando todas as condições forem verdadeiras:

1. EMA21[1] > SMA50[1] e SMA50[1] > SMA50[6].
2. Close[1] > SMA200[1] e SMA200[1] > SMA200[6].
3. ADX14[1] ≥ 20 e +DI[1] > -DI[1]. Não exige ADX subindo.
4. Pelo menos uma das velas [2], [3] ou [4] toca sua própria EMA21: $Low \leq EMA \leq High$. A vela gatilho [1] não satisfaz sozinha esse toque anterior.
5. Close[1] > Open[1], Close[1] > EMA21[1] e Close[1] > High[2].
6. Ask no instante da decisão – EMA21[1] ≥ 0,5 ATR14[1], no preset atual.

Em seguida, passam os filtros de posição, spread se habilitado, stop, volume e margem. O oscilador não limita este caso. Evidências: XAU_H1_Core.mqh:H1Evaluate/H1EvaluateProfile, EntryCore.mqh:EV0Evaluate e EA.mq5:ReadSignal.

Caso 2 - HERMES_PIVO_CONTINUIDADE

Mantém o caso 1 e acrescenta outra rota de compra, considerada somente quando o filtro original EV0Evaluate falha. A nova rota exige:

- Sequência confirmada F1-T-F2 com $F1 < F2 < T$; fechamento anterior ≤ T e fechamento do gatilho > T.
- EMA21[1] > SMA50[1], SMA50[1] > SMA50[6], Close[1] > SMA200[1] e SMA200[1] > SMA200[6].
- ADX14[1] ≥ 20 e +DI[1] > -DI[1].
- Candle altista fechado acima da EMA21; SMA3 – SMA10 maior que no candle anterior.
- Ask – EMA21[1] ≥ 0,5 ATR14[1].

A rota extra não exige o toque anterior na EMA nem o rompimento específico de High[2]; exige o rompimento do topo T do pivô.

Caso 3 - HERMES_PIVO_INICIO

É idêntico ao caso 2, retirando **somente da rota extra** as duas condições de SMA200: preço acima dela e inclinação ascendente. Todas as regras da rota original permanecem. A tendência local EMA21/SMA50 continua obrigatória; não é um detector irrestrito de início de qualquer tendência.

Causalidade e estado do pivô

O extremo de uma vela só fica disponível na abertura observada posterior ao fechamento da segunda vela à direita. O rompimento usa referências disponíveis antes da abertura da vela gatilho. O detector avalia o sinal antes de incorporar o novo pivô confirmado naquela iteração. Empates são excluídos; uma vela que simultaneamente é topo e fundo estritos reinicia a sequência. Pivôs consecutivos do mesmo tipo só substituem o anterior se forem mais extremos.

O padrão comprador é invalidado se uma vela posterior atingir $Low \leq F1$, inclusive a própria vela de rompimento. Cada snapshot de padrão produz no máximo um evento. O núcleo também conta estruturas vendedoras para referência, mas só emite signal.buy para operar. O estado é atualizado mesmo com posição aberta; evento bloqueado não fica na fila para entrada posterior.

O adaptador inicia com 210 velas fechadas em ordem cronológica. Em falha de cópia, não grava estado parcial. Em recuperação, processa o histórico observado e descarta gatilhos antigos; só o mais recente pode operar. A hora de abertura seguinte é testemunho do fechamento; o detector não usa OHLC da vela aberta. A mesma origem histórica é necessária para comparação exata perto da inicialização.

Execução e saídas comuns

Há uma decisão por nova vela M30 e no máximo uma posição/campanha do símbolo. A compra a mercado usa o ask; a trajetória é marcada pelo bid. A prioridade da rota original é resolvida antes da execução: se ela passar no setup e depois falhar no stop, margem ou spread, não há fallback para o pivô.

SL comprador = $\min(\text{Low}[1], \text{Low}[2], \text{Low}[3]) - 0,20 \text{ ATR}$, arredondado para o tick para fora. A distância do ask cotado ao SL precisa ficar entre 1 e 2,5 ATR; caso contrário a entrada é recusada. O fundo geométrico F1 invalida o padrão, mas não é o stop executado. TP nominal = $\text{ask cotado} + 5 \times (\text{ask cotado} - \text{SL})$, com arredondamento de tick. O volume é exatamente 1,00 lote no preset, ou a operação é recusada se volume/margem nativa não permitirem; não há redução automática.

SL/TP acompanham a ordem. O fill é conferido por identificador da posição, deal, volume, símbolo, magic e direção. Uma resposta ambígua bloqueia novas entradas em vez de reenviá-las cegamente. A proteção existente é monitorada, com parada ou fechamento de emergência em inconsistências. O PnL Líquido reconstrói $\text{DEAL_PROFIT} + \text{DEAL_SWAP} + \text{DEAL_COMMISSION} + \text{DEAL_FEE}$ dos ciclos próprios.

Não há BE, parcial, médio, piramidação, reinvestimento, saída por inversão de indicador ou tempo máximo nesta rodada. Normalmente a saída é SL, TP ou encerramento do testador; inconsistências técnicas podem acionar emergência. Não há API, chave de IA, banco online ou aprendizagem automática dentro deste EA.

Código legado presente, porém sem ativação nos três perfis

HPSelectProfile herda DC caso 2 → EVO caso 28 → família 15 e verifica: capital fixo, alvo 5R, channelMode 0, entryPolicy 0, sem parcial/BE/adicionões/reinvestimento. Nomes como Donchian, W1, 20 pontos, 20 unidades de preço, médios e outras famílias aparecem nos módulos/testes porque o motor é compartilhado com releases anteriores. Sua presença textual não significa uso no R200.

PEPivotSignal continua gerando métricas legadas. A decisão extra R200 usa HPStep/HPEXtraGate. pivot_buy_candidates e hp_candidates não são aliases. O robô bloqueia uso fora do Testador, inclusive em conta DEMO online: OnInit exige MQL_TESTER.

Achados e recomendações específicas

Os itens distinguem comportamento intencional, limitação de evidência e melhoria possível; não são 12 bugs comprovados. A severidade expressa impacto para interpretação ou reprodução do projeto.

A01 - O fonte está identificado; o executável efetivamente usado não está vinculado por hash.

ALTA · LIMITE_DE_PROVENIENCIA

Os exports identificam nomes/casos e o parameters.txt registra o SHA do controle antigo. Não exportam SHA do fonte atual, SHA do EX5, build do MetaEditor/terminal ou configuração completa do teste. A reprodução agregada do controle é evidência de consistência, não autenticação criptográfica do binário.

Impacto: Outra compilação ou histórico/configuração diferente pode conservar o mesmo nome de arquivo; o resultado não fica totalmente reproduzível apenas com os quatro CSV.

Proposta: Na próxima release registrar um identificador de build imutável no EA e guardar externamente hashes do MQ5, EX5, preset e relatório nativo junto dos resultados. Não reescrever a proveniência desta rodada como se já existisse.

Evidência: src/EA.mq5 · WriteParameters · linha 547; src/EA.mq5 · WriteParameters · linha 557; src/Reports.mqh · StatNames · linha 5.

A02 - valid_run não atesta custos, ticks ou qualidade de execução real.

ALTA · LIMITE_DE_VALIDACAO

A validade exige reconciliação financeira e ausência de erros internos. Os resultados recebidos indicam comissão/taxas zero, swap negativo e ausência de filtro extra de spread pelo valor padrão 0. Não há nos quatro CSV a configuração nativa de modelagem e latência.

Impacto: Reconciliação aritmética correta não demonstra que todos os custos de uma conta futura estejam representados. A contagem de observações de preço não prova modelagem por ticks reais.

Proposta: Anexar relatório nativo e especificação do contrato/conta e repetir depois, em amostra não usada no ajuste, com custos e latência explicitamente registrados. Não subtrair spread novamente dos resultados: as ordens já usam ask/bid.

Evidência: src/Reports.mqh · OnTester · linha 197; src/EA.mq5 · OpenCycle · linha 449; src/EA.mq5 · OnTick · linha 534.

A03 - Lote exato não é limite percentual de risco ou margem.

MEDIA · DESENHO_FINANCEIRO_CONFIRMADO

Nos três casos, InpFixedLot é utilizado diretamente. InpMaxMarginPct e InpMaxLot permanecem na interface, mas não limitam estas entradas. Só volume mínimo/máximo/passo e margem livre nativa limitam o lote. O risco nominal é medido pelo stop, sem orçamento percentual prévio.

Impacto: O risco por operação varia com ATR e saldo. Os CSV registram máximos de risco de stop sobre saldo de 14,6184%, 16,9772%, 12,2083% nos casos 1/2/3. O campo de margem 20 não significa limite de 20% ativo.

Proposta: Documentar claramente os inputs inativos e manter lote 1 nesta comparação. Qualquer versão com orçamento percentual precisa receber novo caso e não ser apresentada como simples correção do mesmo resultado.

Evidência: src/EA.mq5 · inputs · linha 15; src/EA.mq5 · OpenCycle · linha 408; src/EA.mq5 · OpenCycle · linha 441; src/HermesCore.mqh · HermesExactLot · linha 26.

A04 - Uma falha ou rejeição pode consumir a oportunidade daquela vela.

MEDIA · LIMITACAO_OPERACIONAL_OBSERVAVEL

lastBar é avançado antes de ler indicadores e enviar a entrada. O EA não tenta novamente na mesma vela após G_NO_DATA, spread, stop ou rejeição de ordem. A rodada tem 0 G_NO_DATA e 8/11/13 G_REJECTED; o último retcode de cada caso é 10018, mercado fechado. Não se pode inferir que todas as rejeições tiveram essa causa sem events.csv.

Impacto: Parte das oportunidades perdidas pode ser execução/horário, não padrão ruim. Nenhuma tentativa atrasada foi introduzida nesta análise.

Proposta: Analisar cada ENTRY_REJECTED por horário e retorno usando events.csv. Se houver justificativa, testar uma janela limitada e idempotente de retentativa apenas para rejeições definitivas e snapshot ainda válido, em caso separado.

Evidência: src/EA.mq5 · OnTick · linha 501; src/EA.mq5 · OnTick · linha 514; src/EA.mq5 · OpenCycle · linha 451.

A05 - Os casos com pivô substituem parte das entradas originais.

MEDIA · EFEITO_DE_CARTEIRA_CONFIRMADO

O original tem prioridade quando ambos passam na mesma vela, mas uma posição extra aberta antes impede entradas futuras. Pelos agregados, as entradas BASE são 261/241/231 e as PIVOT 0/58/111. Os sinais originais selecionados seguem 1070 em todos. As reduções 20/30 de BASE são diferenças de contagem, não identificação individual dos trades substituídos.

Impacto: A diferença de lucro entre casos não é o lucro isolado dos pivôs. Pode incluir pivôs, originais perdidos/recuperados e mudanças na trajetória de saldo.

Proposta: Conciliar cycles.csv de cada caso por signal_time e selected_entry_path para separar BASE comum, BASE exclusivo e PIVOT. Medir contribuição e custo de ocupação antes de sugerir filtros.

Evidência: src/PivotEntryCore.mqh · HPRouteEntry · linha 46; src/EA.mq5 · OnTick · linha 532; src/EA.mq5 · SymbolExposure · linha 199.

A06 - Candidatos elegíveis são medidos antes de stop, posição e envio.

BAIXA · AMBIGUIDADE_DE_INSTRUMENTACAO

hp_extra_eligible conta o filtro técnico extra antes de validar o stop. hp_pivot_selected exclui o original vencedor da prioridade; hp_pivot_fills conta fills. Os antigos pivot_buy_candidates/pivot_sell_candidates usam outro detector legado PEPivotSignal e não descrevem o gatilho ativo HPStep.

Impacto: 234/320 elegíveis não podem ser comparados diretamente aos 175/242 do estudo anterior, que incluía stop. Tampouco as colunas legadas devem ser somadas aos 761 candidatos ativos.

Proposta: No relatório usar funil explícito: geométrico → extra técnico → original não selecionado → disponível → stop/margem/execução → preenchido. Prefixar documentação do detector antigo como diagnóstico legado.

Evidência: src/EA.mq5 · OnTick · linha 522; src/EA.mq5 · Record · linha 269; src/Reports.mqh · StatNames · linha 5.

A07 - Entrada Início ainda exige tendência local; não busca qualquer reversão.

MEDIA · LIMITE_DA_HIPOTESE

Caso 3 retira só preço > SMA200 e SMA200 subindo da entrada extra. Continua exigindo EMA21 > SMA50, SMA50 subindo, $ADX \geq 20$, $+DI > -DI$, candle altista acima da EMA, oscilador crescente e $ask \geq EMA + 0,5 ATR$. Usa stop das 3 velas recentes, não o fundo F1 do pivô. Não exige distância máxima da EMA além das restrições indiretas do stop.

Impacto: Pivôs precoces contra a tendência local continuarão bloqueados. O rótulo Início pode sugerir abrangência maior que a efetiva. Geometria e stop são hipóteses distintas.

Proposta: Pedir ao revisor análise da coerência entre geometria, atraso do gatilho e stop, preservando uma única alteração por futura hipótese. Nenhum relaxamento foi feito nesta rodada.

Evidência: src/PivotEntryCore.mqh · HPExtraGate · linha 31; src/ProtectCore.mqh · PEDistance · linha 39; src/EA.mq5 · OpenCycle · linha 384.

A08 - A origem do histórico e a ausência de expiração temporal afetam o catálogo.

BAIXA · LIMITACAO_DE_REPRODUCAO

O detector inicia com 210 velas fechadas e processa continuamente. Não existe expiração fixa de um padrão por número de velas: ele permanece até invalidação, rompimento ou substituição estrutural. Referências na borda de inicialização podem variar em relação a replay iniciado em outra data.

Impacto: Replays com origem diferente podem divergir perto do começo. Candidatos antigos não são automaticamente candidatos melhores ou piores; a idade precisa ser medida.

Proposta: Registrar origem e idade dos três pivôs em qualquer reprodução; avaliar idade como hipótese prospectiva somente depois de medir distribuição, sem escolher retrospectivamente a idade que elimina perdas.

Evidência: src/PivotRuntime.mqh · HPSyncClosedBars · linha 13; src/PivotCore.mqh · HPExposePivot · linha 155; src/PivotCore.mqh · HPStep · linha 195.

A09 - 5R é alvo nominal sobre a cotação anterior ao fill, não ganho líquido garantido.

BAIXA · LIMITE_DE_METRICA

SL/TP são enviados juntos: stop estrutural arredondado e TP arredondado para fora em torno de 5 distâncias do entry cotado. O fill pode diferir. A correção de alvo pós-fill só executa em targetMode > 0; todos R200 usam 0. Custos e gaps também alteram R realizado.

Impacto: O lucro líquido de um vencedor não precisa ser exatamente 5 vezes o risco inicial do fill; pequenas diferenças por tick são esperadas. Não chamar isso automaticamente de stop ou alvo adulterado.

Proposta: Conservar no dossiê `quoted_entry`, `actual_entry`, `quoted_target` e `actual_target_R`; analisar distribuição quando os `cycles` nativos dos casos novos forem anexados.

Evidência: `src/XAU_H1_Core.mqh` · `H1Levels` · linha 60; `src/EA.mq5` · `OpenCycle` · linha 467; `src/Execution.mqh` · `EnforceTargetCap` · linha 50; `src/Reports.mqh` · `ExportDetails` · linha 87.

A10 - O período foi usado repetidamente para escolher hipóteses.

MEDIA · LIMITACAO_DE_VALIDACAO_TEMPORAL

A própria declaração da rodada identifica 2022-2026 como janela já examinada. Esta auditoria recebeu comparação agregada, não um novo período intocado. Mais lucro nessa janela, se surgisse, não provaria repetibilidade; nesta rodada as extras pioraram lucro e meses negativos.

Impacto: Selecionar somente vencedores históricos e eliminar padrões perdedores aumenta risco de ajustar ruído e regime específico. A auditoria não certifica robô pronto para conta real nem capacidade de zerar meses negativos.

Proposta: Manter a versão Referência como controle; pré-declarar poucas hipóteses motivadas por decomposição dos trades e medir em janelas cronológicas separadas e posteriormente forward DEMO.

Evidência: `PLANO_RODADA_200.json` · `dates_note` · linha 15.

A11 - DD mensal reinicia o pico no mês e não equivale ao DD global.

BAIXA · LIMITE_DE_METRICA_MENSAL

Cada mês começa na última equity observada do anterior e reinicia o pico para o DD intramês. O DD global relativo vem de `TesterStatistics`. `equity_change`, `booked_net` e `cycle_net_by_exit` são conceitos diferentes.

Impacto: Somar apenas saídas desloca lucro entre meses de posições carregadas. Máximo de DDs mensais não reproduz necessariamente DD global; percentual no maior DD monetário também é outra estatística.

Proposta: Apresentar consistência por `equity_change`, com saldo realizado como coluna auxiliar. Gráfico de fechamentos mensais deve ser rotulado para não sugerir trajetória intramês ou DD por ticks.

Evidência: `src/ProtectCore.mqh` · `PEStartEquity` · linha 129; `src/EA.mq5` · `TrackEquity` · linha 211; `src/Reports.mqh` · `MonthValues` · linha 11; `src/Reports.mqh` · `OnTester` · linha 151.

A12 - Flags de breakeven não constituem simulação contrafactual completa.

BAIXA · LIMITACAO_DE_METRICA_BE

Os thresholds 1/1,5/2/2,5/3R guardam atingiu/retornou e primeira cotação de retorno; não guardam o instante do retorno. O agregado de otimização publica esses flags somente para o caso 1. Nenhum BE está ativado.

Impacto: Não é possível deduzir PnL mensal, swap recalculado ou novas entradas liberadas por um BE apenas desse CSV. MFE após entrada não pode ser filtro disponível antes de entrar.

Proposta: Solicitar ao Claude distinguir flags observados, proxies e backtests de gestão reais. Qualquer BE proposto requer execução temporal completa em caso próprio.

Evidência: `src/ProtectCore.mqh` · `PEObserve` · linha 110; `src/Reports.mqh` · `ExportDetails` · linha 97; `src/Reports.mqh` · `DrainFrames` · linha 261.

O retorno 10018 significa mercado fechado segundo a [tabela oficial da MetaQuotes](#). Modo, dados e condições selecionados são parte do experimento, conforme a [documentação de testes](#).

Leitura correta dos relatórios

- `profit` é a estatística financeira do testador; `cycle_net` reconstrói negócios; `equity_change` mede a variação patrimonial entre fechamentos de mês. A soma mensal é confrontada com o lucro total, com tolerância de US\$ 0,01.

- booked_net registra realizado no mês de cada deal. cycle_net_by_exit atribui todo o resultado do ciclo ao mês da saída. Podem divergir da equity mensal quando a posição atravessa meses.
- O DD relativo global vem do MT5. O percentual no ponto do maior DD monetário é outra estatística; o DD mensal reinicia o pico a cada mês.
- O código não cria meses sem nenhum tick observado. A cobertura do calendário precisa ser verificada separadamente; período parcial não deve ser apresentado como ano/mês completo.
- Taxas iguais a zero refletem o teste fornecido. O spread já está nas cotações de entrada/saída; acrescentá-lo integralmente outra vez duplicaria esse custo.
- O BE shadow é observacional. O arquivo agregado publica thresholds somente para o caso 1; não são resultados financeiros de uma nova gestão com BE.

Perguntas prioritárias para o Claude

1. A inspeção confirma os filtros ativos descritos? Identifique divergências com função/linha e um contraexemplo reproduzível, se existirem.
2. Como separar o efeito das 58/111 entradas de pivô do efeito das 20/30 entradas BASE a menos? Exigir decomposição por timestamp e rota antes de culpar o padrão.
3. O stop das três velas recentes é consistente com a geometria F1-T-F2? Qual hipótese única e pré-declarada investigaria isso, sem afastar stops retrospectivamente para salvar perdas conhecidas?
4. $\text{Ask} \geq \text{EMA} + 0,5 \text{ ATR}$ ajuda a confirmação ou atrasa o gatilho? Propor comparação causal com validação cronológica; não usar MFE ou resultado futuro como entrada.
5. Há custo de ocupação excessivo de posições com 5R? Quais dados faltam para testar isso, respeitando a preferência por evitar parciais em alvos curtos?
6. Existem regimes identificáveis antes da entrada e estáveis em subperíodos, ou o padrão foi selecionado apenas porque ganhou neste histórico já visto?
7. A melhora de DD do caso 3 justifica menor lucro e mais meses negativos segundo um objetivo definido antes de olhar os resultados?
8. Quais testes mínimos distinguem correção do código, reprodução do backtest e vantagem da estratégia em dados novos e DEMO?

Solicitar hipóteses falsificáveis, evidência contrária e uma ordem curta de testes. Não pedir promessa de eliminar todo mês negativo. Não ativar novos casos, alterar a versão atual nem contar uma proposta como melhoria confirmada.

11 · Escopo e limites da auditoria de resultados

Recebidos quatro arquivos agregados, com três casos e 171 registros mensais. A conciliação cobre totais, continuidade de patrimônio/saldo, contagem de operações e meses, identificadores, flags de integridade e reprodução da referência. Os registros individuais R200 de barras, ciclos, deals e eventos ainda não foram enviados.

Foram aprovadas 2.199 verificações de conciliação. A Referência reproduziu os 124 campos comuns econômicos e operacionais comparados com R190:2 e os 1.026 campos mensais; as cinco diferenças restantes são identificadores e flags de controle.

Os arquivos de detalhes da antiga referência R190:2 existem no banco e apoiam o estudo anterior. Eles não foram copiados para simular recebimento de detalhes da rodada R200. A auditoria agregada não identifica a sequência exata dos trades extras nem verifica individualmente seus preços de execução.

O código foi submetido a oito testes locais de lógica e revisão estática na entrega. Agora existem resultados nativos exportados pelo usuário, mas não recebemos relatório de compilação, executável EX5 ou relatório nativo de qualidade dos ticks. Não foi executado um novo backtest financeiro aqui.

Os cabeçalhos reportam `valid_run=1` e ausência de falhas de sincronização de pivôs. Mesmo assim, existem 8/11/13 rejeições de ordens nos casos 1/2/3; o último retcode reportado é 10018. Isso deve ser interpretado com os eventos e horários antes de atribuir oportunidades perdidas a um filtro de estratégia.

Os paths de breakeven desta comparação pertencem somente ao caso 1. Eles observam a trajetória sem ativar breakeven. Não constituem resultados financeiros de cinco variantes de BE.

Estado da meta

Nenhum caso atingiu todos os meses positivos. O período exportado contém 56 meses completos, não cinco anos completos. Alterar regras para melhorar meses conhecidos continua sendo pesquisa na amostra; confirmação em dados não usados na escolha segue pendente.

Próximo insumo útil

Os três diretórios de detalhes estão identificados em `arquivos_comparacao(3).csv`. Enviar as pastas completas permite separar PIVOT e BASE, reconhecer substituições de entrada e estudar ganhadores/perdedores com contexto conhecido antes da decisão.

Hermes_Pivos_Lab_200\R200_01\CASO_1_M30_2022.01.01_00-00-00_385

Hermes_Pivos_Lab_200\R200_01\CASO_2_M30_2022.01.01_00-00-00_882

Hermes_Pivos_Lab_200\R200_01\CASO_3_M30_2022.01.01_00-00-00_619

12 · Roteiro para revisão pelo Claude

Você é o revisor independente do projeto Hermes. Analise o dossiê, o fonte MQL5 da versão 2.00, a auditoria e os quatro CSVs originais. Não execute ordens nem altere o fonte de referência antes de apresentar seu parecer.

Objetivo do proprietário: aumentar lucro com maior consistência mensal. A meta de nenhum mês negativo é aspiracional e não foi atingida. A amostra foi reutilizada diversas vezes; não a trate como validação inédita.

Confirme primeiro os três casos: Referência, Pivô Continuidade e Pivô Início. Todos são XAUUSD M30, BUY-only, lote fixo 1,00, alvo nominal 5R e stop estrutural. As únicas mudanças planejadas são nas entradas extras. Não substituir iADX por iADXWilder nem SMA3—SMA10 por MACD EMA sem tratar como mudança de estratégia.

1. Recalcule totais e meses completos. Separe setembro/2026 parcial e equity de saldo realizado. Compare retorno anual em moedas iguais com Selic. 2. Audite causalidade: confirmação dos pivôs, disponibilidade temporal, sequência do rompimento, leitura de candles fechados, consumo de sinais, stop e priorização BASE/PIVOT. Cite função e trecho para cada falha concreta. 3. Explique o que podemos concluir com os agregados e o que exige cycles/deals/events/bars individuais. Não atribua a diferença total de lucro aos pivôs isoladamente: a ocupação alterou as entradas BASE. 4. Avalie custos, rejeições, lote, margem e diferenças entre DD relativo e DD máximo em dinheiro. Não interprete redução percentual isolada como redução de todo risco. 5. Proponha no máximo três hipóteses novas e diferentes, com mecanismo, regras exatas, informação disponível no momento da entrada, previsão falsificável e critério de rejeição. Evite grandes grades de otimização e filtros escolhidos só por coincidirem com meses negativos conhecidos. 6. Priorize teste isolado de cada alteração, comparador preservado e validação posterior não usada na escolha. Qualquer análise de padrões deve separar características anteriores à entrada de MFE/MAE e demais resultados conhecidos depois.

Entregue: parecer objetivo; falhas reproduzíveis versus hipóteses; tabela de evidência/impacto; dados faltantes estritamente necessários; três experiências priorizadas. Não invente backtest, execução nativa, lucro, causalidade ou garantia de rentabilidade.

13 · Fontes, versões e rastreabilidade

Fonte entregue: Hermes_Pivos_Lab_200.mq5. SHA256:

69a4e02bfaaea2619aad1dea364e8e5c0d03c768ef46f5eeddd2ca7f1235ec9f

O código abaixo é uma cópia integral desse fonte, com numeração e quebra visual de linhas somente no PDF. Para compilação ou diff, usar o arquivo .mq5 original incluído no pacote. O documento Markdown conserva o código em bloco sem numeração nem quebra inserida.

comparacao(7).csv · SHA256 75a650459ad237b6603bfc66cfc1807e532359a48adb7bb9d1c2f2f798eea68d

meses_comparacao(5).csv · SHA256 c8ff4ffa3d51308f6d68abaf0e3fe7db4a981e9b2f15b1d651f729fa645033fc

arquivos_comparacao(3).csv · SHA256 5c7f1e2cd47b842433016ca51ac646aa25dc02f69e38ec641cef7becc49001e1

breakeven_comparacao(4).csv · SHA256 5559b943efda8dbc839cdc24e605529f9cd29960f250f78879f738f2d856db6e

Fontes técnicas oficiais: [CopyRates](#), [relatório de teste MT5](#), [otimização e forward](#). CopyRates define o índice zero como a barra atual; a ordem física retornada precisa ser respeitada. O relatório MT5 é a fonte necessária para conferir modelagem e qualidade de histórico.

O banco registra a rodada R200 como executada a partir dos exports recebidos. Hipóteses, código, resultados e documentação ficam separados; resultados anteriores não são apagados. Os CSVs originais são preservados no pacote de revisão.

Leitura recomendada para Claude: Dossie_Hermes_R200.md, Pedido_Revisao_Claude.txt, Hermes_Pivos_Lab_200.mq5 e dados originais. As estratégias e limitações devem ser avaliadas antes de sugerir novos parâmetros.

Anexo · Código-fonte completo

Hermes_Pivos_Lab_200.mq5 · versão 2.00 · 2075 linhas originais.

Linhas longas são quebradas visualmente. O prefixo numérico identifica a linha original; trechos de continuação têm prefixo vazio. Esta formatação não modifica o arquivo .mq5 fornecido.

```

0001 | #property strict
0002 | #property version "2.00"
0003 | #property description "XAU M30: original Hermes control plus two causal 1-2-3 pivot hypotheses. Tester
    | only."
0004 | #include <Trade/Trade.mqh>
0005 | #ifndef XAU_H1_CORE
0006 | #define XAU_H1_CORE
0007 | // Shared, platform-independent decision rules; shift 1 is always CLOSED.
0008 | enum H1_VARIANT { ALIGNMENT_A=0, PRICE_FILTER_B=1 };
0009 | enum H1_SIZE { FIXED_LOT=0, PERCENT_RISK=1 };
0010 | enum H1_PROFILE { BASE_A=0, BASE_B=1, B_PRIOR_PULLBACK=2, B_PRIOR_ADX_RISING=3 };
0011 | enum H1_GATE { READY=0, NO_TREND, ADX_LOW, DI_CONFLICT, NO_TOUCH, NO_TRIGGER };
0012 | struct H1Signal
0013 | {
0014 |     double open,close,previousHigh,previousLow;
0015 |     double ema,sma50,sma200,oldSma50,adx,plusDI,minusDI,atr;
0016 |     double lowest,highest;
0017 |     bool touched;
0018 |     double high,low,oldADX;
0019 |     bool priorTouched;
0020 | };
0021 |
0022 | H1_GATE H1Evaluate(const H1Signal &s,const H1_VARIANT variant,
0023 |                   const bool useADX,const double minADX,int &side)
0024 | {
0025 |     side=0;
0026 |     bool buy=(s.ema>s.sma50 && s.sma50>s.oldSma50);
0027 |     bool sell=(s.ema<s.sma50 && s.sma50<s.oldSma50);
0028 |     if(variant==ALIGNMENT_A)
0029 |     { buy=buy && s.sma50>s.sma200; sell=sell && s.sma50<s.sma200; }
0030 |     else
0031 |     { buy=buy && s.close>s.sma200; sell=sell && s.close<s.sma200; }
0032 |     if(!buy && !sell) return NO_TREND;
0033 |     if(useADX && s.adx<minADX) return ADX_LOW;
0034 |     if(useADX && ((buy && s.plusDI<=s.minusDI) || (sell && s.minusDI<=s.plusDI)))
0035 |         return DI_CONFLICT;
0036 |     if(!s.touched) return NO_TOUCH;
0037 |     if(buy && s.close>s.open && s.close>s.ema && s.close>s.previousHigh) side=1;
0038 |     if(sell && s.close<s.open && s.close<s.ema && s.close<s.previousLow) side=-1;
0039 |     return side==0 ? NO_TRIGGER : READY;
0040 | }
0041 |
0042 | H1_VARIANT ProfileVariant(const H1_PROFILE profile)
0043 | { return profile==BASE_A ? ALIGNMENT_A : PRICE_FILTER_B; }
0044 |
0045 | int H1EvaluateProfile(const H1Signal &s,const H1_PROFILE profile,
0046 |                      const bool useADX,const double minADX,int &side)
0047 | {
0048 |     H1Signal selected=s;
0049 |     if(profile==B_PRIOR_PULLBACK || profile==B_PRIOR_ADX_RISING)
0050 |         selected.touched=s.priorTouched;
0051 |     int gate=(int)H1Evaluate(selected,ProfileVariant(profile),useADX,minADX,side);
0052 |     if(gate!=0) return gate;
0053 |     if(profile==B_PRIOR_ADX_RISING && useADX && s.adx<=s.oldADX)
0054 |     { side=0; return 15; }
0055 |     return 0;
0056 | }
0057 |

```

```

0058 | bool H1Levels(const int side,const double entry,const H1Signal &s,
0059 |             const double bufferATR,const double minATR,const double maxATR,
0060 |             const double reward,const double tick,double &sl,double &tp)
0061 | {
0062 |     sl=0; tp=0;
0063 |     if((side!=1 && side!=-1) || s.atr<=0 || tick<=0 || entry<=0) return false;
0064 |     double raw=(side==1 ? s.lowest-bufferATR*s.atr : s.highest+bufferATR*s.atr);
0065 |     sl=(side==1 ? MathFloor(raw/tick+1e-9) : MathCeil(raw/tick-1e-9))*tick;
0066 |     double distance=side*(entry-sl);
0067 |     if(sl<=0 || distance<=0 || distance<s.atr*minATR-1e-8 || distance>s.atr*maxATR+1e-8)
0068 |         return false;
0069 |     raw=entry+side*reward*distance;
0070 |     tp=(side==1 ? MathCeil(raw/tick-1e-9) : MathFloor(raw/tick+1e-9))*tick;
0071 |     return tp>0;
0072 | }
0073 |
0074 | double H1Volume(const H1_SIZE mode,const double fixedLot,const double budget,
0075 |               const double lossPerLot,const double minimum,const double maximum,const double step)
0076 | {
0077 |     if(lossPerLot<=0 || step<=0 || minimum<=0 || maximum<minimum) return 0;
0078 |     if(mode==FIXED_LOT)
0079 |     {
0080 |         if(fixedLot<minimum-1e-9 || fixedLot>maximum+1e-9) return 0;
0081 |         if(MathAbs(fixedLot/step-MathRound(fixedLot/step))>1e-7) return 0;
0082 |         return fixedLot;
0083 |     }
0084 |     if(budget<=0) return 0;
0085 |     double volume=MathFloor(MathMin(budget/lossPerLot,maximum)/step+1e-9)*step;
0086 |     if(volume<minimum-1e-9 || volume*lossPerLot>budget+1e-7) return 0;
0087 |     return volume;
0088 | }
0089 | #endif
0090 |
0091 | #ifndef XAU_PROTECT_CORE_150
0092 | #define XAU_PROTECT_CORE_150
0093 | // Platform-independent decisions, shared unchanged with the local C++ tests.
0094 | // Original entry core is XAU_H1_Core.mqh; only CLOSED bars are read by the EA.
0095 | struct PEProfile { int mode,arm,pivotSide; double bel5,stopFactor,addFraction,capRisk; bool
protect14,partial,reinvest; };
0096 | // mode15/14=original family; 0=coordinated; 30=new confirmed-pivot hypothesis.
0097 | double PETHreshold(const int i)
0098 | {
0099 |     if(i==0) return 1.0; if(i==1) return 1.5; if(i==2) return 2.0;
0100 |     if(i==3) return 2.5; return 3.0;
0101 | }
0102 | bool PESelectProfile(const int id,PEProfile &p)
0103 | {
0104 |     p.mode=15; p.arm=0; p.bel5=0; p.protect14=false; p.partial=false;
0105 |     p.stopFactor=1; p.addFraction=0; p.capRisk=1; p.reinvest=false; p.pivotSide=0;
0106 |     if(id<1 || id>30) return false;
0107 |     if(id<=24) {
0108 |         p.mode=(id<=12 ? 15 : 14); p.arm=(id-1)%12;
0109 |         if(p.arm>=1 && p.arm<=5) p.bel5=PETHreshold(p.arm-1);
0110 |         p.partial=(p.arm==6); p.reinvest=(p.arm==11);
0111 |         if(p.arm==7 || p.arm==8) p.stopFactor=1.25;
0112 |         if(p.arm==8 || p.arm==9) p.addFraction=.5;
0113 |         if(p.arm==10) p.addFraction=1;
0114 |         if(p.arm==9) p.capRisk=1.3;
0115 |         if(p.arm==10) p.capRisk=1.6;
0116 |     } else if(id<=27) { p.mode=0; p.bel5=PETHreshold(id-24); p.protect14=true; }
0117 |     else { p.mode=30; p.pivotSide=(id==28 ? 1 : (id==29 ? -1 : 0)); }
0118 |     return true;
0119 | }
0120 | // Base setup and ATR-distance filter must have passed before this selection.
0121 | int PEChooseOrigin(const PEProfile &p,const double sma200,const double oldSma200)
0122 | {
0123 |     if(!MathIsValidNumber(sma200) || !MathIsValidNumber(oldSma200) || sma200<=0 || oldSma200<=0) return
0;
0124 |     bool rising=sma200>oldSma200;

```

```

0125 |     if(p.mode==14) return 14;
0126 |     if(p.mode==15) return rising ? 15 : 0;
0127 |     return rising ? 15 : 14;
0128 | }
0129 | bool PEDistance(const double ask,const double ema,const double atr,const double minimum)
0130 | {
0131 |     return MathIsValidNumber(ask) && MathIsValidNumber(ema) && MathIsValidNumber(atr) &&
0132 |         MathIsValidNumber(minimum) && ask>0 && ema>0 && atr>0 && minimum>=0 &&
0133 |         ask-ema>=minimum*atr-1e-8;
0134 | }
0135 | bool PEHalfVolumes(const double volume,const double minimum,const double maximum,const double step)
0136 | {
0137 |     return H1Volume(FIXED_LOT,volume,1,1,minimum,maximum,step)>0 &&
0138 |         H1Volume(FIXED_LOT,volume*0.5,1,1,minimum,maximum,step)>0;
0139 | }
0140 | bool PENear(const double a,const double b,const double tick)
0141 | { return tick>0 && MathAbs(a-b)<tick*0.5; }
0142 | bool PEGCanMoveBE(const double bid,const double entry,const double target,
0143 |     const double stops,const double freeze,const double tick)
0144 | {
0145 |     if(!MathIsValidNumber(bid) || !MathIsValidNumber(entry) || !MathIsValidNumber(target) ||
0146 |         bid<=0 || entry<=0 || target<=entry || stops<0 || freeze<0 || tick<=0) return false;
0147 |     double distance=MathMax(stops,freeze)+tick;
0148 |     return bid-entry>=distance-1e-8 && target-bid>=distance-1e-8;
0149 | }
0150 | bool PEPartialPriceReady(const double bid,const double entry,const double originalStop,const double
0151 |     target)
0152 | { return originalStop>0 && entry>originalStop && bid>=entry+(entry-originalStop)-1e-8 && bid<target;
0153 | }
0154 | // 0 unchanged; 1 planned half remains; 2 completely closed; -1 unexpected reduction.
0155 | int PEVolumeState(const double initial,const double live)
0156 | {
0157 |     if(initial<=0 || live<0) return -1;
0158 |     if(live<1e-8) return 2;
0159 |     if(MathAbs(live-initial)<1e-8) return 0;
0160 |     if(MathAbs(live-initial*0.5)<1e-8) return 1;
0161 |     return -1;
0162 | }
0163 | // Numeric MT5 retcodes, documented in REFERENCIAS.md.
0164 | // Ambiguous replies MUST NOT cause blind retries of a volume-changing request.
0165 | // 0 success; 1 definitive temporary rejection; 2 ambiguous/pending; 3 permanent/unknown.
0166 | int PERetCodeClass(const long ret)
0167 | {
0168 |     if(ret==10009 || ret==10025) return 0;
0169 |     if(ret==10004 || ret==10015 || ret==10016 || ret==10017 || ret==10018 ||
0170 |         ret==10020 || ret==10021 || ret==10024 || ret==10026 || ret==10027 || ret==10029) return 1;
0171 |     if(ret==10008 || ret==10010 || ret==10012 || ret==10028 || ret==10031 || ret==10039) return 2;
0172 |     return 3;
0173 | }
0174 | long PERetryDelayMs(const long ret)
0175 | {
0176 |     if(ret==10018 || ret==10017 || ret==10026 || ret==10027) return 60000;
0177 |     if(ret==10024) return 30000;
0178 |     if(ret==10029 || ret==10016) return 10000;
0179 |     return 5000;
0180 | }
0181 | struct PERequest
0182 | {
0183 |     bool armed,done,uncertain;
0184 |     long attempts,firstArmMs,lastTryMs,nextTryMs,lastRet;
0185 | };
0186 | bool PERequestDue(const PERequest &r,const long now)
0187 | { return r.armed && !r.done && !r.uncertain && now>=r.nextTryMs; }
0188 | void PEGArm(PERequest &r,const long now)
0189 | { if(!r.armed) { r.armed=true; r.firstArmMs=now; } }
0190 | void PEGAttempt(PERequest &r,const long now)
0191 | { r.attempts++; r.lastTryMs=now; r.nextTryMs=now+5000; }
0192 | void PEGRejected(PERequest &r,const long now,const long ret)
0193 | { r.lastRet=ret; r.nextTryMs=now+PERetryDelayMs(ret); }

```

```

0192 | // Reached/returned flags are observational; they are not simulated BE fills.
0193 | struct PEPATH
0194 | {
0195 |     double entry,stop,maxR,minR; int side;
0196 |     long observations;
0197 |     bool reached[5],returned[5];
0198 |     double returnQuoteR[5];
0199 | };
0200 | bool PEObserve(PEPATH &p,const double bid)
0201 | {
0202 |     if(!MathIsValidNumber(bid) || bid<=0 || p.side*p.entry<=p.side*p.stop || p.stop<=0) return false;
0203 |     double r=p.side*(bid-p.entry)/MathAbs(p.entry-p.stop);
0204 |     p.maxR=MathMax(p.maxR,r); p.minR=MathMin(p.minR,r); p.observations++;
0205 |     for(int i=0;i<5;i++)
0206 |     {
0207 |         if(r>=PETHreshold(i)-1e-8) p.reached[i]=true;
0208 |         if(p.reached[i] && !p.returned[i] && r<=0)
0209 |             { p.returned[i]=true; p.returnQuoteR[i]=r; }
0210 |     }
0211 |     return true;
0212 | }
0213 | // Monthly mark-to-market starts from the preceding month's last observed equity.
0214 | struct PEEquityPath
0215 | {
0216 |     bool initialized;
0217 |     double startEquity,endEquity,startBalance,endBalance,peak,maxRelativeDD,maxMoneyDD;
0218 | };
0219 | void PEStartEquity(PEEquityPath &p,const double equity,const double balance)
0220 | {
0221 |     p.initialized=true; p.startEquity=equity; p.endEquity=equity;
0222 |     p.startBalance=balance; p.endBalance=balance; p.peak=equity;
0223 |     p.maxRelativeDD=0; p.maxMoneyDD=0;
0224 | }
0225 | bool PEObserveEquity(PEEquityPath &p,const double equity,const double balance)
0226 | {
0227 |     if(!p.initialized || !MathIsValidNumber(equity) || !MathIsValidNumber(balance)) return false;
0228 |     p.endEquity=equity; p.endBalance=balance; p.peak=MathMax(p.peak,equity);
0229 |     p.maxMoneyDD=MathMax(p.maxMoneyDD,p.peak-equity);
0230 |     if(p.peak>0) p.maxRelativeDD=MathMax(p.maxRelativeDD,100*(p.peak-equity)/p.peak);
0231 |     return true;
0232 | }
0233 | // Positive, realised profit only; volume is rounded DOWN and capped. No floating-profit sizing.
0234 | double PEReinvestLot(const double base,const double deposit,const double balance,const double
cap,const double step)
0235 | { if(base<=0 || deposit<=0 || cap<base || step<=0) return 0;
0236 |   double effective=base*(1+.5*MathMax(0,balance-deposit)/deposit);
0237 |   return MathFloor(MathMin(cap,effective)/step+1e-8)*step;
0238 | }
0239 | bool PERiskWithin(const double current,const double extra,const double original,const double multiple)
0240 | { return MathIsValidNumber(current) && MathIsValidNumber(extra) && current>=0 && extra>=0 &&
original>0 &&
0241 |   multiple>=1 && current+extra<=original*multiple+.01; }
0242 | bool PEAddWindow(const int arm,const int filled,const double r,const bool closedRecovery)
0243 | {
0244 |     if(arm==8) return filled<2 && r>=filled+1-1e-8 && r<=filled+1.25+1e-8;
0245 |     if(arm==9 || arm==10) return filled==0 && closedRecovery && r>=-.65-1e-8 && r<=-.35+1e-8;
0246 |     return false;
0247 | }
0248 | // Arrays below are ordered oldest to newest and contain only CLOSED candles.
0249 | // A pivot requires two complete bars on each side. Consecutive same-kind pivots
0250 | // collapse to the more extreme one; ambiguous outside bars are ignored.
0251 | struct PEPivot { int index,kind; double price; };
0252 | int PEPivotSignal(const double &high[],const double &low[],const int count,
0253 |                  const double close,const double previousClose,double &level,double &swing,int &key)
0254 | {
0255 |     PEPivot p[128]; int n=0; level=0; swing=0; key=-1;
0256 |     if(count<9 || count>128) return 0;
0257 |     for(int i=2;i<count-2;i++) {
0258 |         bool top=true,bottom=true;

```

```

0259 |     for(int j=-2;j<=2;j++) if(j!=0) { if(high[i]<=high[i+j]) top=false; if(low[i]>=low[i+j])
| bottom=false; }
0260 |     if(top==bottom) continue;
0261 |     int kind=top ? 1 : -1; double price=top ? high[i] : low[i];
0262 |     if(n>0 && p[n-1].kind==kind) { if(kind*price>kind*p[n-1].price) { p[n-1].price=price;
| p[n-1].index=i; } continue; }
0263 |     p[n].index=i; p[n].kind=kind; p[n].price=price; n++;
0264 | }
0265 | if(n<5) return 0;
0266 | int a=n-5,b=n-4,c=n-3,d=n-2,e=n-1;
0267 | if(p[a].kind==1 && p[c].price<p[a].price && p[d].price<p[b].price && p[e].price>p[c].price &&
0268 | close>p[d].price && previousClose<=p[d].price) {
0269 |     level=p[d].price; swing=p[e].price; key=p[e].index; return 1;
0270 | }
0271 | if(p[a].kind==1 && p[c].price>p[a].price && p[d].price>p[b].price && p[e].price<p[c].price &&
0272 | close<p[d].price && previousClose>=p[d].price) {
0273 |     level=p[d].price; swing=p[e].price; key=p[e].index; return -1;
0274 | }
0275 | return 0;
0276 | }
0277 | bool PEInSession(const int second,const int start,const int finish)
0278 | { if(start==finish) return true; return finish>start ? (second>=start && second<finish) :
| (second>=start || second<finish); }
0279 | #endif
0280 |
0281 | #ifndef XAU_ENTRY_CORE_170
0282 | #define XAU_ENTRY_CORE_170
0283 | // Fixed, predeclared research hypotheses. No fitting or future bars at runtime.
0284 | struct EV0Profile {
0285 |     int family,entryPolicy,capitalMode,matchedControl;
0286 |     double reinvestFraction,lotCap;
0287 | };
0288 | struct EV0Features {
0289 |     bool oscillatorValid;
0290 |     double fast310,previousFast310,signal310,previousClose,trueRangeATR,bodyATR,closeLocation;
0291 | };
0292 | bool EV0SelectProfile(const int id,EV0Profile &e,PEProfile &p)
0293 | {
0294 |     if(id<1 || id>56) return false;
0295 |     int offset=id<=28 ? 0 : 28,slot=id-offset-1;
0296 |     e.family=offset==0 ? 15 : 14; e.entryPolicy=slot<20 ? slot%10 : 0;
0297 |     e.capitalMode=slot<10 ? 0 : 1; e.reinvestFraction=slot<10 ? 0 : .50; e.lotCap=slot<10 ? .10 : .30;
0298 |     e.matchedControl=offset+(slot<10 ? 1 : 11);
0299 |     if(slot>=20) {
0300 |         int grid=slot-19; // Grid indices 1..8; index0 (.50,.30) is the existing control.
0301 |         int fraction=grid/3,cap=grid%3;
0302 |         e.reinvestFraction=fraction==0 ? .50 : (fraction==1 ? .75 : 1.0);
0303 |         e.lotCap=cap==0 ? .30 : (cap==1 ? .50 : 1.0);
0304 |     }
0305 |     // Reuse the validated execution engine, with partials/BE/additions disabled.
0306 |     p.mode=e.family; p.arm=e.capitalMode==0 ? 0 : 11; p.pivotSide=0;
0307 |     p.be15=0; p.stopFactor=1; p.addFraction=0; p.capRisk=1;
0308 |     p.protect14=false; p.partial=false; p.reinvest=e.capitalMode!=0;
0309 |     return true;
0310 | }
0311 | // close[0] is the most recently CLOSED bar; 25 bars suffice for SMA16(SMA3-SMA10).
0312 | bool EV0Oscillator(const double &close[],const int count,double &fast,double &previous,double &signal)
0313 | {
0314 |     fast=0; previous=0; signal=0;
0315 |     if(count<25) return false;
0316 |     for(int i=0;i<25;i++) if(!MathIsValidNumber(close[i]) || close[i]<=0) return false;
0317 |     for(int j=0;j<16;j++) {
0318 |         double shortSum=0,longSum=0;
0319 |         for(int k=0;k<10;k++) { longSum+=close[j+k]; if(k<3) shortSum+=close[j+k]; }
0320 |         double diff=shortSum/3.0-longSum/10.0;
0321 |         if(j==0) fast=diff; if(j==1) previous=diff; signal+=diff/16.0;
0322 |     }
0323 |     return true;
0324 | }

```

```

0325 | double EVOCapitalLot(const double base,const double deposit,const double balance,
0326 |                     const double fraction,const double cap,const double step)
0327 | {
0328 |     if(!MathIsValidNumber(base) || !MathIsValidNumber(deposit) || !MathIsValidNumber(balance) ||
0329 |        !MathIsValidNumber(fraction) || !MathIsValidNumber(cap) || !MathIsValidNumber(step) ||
0330 |        base<=0 || deposit<=0 || balance<=0 || fraction<0 || fraction>1 || cap<base || step<=0) return 0;
0331 |     double raw=base*(1+fraction*MathMax(0,balance-deposit)/deposit);
0332 |     return MathFloor(MathMin(raw,cap)/step+1e-8)*step;
0333 | }
0334 | // 0 accepted; original H1 gate codes 1..15; 100+ identify the new filters.
0335 | int EV0Evaluate(const EV0Profile &e,const H1Signal &s,const EV0Features &f,
0336 |                const double ask,const double old200,const double minimumDistance,
0337 |                const bool freshConfirmedBullPivot,int &side)
0338 | {
0339 |     side=0; if(s.atr<=0 || ask<=0) return 100;
0340 |     int gate=0;
0341 |     if(e.entryPolicy==7 || e.entryPolicy==8) {
0342 |         if(!(s.ema>s.sma50 && s.sma50>s.oldSma50 && s.close>s.sma200)) return NO_TREND;
0343 |         if(s.adx<20) return ADX_LOW;
0344 |         if(s.plusDI<=s.minusDI) return DI_CONFLICT;
0345 |         if(!(s.close>s.open && s.close>s.ema)) return NO_TRIGGER;
0346 |         if(e.entryPolicy==7) {
0347 |             if(!s.priorTouched) return NO_TOUCH;
0348 |             if(!(s.close>f.previousClose && f.bodyATR>=.20)) return 107;
0349 |             if(!f.oscillatorValid || f.fast310<=f.previousFast310) return 103;
0350 |         } else {
0351 |             if(!freshConfirmedBullPivot) return 108;
0352 |             if(s.adx<=s.oldADX) return 101;
0353 |         }
0354 |         side=1;
0355 |     } else {
0356 |         gate=H1EvaluateProfile(s,B_PRIOR_PULLBACK,true,20.0,side);
0357 |         if(gate!=0) return gate;
0358 |         if(side!=1) return 109;
0359 |     }
0360 |     double minimum=e.entryPolicy==6 ? .25 : minimumDistance;
0361 |     if(!PEDistance(ask,s.ema,s.atr,minimum)) return 110;
0362 |     if(e.family==15 && s.sma200<=old200) return 111;
0363 |     if((e.entryPolicy==1 || e.entryPolicy==9) && s.adx<=s.oldADX) return 101;
0364 |     if(e.entryPolicy==2 && s.adx<25) return 102;
0365 |     if((e.entryPolicy==3 || e.entryPolicy==9) &&
0366 |        (!f.oscillatorValid || f.fast310<=f.previousFast310 || f.fast310<=f.signal310)) return 103;
0367 |     if((e.entryPolicy==4 || e.entryPolicy==9) && (ask-s.ema)/s.atr>1.0+1e-8) return 104;
0368 |     if(e.entryPolicy==5 && f.trueRangeATR>1.8+1e-8) return 105;
0369 |     return 0;
0370 | }
0371 | #endif
0372 |
0373 | #ifndef APOLO_DONCHIAN_CORE_180
0374 | #define APOLO_DONCHIAN_CORE_180
0375 | // Predeclared hypotheses. All inputs are CLOSED-bar snapshots.
0376 | struct DCProfile {
0377 |     int id,channelMode,matchedControl,sourceControl180,targetMode;
0378 |     bool weekly,originalCapital,fixedLot;
0379 |     double riskPercent,minimumStopATR;
0380 | };
0381 | bool DCSelectProfile(const int id,DCProfile &d,EV0Profile &e,PEProfile &p)
0382 | {
0383 |     if(id<1 || id>6 || !EV0SelectProfile(28,e,p)) return false;
0384 |     d.id=id; d.weekly=id>=5; d.originalCapital=false; d.fixedLot=id>=2 && id<=4;
0385 |     d.channelMode=id==6 ? 1 : 0;
0386 |     d.riskPercent=d.fixedLot ? 0.0 : 2.0; d.minimumStopATR=1.0;
0387 |     d.targetMode=id==3 ? 1 : (id==4 ? 2 : 0);
0388 |     d.sourceControl180=id==5 ? 8 : (id==6 ? 10 : 3);
0389 |     d.matchedControl=(id==3 || id==4) ? 2 : (id>=5 ? 5 : 1);
0390 |     e.capitalMode=d.fixedLot ? 3 : 2; e.reinvestFraction=0; p.reinvest=false; p.arm=0;
0391 |     e.matchedControl=d.matchedControl;
0392 |     return true;
0393 | }

```

```

0394 | // Entire window must be completed BEFORE the trigger candle starts.
0395 | // The containing W1 bar is excluded, also when evaluating the final M30 bar of a week.
0396 | bool DCWindow(const double &high[],const double &low[],const int count,
0397 |               const long newestClose,const long cutoff,double &upper,double &lower)
0398 | {
0399 |     upper=0; lower=0;
0400 |     if(count!=52 || newestClose<=0 || newestClose>cutoff) return false;
0401 |     for(int i=0;i<52;i++) {
0402 |         if(!MathIsValidNumber(high[i]) || !MathIsValidNumber(low[i]) || low[i]<=0 || high[i]<low[i]) return
false;
0403 |         if(i==0) { upper=high[i]; lower=low[i]; }
0404 |         else { upper=MathMax(upper,high[i]); lower=MathMin(lower,low[i]); }
0405 |     }
0406 |     return upper>lower;
0407 | }
0408 | int DCChannelGate(const int mode,const bool ready,const double close,const double previousClose,
0409 |                  const double upper,const double lower)
0410 | {
0411 |     if(mode==0) return 0;
0412 |     if(!ready || upper<=lower || lower<=0) return 120;
0413 |     if(mode==1) return close>(upper+lower)/2.0 ? 0 : 121;
0414 |     if(mode==2) return close>upper && previousClose<=upper ? 0 : 122;
0415 |     return 123;
0416 | }
0417 | int DCWeeklyGate(const H1Signal &s,int &side)
0418 | {
0419 |     side=0;
0420 |     if(s.atr<=0 || s.ema<=0) return 120;
0421 |     if(s.adx<=25.0) return ADX_LOW;
0422 |     if(s.plusDI<=s.minusDI) return DI_CONFLICT;
0423 |     if(!s.priorTouched) return NO_TOUCH;
0424 |     if(!(s.close>s.open && s.close>s.ema && s.close>s.previousHigh)) return NO_TRIGGER;
0425 |     side=1; return 0;
0426 | }
0427 | bool DCStructuralStopAccepted(const double entry,const double stop,const double atr,const double
minimumATR)
0428 | {
0429 |     if(!MathIsValidNumber(entry) || !MathIsValidNumber(stop) || !MathIsValidNumber(atr) ||
!MathIsValidNumber(minimumATR) || atr<=0) return false;
0430 |     return MathAbs(entry-stop)/atr+1e-8>=minimumATR;
0431 | }
0432 | // Risk is stop-distance risk before slippage, gaps, commission and swap.
0433 | // Floor only: never force the minimum lot when it exceeds either budget.
0434 | double DCRiskLot(const double equity,const double freeMargin,const double currentMargin,
0435 |                  const double riskPercent,const double marginPercent,const double lossPerLot,const double
marginPerLot,
0436 |                  const double minimum,const double maximum,const double step,const double hardLotCap)
0437 | {
0438 |     double values[11]={equity,freeMargin,currentMargin,riskPercent,marginPercent,lossPerLot,marginPerLot
,minimum,maximum,step,hardLotCap};
0439 |     for(int i=0;i<11;i++) if(!MathIsValidNumber(values[i])) return 0;
0440 |     if(equity<=0 || freeMargin<=0 || currentMargin<0 || riskPercent<=0 || riskPercent>2 ||
marginPercent<=0 || marginPercent>50 || lossPerLot<=0 || marginPerLot<0 || minimum<=0 ||
0441 |         maximum<minimum || step<=0 || hardLotCap<=0) return 0;
0442 |     double budget=equity*riskPercent/100.0;
0443 |     double available=MathMin(freeMargin,MathMax(0,equity*marginPercent/100.0-currentMargin));
0444 |     if(available<=0) return 0;
0445 |     double raw=MathMin(maximum,MathMin(hardLotCap,budget/lossPerLot));
0446 |     if(marginPerLot>0) raw=MathMin(raw,available/marginPerLot);
0447 |     double volume=MathFloor(raw/step+1e-9)*step;
0448 |     if(volume<minimum*1e-9 || volume*lossPerLot>budget+1e-7 || volume*marginPerLot>available+1e-7)
return 0;
0449 |     return volume;
0450 | }
0451 | }
0452 | #endif
0453 |
0454 | #ifndef OLIMPO_HERMES_CORE_190
0455 | #define OLIMPO_HERMES_CORE_190
0456 | // Target mode: 0=5 initial quoted stop distances, 1=20 quote-price units,

```

```

0457 | // 2=20 native SYMBOL_POINT units. A point is not a pip or a dollar profit.
0458 | double HermesTargetDistance(const int mode,const double point)
0459 | {
0460 |     if(mode==1) return 20.0;
0461 |     if(mode==2 && MathIsValidNumber(point) && point>0) return 20.0*point;
0462 |     return 0;
0463 | }
0464 | bool HermesTargetPrice(const int side,const double entry,const double distance,const double
tick,double &target)
0465 | {
0466 |     target=0;
0467 |     if((side!=1 && side!=-1) || !MathIsValidNumber(entry) || !MathIsValidNumber(distance) ||
0468 |         !MathIsValidNumber(tick) || entry<=0 || tick<=0 || distance<tick) return false;
0469 |     double raw=entry+side*distance;
0470 |     if(!MathIsValidNumber(raw) || raw<=0) return false;
0471 |     // Round toward entry, never increase the requested target distance.
0472 |     target=(side==1 ? MathFloor(raw/tick+1e-9) : MathCeil(raw/tick-1e-9))*tick;
0473 |     double tolerance=MathMax(1e-10,tick*1e-7);
0474 |     if(side*(target-entry)>distance+tolerance) target-=side*tick;
0475 |     double actual=side*(target-entry);
0476 |     return target>0 && actual>=tick-tolerance && actual<=distance+tolerance;
0477 | }
0478 | // Fixed lot means exact or denied; it is never silently reduced to fit capital.
0479 | bool HermesExactLot(const double requested,const double minimum,const double maximum,const double
step)
0480 | {
0481 |     if(!MathIsValidNumber(requested) || !MathIsValidNumber(minimum) || !MathIsValidNumber(maximum) ||
0482 |         !MathIsValidNumber(step) || requested<=0 || minimum<=0 || maximum<minimum || step<=0 ||
0483 |         requested<minimum-1e-9 || requested>maximum+1e-9) return false;
0484 |     return MathAbs(requested-MathRound(requested/step)*step)<1e-8;
0485 | }
0486 | bool HermesMarginAvailable(const double required,const double freeMargin)
0487 | {
0488 |     return MathIsValidNumber(required) && MathIsValidNumber(freeMargin) && required>=0 &&
0489 |         freeMargin>0 && required<=freeMargin;
0490 | }
0491 | #endif
0492 |
0493 | #ifndef HERMES_PIVOT_CORE_MQH
0494 | #define HERMES_PIVOT_CORE_MQH
0495 |
0496 | // Causal geometric detector, not an order or portfolio engine.
0497 | // Matches HERMES_PIVOS_2x2_V1 (catalogar_pivos.py) LONG 1-2-3 events.
0498 | // No STL, allocation, MQL indicator handles, current-bar prices or market calls.
0499 | // Feed every CLOSED observed bar once, plus the next observed opening time.
0500 | // A next-open witness never grants access to that next bar's OHLC.
0501 | // Return false: invalid OHLC, duplicate/out-of-order or skipped input; state
0502 | // remains unchanged. Return true: accepted; inspect signal.buy for a trigger.
0503 | // Price ties are excluded. A double-extreme bar resets 1-2-3 sequencing.
0504 | // Replays must begin from the same historical origin to match the catalog.
0505 |
0506 | struct HPBar
0507 | {
0508 |     long time;
0509 |     double open;
0510 |     double high;
0511 |     double low;
0512 |     double close;
0513 | };
0514 |
0515 | struct HPPivot
0516 | {
0517 |     int type; // +1 HIGH, -1 LOW
0518 |     double price;
0519 |     long origin_time;
0520 |     long confirmation_bar_time;
0521 |     long available_time; // opening after the second right bar
0522 | };
0523 |

```

```

0524 | struct HPSignal
0525 | {
0526 |     bool buy;
0527 |     long signal_time;           // closed breakout bar's opening
0528 |     long available_time;       // first possible decision/entry opening
0529 |     double reference_price;     // neckline, confirmed P2 HIGH
0530 |     double stop_f1;            // P1 LOW, geometric invalidation level
0531 |     double pullback_f2;        // P3 LOW, higher low (not a chosen stop)
0532 |     long p1_time;
0533 |     long p2_time;
0534 |     long p3_time;
0535 |     long p1_confirmation_time;
0536 |     long p2_confirmation_time;
0537 |     long p3_confirmation_time;
0538 |     long p1_available_time;
0539 |     long p2_available_time;
0540 |     long p3_available_time;
0541 |     long pattern_available_time;
0542 | };
0543 |
0544 | struct HPState
0545 | {
0546 |     HPBar window[5];
0547 |     int window_count;
0548 |     HPPivot alternating[3];
0549 |     int alternating_count;
0550 |     HPPivot p1;
0551 |     HPPivot p2;
0552 |     HPPivot p3;
0553 |     bool active_pattern;
0554 |     bool active_long;
0555 |     bool active_invalid;       // includes a pattern already triggered
0556 |     long pattern_available_time;
0557 |     long expected_open;
0558 |     long last_time;
0559 |     long bars_processed;
0560 |     long pivots_high;
0561 |     long pivots_low;
0562 |     long double_extremes;
0563 |     long tie_high_windows;
0564 |     long tie_low_windows;
0565 |     long patterns_long;
0566 |     long patterns_short;
0567 |     long events_long;
0568 |     long events_short_reference_only;
0569 | };
0570 |
0571 | void HPClearPivot(HPPivot &p)
0572 | {
0573 |     p.type=0; p.price=0; p.origin_time=0;
0574 |     p.confirmation_bar_time=0; p.available_time=0;
0575 | }
0576 |
0577 | void HPClearSignal(HPSignal &s)
0578 | {
0579 |     s.buy=false; s.signal_time=0; s.available_time=0;
0580 |     s.reference_price=0; s.stop_f1=0; s.pullback_f2=0;
0581 |     s.p1_time=0; s.p2_time=0; s.p3_time=0;
0582 |     s.p1_confirmation_time=0; s.p2_confirmation_time=0;
0583 |     s.p3_confirmation_time=0;
0584 |     s.p1_available_time=0; s.p2_available_time=0; s.p3_available_time=0;
0585 |     s.pattern_available_time=0;
0586 | }
0587 |
0588 | void HPCoreReset(HPState &s)
0589 | {
0590 |     s.window_count=0; s.alternating_count=0;
0591 |     for(int i=0; i<5; ++i)
0592 |     {

```

```

0593 |         s.window[i].time=0; s.window[i].open=0; s.window[i].high=0;
0594 |         s.window[i].low=0; s.window[i].close=0;
0595 |     }
0596 |     for(int i=0; i<3; ++i) HPClearPivot(s.alternating[i]);
0597 |     HPClearPivot(s.p1); HPClearPivot(s.p2); HPClearPivot(s.p3);
0598 |     s.active_pattern=false; s.active_long=false; s.active_invalid=false;
0599 |     s.pattern_available_time=0; s.expected_open=0; s.last_time=0;
0600 |     s.bars_processed=0;
0601 |     s.pivots_high=0; s.pivots_low=0; s.double_extremes=0;
0602 |     s.tie_high_windows=0; s.tie_low_windows=0;
0603 |     s.patterns_long=0; s.patterns_short=0;
0604 |     s.events_long=0; s.events_short_reference_only=0;
0605 | }
0606 |
0607 | bool HPFinite(const double x)
0608 | {
0609 |     // These finite bounds cover any meaningful quoted market price and reject
0610 |     // +/-infinity and NaN without compiler-specific math functions.
0611 |     return x==x && x>-1.0e100 && x<1.0e100;
0612 | }
0613 |
0614 | bool HPValidBar(const HPBar &b)
0615 | {
0616 |     return HPFinite(b.open) && HPFinite(b.high) && HPFinite(b.low)
0617 |         && HPFinite(b.close) && b.low<=b.open && b.low<=b.close
0618 |         && b.high>=b.open && b.high>=b.close && b.high>=b.low;
0619 | }
0620 |
0621 | void HPExposePivot(HPState &s, const HPPivot &p, const long next_open)
0622 | {
0623 |     bool changed=false;
0624 |     const int count=s.alternating_count;
0625 |     if(count==0 || s.alternating[count-1].type!=p.type)
0626 |     {
0627 |         if(count<3)
0628 |         {
0629 |             s.alternating[count]=p;
0630 |             ++s.alternating_count;
0631 |         }
0632 |         else
0633 |         {
0634 |             s.alternating[0]=s.alternating[1];
0635 |             s.alternating[1]=s.alternating[2];
0636 |             s.alternating[2]=p;
0637 |         }
0638 |         changed=true;
0639 |     }
0640 |     else if((p.type==1 && p.price>s.alternating[count-1].price)
0641 |         || (p.type==-1 && p.price<s.alternating[count-1].price))
0642 |     {
0643 |         s.alternating[count-1]=p;
0644 |         changed=true;
0645 |     }
0646 |     if(!changed) return;
0647 |     s.active_pattern=false; s.active_invalid=false;
0648 |     if(s.alternating_count!=3) return;
0649 |     s.p1=s.alternating[0]; s.p2=s.alternating[1]; s.p3=s.alternating[2];
0650 |     const bool is_long=s.p1.type==-1 && s.p1.price<s.p3.price
0651 |         && s.p3.price<s.p2.price;
0652 |     const bool is_short=s.p1.type==1 && s.p1.price>s.p3.price
0653 |         && s.p3.price>s.p2.price;
0654 |     if(!is_long && !is_short) return;
0655 |     s.active_pattern=true; s.active_long=is_long;
0656 |     s.pattern_available_time=next_open;
0657 |     if(is_long) ++s.patterns_long;
0658 |     else ++s.patterns_short;
0659 | }
0660 |
0661 | void HPEmitLong(const HPState &s, const HPBar &b, const long next_open,

```

```

0662 |         HPSignal &signal)
0663 |     {
0664 |         signal.buy=true; signal.signal_time=b.time; signal.available_time=next_open;
0665 |         signal.reference_price=s.p2.price;
0666 |         signal.stop_f1=s.p1.price; signal.pullback_f2=s.p3.price;
0667 |         signal.p1_time=s.p1.origin_time; signal.p2_time=s.p2.origin_time;
0668 |         signal.p3_time=s.p3.origin_time;
0669 |         signal.p1_confirmation_time=s.p1.confirmation_bar_time;
0670 |         signal.p2_confirmation_time=s.p2.confirmation_bar_time;
0671 |         signal.p3_confirmation_time=s.p3.confirmation_bar_time;
0672 |         signal.p1_available_time=s.p1.available_time;
0673 |         signal.p2_available_time=s.p2.available_time;
0674 |         signal.p3_available_time=s.p3.available_time;
0675 |         signal.pattern_available_time=s.pattern_available_time;
0676 |     }
0677 |
0678 | bool HPStep(HPState &s, const HPBar &closed, const long next_open,
0679 |             HPSignal &signal)
0680 | {
0681 |     HPClearSignal(signal);
0682 |     if(!HPValidBar(closed) || next_open<=closed.time) return false;
0683 |     if(s.bars_processed>0 && closed.time!=s.expected_open) return false;
0684 |
0685 |     // Evaluate the just-closed bar using only references already available at
0686 |     // its opening. A newly confirmed pivot must not change this bar's signal.
0687 |     if(s.window_count>0 && s.active_pattern)
0688 |     {
0689 |         const double previous_close=s.window[s.window_count-1].close;
0690 |         bool broke=false;
0691 |         if(s.active_long)
0692 |         {
0693 |             if(closed.low<=s.p1.price) s.active_invalid=true;
0694 |             broke=previous_close<=s.p2.price && s.p2.price<closed.close;
0695 |         }
0696 |         else
0697 |         {
0698 |             if(closed.high>=s.p1.price) s.active_invalid=true;
0699 |             broke=previous_close>=s.p2.price && s.p2.price>closed.close;
0700 |         }
0701 |         const bool known_at_open=s.pattern_available_time<=closed.time
0702 |             && s.p2.available_time<=closed.time;
0703 |         if(broke && !s.active_invalid && known_at_open)
0704 |         {
0705 |             if(s.active_long)
0706 |             {
0707 |                 HPEmitLong(s,closed,next_open,signal);
0708 |                 ++s.events_long;
0709 |             }
0710 |             else ++s.events_short_reference_only;
0711 |             s.active_invalid=true; // exactly one event per pattern snapshot
0712 |         }
0713 |     }
0714 |
0715 |     if(s.window_count<5)
0716 |     {
0717 |         s.window[s.window_count]=closed;
0718 |         ++s.window_count;
0719 |     }
0720 |     else
0721 |     {
0722 |         for(int i=0; i<4; ++i) s.window[i]=s.window[i+1];
0723 |         s.window[4]=closed;
0724 |     }
0725 |     s.expected_open=next_open; s.last_time=closed.time; ++s.bars_processed;
0726 |     if(s.window_count<5) return true;
0727 |
0728 |     const HPBar center=s.window[2];
0729 |     bool strict_high=true, strict_low=true, max_high=true, min_low=true;
0730 |     for(int i=0; i<5; ++i)

```

```

0731 | {
0732 |     if(i==2) continue;
0733 |     if(center.high<=s.window[i].high) strict_high=false;
0734 |     if(center.low>=s.window[i].low) strict_low=false;
0735 |     if(center.high<s.window[i].high) max_high=false;
0736 |     if(center.low>s.window[i].low) min_low=false;
0737 | }
0738 | if(max_high && !strict_high) ++s.tie_high_windows;
0739 | if(min_low && !strict_low) ++s.tie_low_windows;
0740 | if(strict_high) ++s.pivots_high;
0741 | if(strict_low) ++s.pivots_low;
0742 |
0743 | if(strict_high && strict_low)
0744 | {
0745 |     ++s.double_extremes;
0746 |     s.alternating_count=0;
0747 |     s.active_pattern=false; s.active_invalid=false;
0748 |     return true;
0749 | }
0750 | if(!strict_high && !strict_low) return true;
0751 | HPPivot pivot;
0752 | pivot.type=strict_high ? 1 : -1;
0753 | pivot.price=strict_high ? center.high : center.low;
0754 | pivot.origin_time=center.time;
0755 | pivot.confirmation_bar_time=closed.time;
0756 | pivot.available_time=next_open;
0757 | HPExposePivot(s,pivot,next_open);
0758 | return true;
0759 | }
0760 |
0761 | #endif
0762 |
0763 | #ifndef HERMES_PIVOT_ENTRY_CORE_200
0764 | #define HERMES_PIVOT_ENTRY_CORE_200
0765 |
0766 | // Production routing shared by the EA and portable decision tests.
0767 | // No entry count, loss outcome, future price or calendar outcome is an input.
0768 | enum HP_ENTRY_PATH { HP_NONE=0, HP_BASE=1, HP_PIVOT=2 };
0769 | enum HP_EXTRA_GATE { HP_EXTRA_READY=0, HP_DISABLED=200, HP_DATA_PENDING,
0770 |     HP_NO_CANDIDATE, HP_TREND50, HP_TREND200, HP_ADX, HP_DI, HP_TRIGGER,
0771 |     HP_OSCILLATOR, HP_DISTANCE, HP_INVALID_CASE };
0772 | struct HPDecision
0773 | {
0774 |     int original_gate,extra_gate,setup,side,path;
0775 | };
0776 | bool HPSelectProfile(const int id,DCProfile &d,EV0Profile &e,PEProfile &p)
0777 | {
0778 |     if(id<1 || id>3 || !DCSelectProfile(2,d,e,p)) return false;
0779 |     d.id=id; d.matchedControl=2; e.matchedControl=2;
0780 |     // The inherited reference is exact fixed volume, 5R, buy only, one leg.
0781 |     return d.fixedLot && !d.weekly && d.targetMode==0 && d.channelMode==0
0782 |         && e.entryPolicy==0 && e.family==15 && !p.reinvest && !p.partial
0783 |         && !p.protect14 && p.be15==0 && p.addFraction==0 && p.arm==0;
0784 | }
0785 | int HPExtraGate(const int id,const H1Signal &s,const EV0Features &f,
0786 |     const double ask,const double old200,const double minimumDistance,
0787 |     const bool pivotDataReady,const HPSignal &pivot)
0788 | {
0789 |     if(id<1 || id>3) return HP_INVALID_CASE;
0790 |     if(id==1) return HP_DISABLED;
0791 |     if(!pivotDataReady) return HP_DATA_PENDING;
0792 |     if(!pivot.buy) return HP_NO_CANDIDATE;
0793 |     if(!(s.ema>s.sma50 && s.sma50>s.oldSma50)) return HP_TREND50;
0794 |     // This is the ONLY difference between the two extra-entry variants.
0795 |     if(id==2 && !(s.close>s.sma200 && s.sma200>old200)) return HP_TREND200;
0796 |     if(!(s.adx>=20.0)) return HP_ADX;
0797 |     if(!(s.plusDI>s.minusDI)) return HP_DI;
0798 |     if(!(s.close>s.open && s.close>s.ema)) return HP_TRIGGER;
0799 |     if(!f.oscillatorValid || !(f.fast310>f.previousFast310)) return HP_OSCILLATOR;

```

```

0800 | if(!PEDistance(ask,s.ema,s.atr,minimumDistance)) return HP_DISTANCE;
0801 | return HP_EXTRA_READY;
0802 | }
0803 | void HPRouteEntry(const int id,const EVOProfile &e,const H1Signal &s,const EVOFeatures &f,
0804 |                 const double ask,const double old200,const double minimumDistance,
0805 |                 const bool pivotDataReady,const HPSignal &pivot,HPDecision &out)
0806 | {
0807 |     out.side=0; out.path=HP_NONE;
0808 |     out.original_gate=EVOEvaluate(e,s,f,ask,old200,minimumDistance,false,out.side);
0809 |     out.extra_gate=HPEXtraGate(id,s,f,ask,old200,minimumDistance,pivotDataReady,pivot);
0810 |     out.setup=out.original_gate;
0811 |     if(id<1 || id>3) { out.setup=HP_INVALID_CASE; out.side=0; return; }
0812 |     if(out.original_gate==0) { out.path=HP_BASE; return; }
0813 |     if(out.extra_gate==0) { out.setup=0; out.side=1; out.path=HP_PIVOT; }
0814 | }
0815 | #endif
0816 |
0817 |
0818 | // Three predeclared M30 cases. Exact fixed lot and 5R in every case.
0819 | input int InpCase=1;
0820 | input double InpMaxMarginPct=20.0;
0821 | input double InpMaxLot=1.0;
0822 | input double InpFixedLot=1.00;
0823 | input double InpMinEntryATR=0.50;
0824 | input double InpMaxSpreadPoints=0;
0825 | input ulong InpDeviationPoints=20;
0826 | input ulong InpMagic=26120200;
0827 | input bool InpExportCSV=true;
0828 | input bool InpShowIndicators=false;
0829 | input bool InpExportOptimizationDetails=true;
0830 | input bool InpExportAllBars=true;
0831 | input string InpRunTag="R200_01";
0832 |
0833 | enum PE_GATE { G_NO_DATA=0,G_BASE,G_DIRECTION,G_DISTANCE,G_REGIME,G_POSITION,
0834 |               G_SPREAD,G_STOP,G_BROKER_STOPS,G_RISK,G_MARGIN_ERROR,G_MARGIN_BLOCK,G_REJECTED,G_FILLED,G_HALTED,G_EN
TRY_FILTER,G_TARGET,G_COUNT };
0835 | enum PE_STAT {
0836 |     S_VALID=0,S_PROFIT,S_DEPOSIT,S_FINAL_BALANCE,S_MT5_TRADES,S_MT5_PF,S_EQUITY_DD_REL,S_EQUITY_DD_MONEY,
S_EQUITY_DD_AT_MONEY,
0837 |     S_CYCLES_OPENED,S_CYCLES_CLOSED,S_WINS,S_LOSSES,S_ZERO,S_WIN_PCT,S_CYCLE_PF,S_CYCLE_NET,S_AVG_NET_R,
0838 |     S_MAX_LOSS_STREAK,S_AVG_HOLD,S_MAX_HOLD,S_SWAP,S_COSTS,S_MAX_LOTS,S_MAX_RISK,S_MAX_RISK_PCT,S_MAX_MAR
GIN,
0839 |     S_FIRST_EVAL,S_LAST_EVAL,S_FIRST_ENTRY,S_LAST_EXIT,S_BARS,S_SIGNALS,S_FIRST_READY,
0840 |     S_ORIGIN15_CYCLES,S_ORIGIN14_CYCLES,S_ORIGIN15_NET,S_ORIGIN14_NET,
0841 |     S_BE_ARMED,S_BE_CONFIRMED,S_BE_STOP_EXITS,S_PARTIAL_ARMED,S_PARTIAL_DEFERRED,S_BE_DEFE
RRED,
0842 |     S_SESSION_DEFERS,S_STOPS_DEFERS,S_BE_ATTEMPTS,S_PARTIAL_ATTEMPTS,S_AMBIGUOUS_PARTIALS,S_ERROR_CODE,S_
LAST_RETCODE,S_LAST_ERROR_TIME,
0843 |     S_RECONCILE,S_MONTH_EQ_RECONCILE,S_MONTH_BOOK_RECONCILE,S_UNMATCHED,S_MONTHS,S_POS_EQ_MONTHS,S_NEG_EQ
MONTHS,S_POS_BOOK_MONTHS,S_NEG_BOOK_MONTHS,
0844 |     S_MFE_AVG,S_MFE_WIN_AVG,S_MFE_LOSS_AVG,S_MAE_AVG,S_QUOTE_OBSERVATIONS,S_SHADOW_CONTROL_ELIGIBLE,
0845 |     S_FIXED_LOT,S_BE15_R,S_MODE,S_PROTECT14,S_BE_UNFULFILLED,S_PARTIAL_UNFULFILLED,S_EMPTY_PATHS,
0846 |     S_MONTH_DD_MAX,S_ADD_FILLS,S_ADD_CYCLES,S_ADD_NET,S_ADD_REJECTS,S_RISK_BLOCKS,S_STOP_FACTOR,
0847 |     S_REINVEST,S_ARM,S_INITIAL_LOTS_SUM,S_ADD_LOTS_SUM,S_PATTERN_BUY,S_PATTERN_SELLS,S_ENTRY_POLICY,S_RE
INVEST_FRACTION,S_REINVEST_CAP,S_CAPITAL_MODE,S_MATCHED_CONTROL,S_DETAIL_EXPORT,S_BAR_ROWS,S_RISK_PERC
ENT,S_MARGIN_CAP_PERCENT,S_DONCHIAN_MODE,S_SOURCE_CONTROL180,S_FIXED_MODE,S_TARGET_MODE,S_TARGET_VALUE
,S_TARGET_DISTANCE,S_NOMINAL_TARGET_R,S_TARGET_REJECTS,S_TARGET_ADJUSTMENTS,S_TARGET_FAILURES,S_FIXED
VOLUME_REJECTS,S_HP_CANDIDATES,S_HP_EXTRA_ELIGIBLE,S_HP_BASE_SELECTED,S_HP_PIVOT_SELECTED,S_HP_PIVOT_F
ILLS,S_HP_DATA_FAILURES,S_HP_POSITION_BLOCKS,S_HP_CATCHUP_IGNORED,S_COUNT };
0848 | const int MONTH_FIELDS=18;
0849 | const int SHADOW_FIELDS=8;
0850 | struct PECycle {
0851 |     int number,origin,side,legs,adds;
0852 |     int entryPath; HPSignal entryPivot;
0853 |     ulong pid,ticket,partialDeal,partialOrder;
0854 |     ulong pids[3]; double fills[3],volumes[3],expectedStops[3];
0855 |     datetime start,finish,partialTime,beTime;
0856 |     double quotedEntry,quotedTarget,targetDistance,targetProfitEstimate; int targetAdjustments;
0857 |     double entry,stop,target,volume,initialRisk,balanceBefore,beTrigger,liveStop,peakRisk,desiredStop;

```

```

0858 | double gross,swap, costs,net,inVolume,outVolume,finalPrice,partialPrice,partialVolume;
0859 | long finalReason;
0860 | bool closed,partialEnabled,finalObserved,addPending; long addNextMs,addAttempts;
0861 | PERequest be,part;
0862 | EVOFeatures entryFeatures; int entryPolicy;
0863 | PEPATH path; double featureADXChange,featureADX,featureATR,featureDistance,featureSlope; datetime
    | signalTime;
0864 | };
0865 | struct PEMonth {
0866 |     int key; long bars,signals,opened,closed,bes,partials;
0867 |     double cycleNet,bookedNet;
0868 |     datetime firstTick,lastTick;
0869 |     PEEquityPath path;
0870 | };
0871 | PEProfile profile; EVOProfile evo; EVOFeatures features; DCPProfile dc;
0872 | HPState hpState; HPSignal hpSignal; HPDecision hpRoute;
0873 | bool hpDataReady=false; string hpStatus="NOT_STARTED"; int hpProcessedThisBar=0;
0874 | long hpCandidates=0,hpExtraEligible=0,hpBaseSelected=0,hpPivotSelected=0,hpPivotFills=0,hpDataFailures
    | =0,hpPositionBlocks=0,hpCatchupIgnored=0;
0875 | ENUM_TIMEFRAMES signalTF=PERIOD_M30;
0876 | bool channelReady=false; double channelUpper=0,channelLower=0;
0877 | datetime channelOldest=0,channelNewest=0,channelClosedAt=0;
0878 | int channelBars=0; string channelStatus="NOT_REQUESTED";
0879 | double sizedLot=0,sizingRiskBudget=0,sizingMarginBudget=0,sizingMargin=0;
0880 | long targetRejects=0,targetAdjustments=0,targetFailures=0,fixedVolumeRejects=0;
0881 | double plannedTargetDistance=0,plannedTargetRiskRatio=0,plannedTargetProfit=0;
0882 | double minimumLotRiskMoney=0,minimumLotRiskPercent=0,minimumEquityForLot=0,minimumLotMargin=0;
0883 | CTrade trade;
0884 | PECycle cycles[];
0885 | PEMonth months[];
0886 | int active=-1;
0887 | int
    | hEMA=INVALID_HANDLE,h50=INVALID_HANDLE,h200=INVALID_HANDLE,hADX=INVALID_HANDLE,hATR=INVALID_HANDLE;
0888 | int barsFile=INVALID_HANDLE,eventsFile=INVALID_HANDLE,optFile=INVALID_HANDLE,optMonths=INVALID_HANDLE,
    | optShadow=INVALID_HANDLE,optFiles=INVALID_HANDLE;
0889 | string folder="",optFolder="",lastReason="";
0890 | bool ioFailure=false; long barRows=0;
0891 | bool initialized=false,integrity=true,haltEntries=false,closeEmergency=false,finalizing=false;
0892 | int errorCode=0,lastRetcode=0;
0893 | datetime firstEval=0,lastEval=0,firstReady=0,lastBar=0,lastErrorTime=0,nextMonth=0;
0894 | long barsSeen=0,signals=0,counters[G_COUNT],lastEmergencyMs=0;
0895 | long partialDefers=0,beDefers=0,sessionDefers=0,stopDefers=0,ambiguousPartials=0,unmatchedDeals=0;
0896 | int liveMonth=-1;
0897 | double priorEquity=0,priorBalance=0,oldSMA200=0,maxLots=0,maxRisk=0,maxRiskPct=0,maxMargin=0;
0898 | double initialDeposit=0,previousClose=0,pivotLevel=0,pivotSwing=0;
0899 | int pivotSide=0; datetime pivotTime=0,lastPivotUsed=0;
0900 | long addRejects=0,riskBlocks=0,patternBuys=0,patternSells=0;
0901 | bool sessionKnown=false; int sessionDay[128],sessionFrom[128],sessionTo[128],sessionCount=0;
0902 | ulong receivedPasses[];
0903 |
0904 | string TS(const datetime t) { return t>0 ? TimeToString(t,TIME_DATE|TIME_SECONDS) : ""; }
0905 | string N(const double v) { return MathIsValidNumber(v) ? DoubleToString(v,8) : ""; }
0906 | string I(const long v) { return (string)v; }
0907 | string U(const ulong v) { return (string)v; }
0908 | long NowMs() { MqlTick q; if(SymbolInfoTick(_Symbol,q) && q.time_msc>0) return q.time_msc; return
    | (long)TimeCurrent()*1000; }
0909 | string EntryName(const int k)
0910 | {
0911 |     string names[10]={"BASE","ADX_SUBINDO","ADX25","MOMENTO_3_10","DIST_1ATR","ATR_SEM_CHOQUE",
0912 |         "PULLBACK_PROXIMO","RETOMADA_CEDO","PIVO_FILTRADO","ADX_MACD_DIST"};
0913 |     return k>=0 && k<10 ? names[k] : "INVALID";
0914 | }
0915 | string DCName(const int id)
0916 | {
0917 |     string names[3]={"HERMES_REFERENCIA","HERMES_PIVO_CONTINUIDADE","HERMES_PIVO_INICIO"};
0918 |     return id>=1 && id<=3 ? names[id-1] : "INVALID";
0919 | }
0920 | string ProfileName(const int id) { return DCName(id); }
0921 | string TFName(const int id) { return "M30"; }

```

```

0922 | string TargetUnitName() { return dc.targetMode==1 ? "QUOTE_PRICE" : (dc.targetMode==2 ? "MT5_POINT" :
    | "INITIAL_STOP_R"); }
0923 | string TargetLabel() { return dc.targetMode==1 ? "20.00 unidades de preco" : (dc.targetMode==2 ? "20
    | pontos MT5" : "5R"); }
0924 | bool LoadDonchian(const datetime cutoff)
0925 | {
0926 |     channelReady=false; channelUpper=0; channelLower=0; channelBars=0;
0927 |     channelOldest=0; channelNewest=0; channelClosedAt=0; channelStatus="W1_HISTORY_PENDING";
0928 |     // Excludes the W1 candle containing the CLOSED trigger candle, never its own high.
0929 |     int containing=iBarShift(_Symbol,PERIOD_W1,cutoff,false);
0930 |     if(containing<0) return false;
0931 |     int shift=containing+1;
0932 |     MqlRates w[]; ArraySetAsSeries(w,true);
0933 |     int copied=CopyRates(_Symbol,PERIOD_W1,shift,52,w); channelBars=(int)MathMax(0,copied);
0934 |     if(copied!=52) return false;
0935 |     channelClosedAt=iTime(_Symbol,PERIOD_W1,shift-1);
0936 |     channelNewest=w[0].time; channelOldest=w[51].time;
0937 |     if(channelClosedAt<=channelNewest || channelClosedAt>cutoff) { channelStatus="NON_CAUSAL_WINDOW";
    | return false; }
0938 |     double hi[52],lo[52];
0939 |     for(int i=0;i<52;i++) {
0940 |         hi[i]=w[i].high; lo[i]=w[i].low;
0941 |         // A missing full week must be diagnosed; DST offsets remain allowed.
0942 |         if(i>0 && (w[i-1].time-w[i].time<5*86400 || w[i-1].time-w[i].time>9*86400)) {
0943 |             channelStatus="W1_GAP"; return false;
0944 |         }
0945 |     }
0946 |     channelReady=DCWindow(hi,lo,52,(long)channelClosedAt,(long)cutoff,channelUpper,channelLower);
0947 |     channelStatus=channelReady ? "READY" : "INVALID_W1_VALUES"; return channelReady;
0948 | }
0949 | bool ValidRunTag()
0950 | {
0951 |     int n=StringLen(InpRunTag); if(n<1 || n>32) return false;
0952 |     for(int i=0;i<n;i++) { ushort c=StringGetCharacter(InpRunTag,i);
0953 |         if(!((c>=48 && c<=57) || (c>=65 && c<=90) || (c>=97 && c<=122) || c==95)) return false;
0954 |     }
0955 |     return true;
0956 | }
0957 | string RunRoot() { return "Hermes_Pivos_Lab_200\\"+InpRunTag; }
0958 | string Stamp() { string s=TimeToString(TimeLocal(),TIME_DATE|TIME_SECONDS); StringReplace(s,":","-");
    | StringReplace(s," ","_"); return s+"_"+U(GetMicrosecondCount()); }
0959 | string Q(string s) { StringReplace(s,"\\","\\\\"); StringReplace(s,"\\r"," "); StringReplace(s,"\\n","
    | "); return "\\\""+s+"\\\""; }
0960 | void Cell(string &s,const string v) { if(s!="") s+=";"; s+=Q(v); }
0961 | void Row(const int f,const string s)
0962 | { if(f!=INVALID_HANDLE && FileWriteString(f,s+"\\r\\n")==0) { ioFailure=true; Print("HP200 CSV write
    | failed: ",GetLastError()); } }
0963 | int OpenText(const string f) { return FileOpen(f,FILE_WRITE|FILE_TXT|FILE_ANSI|FILE_COMMON,0,CP_UTF8);
    | }
0964 | void KV(const int f,const string k,const string v) { string row=""; Cell(row,k); Cell(row,v);
    | Row(f,row); }
0965 | int MonthIndex(const datetime t)
0966 | {
0967 |     MqlDateTime dt; TimeToStruct(t,dt); int key=dt.year*100+dt.mon;
0968 |     for(int i=0;i<ArraySize(months);i++) if(months[i].key==key) return i;
0969 |     int n=ArraySize(months); ArrayResize(months,n+1); ZeroMemory(months[n]); months[n].key=key; return
    | n;
0970 | }
0971 | void Event(const string kind,const int index,const string detail,const long ret=0)
0972 | {
0973 |     int number=index>=0 && index<ArraySize(cycles) ? cycles[index].number : 0;
0974 |     int origin=index>=0 && index<ArraySize(cycles) ? cycles[index].origin : 0;
0975 |     string s=""; Cell(s,TS(TimeCurrent())); Cell(s,I(InpCase)); Cell(s,I(number)); Cell(s,I(origin));
0976 |     Cell(s,kind); Cell(s,I(ret)); Cell(s,detail); Row(eventsFile,s);
0977 |     if(kind=="ERROR" || kind=="PARTIAL_DEFERRED" || kind=="BE_DEFERRED" || kind=="AMBIGUOUS_PARTIAL")
0978 |         Print("HP200 ",kind," case=",InpCase," cycle=",number," ret=",ret," ",detail);
0979 | }
0980 | void Invalid(const int code,const string reason,const bool emergency=false)
0981 | {

```

```

0982 | integrity=false; haltEntries=true; closeEmergency=closeEmergency || emergency;
0983 | if(errorCode==0) errorCode=code;
0984 | if(reason!=lastReason) { lastReason=reason; Event("ERROR",active,reason,lastRetcode); }
0985 | }
0986 | int CycleByPid(const ulong pid)
0987 | { if(pid==0) return -1; for(int i=ArraySize(cycles)-1;i>=0;i--) for(int j=0;j<cycles[i].legs;j++)
0988 | if(cycles[i].pids[j]==pid) return i; return -1; }
0989 | int LegByPid(const int n,const ulong pid)
0990 | { for(int j=0;j<cycles[n].legs;j++) if(cycles[n].pids[j]==pid) return j; return -1; }
0991 | bool OwnSelected()
0992 | {
0993 |     return PositionGetString(POSITION_SYMBOL)==_Symbol &&
0994 |     (CycleByPid((ulong)PositionGetInteger(POSITION_IDENTIFIER))>=0 ||
0995 |     (ulong)PositionGetInteger(POSITION_MAGIC)==InpMagic);
0996 | }
0997 | int OwnPositions(double &volume,ulong &ticket)
0998 | {
0999 |     int count=0; volume=0; ticket=0;
1000 |     for(int i=PositionsTotal()-1;i>=0;i--) {
1001 |         ulong t=PositionGetTicket(i); if(t==0 || !OwnSelected()) continue;
1002 |         count++; volume+=PositionGetDouble(POSITION_VOLUME); ticket=t;
1003 |     }
1004 |     return count;
1005 | }
1006 | bool SymbolExposure()
1007 | {
1008 |     for(int i=PositionsTotal()-1;i>=0;i--) if(PositionGetSymbol(i)==_Symbol) return true;
1009 |     for(int i=OrdersTotal()-1;i>=0;i--) if(OrderGetTicket(i)>0 && OrderGetString(ORDER_SYMBOL)==_Symbol)
1010 |     return true;
1011 |     return false;
1012 | }
1013 | bool OwnPendingOrder()
1014 | {
1015 |     for(int i=OrdersTotal()-1;i>=0;i--) if(OrderGetTicket(i)>0 && OrderGetString(ORDER_SYMBOL)==_Symbol
1016 |     &&
1017 |     (ulong)OrderGetInteger(ORDER_MAGIC)==InpMagic) return true;
1018 |     return false;
1019 | }
1020 | void TrackEquity()
1021 | {
1022 |     datetime now=TimeCurrent(); if(now<=0) return;
1023 |     if(liveMonth<0 || now>=nextMonth) {
1024 |         liveMonth=MonthIndex(now);
1025 |         if(!months[liveMonth].path.initialized) {
1026 |             PESTartEquity(months[liveMonth].path,priorEquity,priorBalance); months[liveMonth].firstTick=now; }
1027 |         MqlDateTime dt; TimeToStruct(now,dt); dt.day=1; dt.hour=0; dt.min=0; dt.sec=0; dt.mon++;
1028 |         if(dt.mon>12) { dt.mon=1; dt.year++; } nextMonth=StructToTime(dt);
1029 |     }
1030 |     double eq=AccountInfoDouble(ACCOUNT_EQUITY),bal=AccountInfoDouble(ACCOUNT_BALANCE);
1031 |     if(!PEObserveEquity(months[liveMonth].path,eq,bal)) { Invalid(101,"Invalid monthly equity value.");
1032 |     return; }
1033 |     months[liveMonth].lastTick=now; priorEquity=eq; priorBalance=bal;
1034 | }
1035 | bool ReadValue(const int h,const int buffer,const int shift,double &v)
1036 | { double a[1]; if(shift<1 || CopyBuffer(h,buffer,shift,1,a)!=1 || a[0]==EMPTY_VALUE ||
1037 | !MathIsValidNumber(a[0])) return false; v=a[0]; return true; }
1038 | bool ReadSignal(H1Signal &s,datetime &signalTime)
1039 | {
1040 |     int required=dc.weekly ? 55 : 209;
1041 |     if(Bars(_Symbol,signalTF)<required || BarsCalculated(hEMA)<required || BarsCalculated(hADX)<required
1042 |     || BarsCalculated(hATR)<required ||
1043 |     (!dc.weekly && BarsCalculated(h200)<209)) return false;
1044 |     MqlRates r[]; ArraySetAsSeries(r,true); if(CopyRates(_Symbol,signalTF,1,25,r)!=25) return false;
1045 |     signalTime=r[0].time; s.open=r[0].open; s.high=r[0].high; s.low=r[0].low; s.close=r[0].close;
1046 |     s.previousHigh=r[1].high; s.previousLow=r[1].low;
1047 |     if(!ReadValue(hEMA,0,1,s.ema) || !ReadValue(hATR,0,1,s.atr) ||
1048 |     !ReadValue(hADX,0,1,s.adx) || !ReadValue(hADX,1,1,s.plusDI) || !ReadValue(hADX,2,1,s.minusDI))
1049 |     return false;
1050 |     if(!dc.weekly && (!ReadValue(h50,0,1,s.sma50) || !ReadValue(h200,0,1,s.sma200) ||

```

```

1042 |         !ReadValue(h50,0,6,s.oldSma50) || !ReadValue(h200,0,6,oldSMA200))) return false;
1043 |     s.lowest=r[0].low; s.highest=r[0].high; s.touched=false; s.priorTouched=false;
1044 |     for(int i=0;i<4;i++) {
1045 |         if(i<3) { s.lowest=MathMin(s.lowest,r[i].low); s.highest=MathMax(s.highest,r[i].high); }
1046 |         double ema=0; if(!ReadValue(hEMA,0,i+1,ema)) return false;
1047 |         if(r[i].low<=ema && r[i].high>=ema) { if(i<3) s.touched=true; if(i>=1) s.priorTouched=true; }
1048 |     }
1049 |     if(!ReadValue(hADX,0,2,s.oldADX)) return false;
1050 |     previousClose=r[1].close; ZeroMemory(features); features.previousClose=previousClose;
1051 |     double closes[25]; for(int j=0;j<25;j++) closes[j]=r[j].close;
1052 |     features.oscillatorValid=EV00scillator(closes,25,features.fast310,features.previousFast310,features.
signal310);
1053 |     if(s.atr>0) {
1054 |         features.trueRangeATR=MathMax(s.high-s.low,MathMax(MathAbs(s.high-previousClose),MathAbs(s.low-prev
iousClose)))/s.atr;
1055 |         features.bodyATR=(s.close-s.open)/s.atr;
1056 |     }
1057 |     features.closeLocation=s.high>s.low ? (s.close-s.low)/(s.high-s.low) : .5;
1058 |     pivotSide=0; pivotTime=0;
1059 |     MqlRates pr[]; ArraySetAsSeries(pr,false);
1060 |     if(!dc.weekly && CopyRates(_Symbol,signalTF,1,86,pr)==86) {
1061 |         double hi[86],lo[86]; for(int k=0;k<86;k++) { hi[k]=pr[k].high; lo[k]=pr[k].low; }
1062 |         int key=-1; pivotSide=PEPivotSignal(hi,lo,86,s.close,previousClose,pivotLevel,pivotSwing,key);
1063 |         if(key>=0) pivotTime=pr[key].time;
1064 |     }
1065 |     channelStatus="NOT_USED_M30_PIVOT_PROTOCOL";
1066 |     return s.atr>0;
1067 | }
1068 | // MT5 history adapter. No current candle OHLC enters the detector.
1069 | // Copy/validation is transactional: incomplete catch-up never corrupts state.
1070 | string HPPathName(const int path)
1071 | { return path==HP_BASE ? "BASE" : (path==HP_PIVOT ? "PIVOT" : "NONE"); }
1072 | bool HPSyncClosedBars(const datetime decisionOpen)
1073 | {
1074 |     HPClearSignal(hpSignal); hpProcessedThisBar=0; hpStatus="HISTORY_PENDING";
1075 |     datetime latestClosed=iTime(_Symbol,signalTF,1);
1076 |     if(latestClosed<=0 || decisionOpen<=latestClosed) return false;
1077 |     MqlRates observed[]; ArraySetAsSeries(observed,false);
1078 |     bool seed=hpState.bars_processed==0;
1079 |     int copied=0;
1080 |     if(seed) copied=CopyRates(_Symbol,signalTF,1,210,observed);
1081 |     else {
1082 |         if(hpState.expected_open<=0 || hpState.expected_open>(long)latestClosed) {
1083 |             hpStatus="CHRONOLOGY_MISMATCH"; return false;
1084 |         }
1085 |         copied=CopyRates(_Symbol,signalTF,(datetime)hpState.expected_open,latestClosed,observed);
1086 |     }
1087 |     if(copied<1 || (seed && copied!=210)) return false;
1088 |     if(observed[copied-1].time!=latestClosed ||
1089 |         (!seed && (long)observed[0].time!=hpState.expected_open)) {
1090 |         hpStatus="INCOMPLETE_CATCHUP"; return false;
1091 |     }
1092 |     HPState trial=hpState; HPSignal current; HPClearSignal(current);
1093 |     long ignored=0;
1094 |     for(int i=0;i<copied;i++) {
1095 |         HPBar closed;
1096 |         closed.time=(long)observed[i].time; closed.open=observed[i].open;
1097 |         closed.high=observed[i].high; closed.low=observed[i].low; closed.close=observed[i].close;
1098 |         long witness=i+1<copied ? (long)observed[i+1].time : (long)decisionOpen;
1099 |         HPSignal candidate;
1100 |         if(!HPStep(trial,closed,witness,candidate)) { hpStatus="INVALID_HISTORY_OR_SEQUENCE"; return false;
}
1101 |         if(i==copied-1) current=candidate;
1102 |         else if(candidate.buy && !seed) ignored++;
1103 |     }
1104 |     if(current.buy && (current.signal_time!=(long)latestClosed ||
current.available_time!=(long)decisionOpen)) {
1105 |         hpStatus="STALE_SIGNAL_REJECTED"; return false;
1106 |     }

```

```

1107 | hpState=trial; hpSignal=current; hpProcessedThisBar=copied; hpCatchupIgnored+=ignored;
1108 | hpStatus=seed ? "READY_SEED_210" : (copied>1 ? "READY_CATCHUP" : "READY");
1109 | return true;
1110 | }
1111 |
1112 |
1113 | void Record(const PE_GATE gate,const int setup,const int origin,const H1Signal &s,const MqlTick
1114 | &q,const datetime signalTime,
1115 |             const double sl=0,const double tp=0,const double risk=0,const string detail="")
1116 | {
1117 |     counters[(int)gate]++;
1118 |     if(hpSignal.buy) {
1119 |         if(gate==G_POSITION && hpRoute.path==HP_PIVOT) hpPositionBlocks++;
1120 |         string candidate="signal="+TS((datetime)hpSignal.signal_time)+";
1121 |         available="+TS((datetime)hpSignal.available_time);
1122 |         candidate+="; p1="+TS((datetime)hpSignal.p1_time)+"; p2="+TS((datetime)hpSignal.p2_time)+";
1123 |         p3="+TS((datetime)hpSignal.p3_time);
1124 |         candidate+="; original_gate="+I(hpRoute.original_gate)+"; extra_gate="+I(hpRoute.extra_gate)+";
1125 |         entry_path="+HPPathName(hpRoute.path)+"; gate="+EnumToString(gate);
1126 |         Event("PIVOT_CANDIDATE",active,candidate);
1127 |     }
1128 |     bool saveBar=InpExportAllBars || setup==0 || gate==G_NO_DATA || gate==G_HALTED ||
1129 |         (s.close>s.open && s.ema>s.sma50 && s.close>s.sma200) || gate==G_FILLED;
1130 |     string row="";
1131 |     if(saveBar && barsFile!=INVALID_HANDLE) {
1132 |         Cell(row,TS(TimeCurrent())); Cell(row,TS(signalTime)); Cell(row,I(InpCase)); Cell(row,I(origin));
1133 |         Cell(row,EnumToString(gate));
1134 |         Cell(row,I(setup)); Cell(row,I(active>=0 ? cycles[active].number : 0));
1135 |         Cell(row,N(s.open)); Cell(row,N(s.high)); Cell(row,N(s.low)); Cell(row,N(s.close));
1136 |         Cell(row,N(s.ema));
1137 |         Cell(row,N(s.sma50)); Cell(row,N(s.sma200)); Cell(row,N(s.oldSma50)); Cell(row,N(oldSMA200));
1138 |         Cell(row,N(s.adx)); Cell(row,N(s.plusDI)); Cell(row,N(s.minusDI)); Cell(row,N(s.atr));
1139 |         Cell(row,N(q.bid)); Cell(row,N(q.ask)); Cell(row,N(sl)); Cell(row,N(tp)); Cell(row,N(risk));
1140 |         Cell(row,N(AccountInfoDouble(ACCOUNT_BALANCE))); Cell(row,N(AccountInfoDouble(ACCOUNT_EQUITY)));
1141 |         Cell(row,detail);
1142 |         Cell(row,I(pivotSide)); Cell(row,N(pivotLevel)); Cell(row,N(pivotSwing)); Cell(row,TS(pivotTime));
1143 |         Cell(row,N(s.atr>0 ? (s.close-s.ema)/s.atr : 0)); Cell(row,N(s.atr>0 ? (s.sma200-oldSMA200)/s.atr :
1144 |         0));
1145 |         Cell(row,N(s.oldADX)); Cell(row,N(previousClose));
1146 |         bool pc=pivotSide!=0 && s.adx>=20 && s.adx>s.oldADX &&
1147 |             (pivotSide==1 ? (s.plusDI>s.minusDI && s.close>s.open && s.close>s.ema) : (s.minusDI>s.plusDI &&
1148 |             s.close<s.open && s.close<s.ema));
1149 |         int sd=0; bool baseOK=H1EvaluateProfile(s,B_PRIOR_PULLBACK,true,20.0,sd)==0 && sd==1 &&
1150 |             PEDistance(q.ask,s.ema,s.atr,InpMinEntryATR);
1151 |         Cell(row,I(pc)); Cell(row,I(baseOK && s.sma200>oldSMA200)); Cell(row,I(baseOK));
1152 |         Cell(row,N(s.atr>0 ? (q.ask-q.bid)/s.atr : 0)); Cell(row,N(s.atr>0 ? (s.close-s.open)/s.atr : 0));
1153 |         Cell(row,I(evo.entryPolicy)); Cell(row,I(features.oscillatorValid)); Cell(row,N(features.fast310));
1154 |         Cell(row,N(features.previousFast310)); Cell(row,N(features.signal310));
1155 |         Cell(row,N(features.trueRangeATR));
1156 |         Cell(row,N(features.closeLocation));
1157 |         Cell(row,TFName(InpCase)); Cell(row,I(dc.channelMode)); Cell(row,I(channelReady));
1158 |         Cell(row,I(channelBars));
1159 |         Cell(row,N(channelUpper)); Cell(row,N(channelLower)); Cell(row,TS(channelOldest));
1160 |         Cell(row,TS(channelNewest)); Cell(row,TS(channelClosedAt)); Cell(row,channelStatus);
1161 |         Cell(row,N(dc.riskPercent)); Cell(row,N(dc.fixedLot ? 0 : InpMaxMarginPct)); Cell(row,N(sizedLot));
1162 |         Cell(row,N(sizingRiskBudget));
1163 |         Cell(row,N(sizingMarginBudget)); Cell(row,N(sizingMargin));
1164 |         Cell(row,N(AccountInfoDouble(ACCOUNT_MARGIN_FREE)));
1165 |         Cell(row,N(AccountInfoDouble(ACCOUNT_MARGIN))); Cell(row,N(minimumLotRiskMoney));
1166 |         Cell(row,N(minimumLotRiskPercent));
1167 |         Cell(row,N(minimumEquityForLot)); Cell(row,N(minimumLotMargin));
1168 |         Cell(row,I(dc.sourceControl180)); Cell(row,I(dc.matchedControl)); Cell(row,I(dc.fixedLot));
1169 |         Cell(row,TargetUnitName());
1170 |         Cell(row,N(dc.targetMode==0 ? 5 : 20)); Cell(row,N(plannedTargetDistance));
1171 |         Cell(row,N(plannedTargetRiskRatio)); Cell(row,N(plannedTargetProfit));
1172 |         Cell(row,I(hpDataReady)); Cell(row,hpStatus); Cell(row,I(hpSignal.buy));
1173 |         Cell(row,I(hpRoute.original_gate)); Cell(row,I(hpRoute.extra_gate));
1174 |         Cell(row,HPPathName(hpRoute.path));
1175 |         Cell(row,TS((datetime)hpSignal.signal_time)); Cell(row,TS((datetime)hpSignal.available_time));

```

```

1157 | Cell(row,N(hpSignal.stop_f1)); Cell(row,N(hpSignal.reference_price));
      | Cell(row,N(hpSignal.pullback_f2));
1158 | Cell(row,TS((datetime)hpSignal.p1_time)); Cell(row,TS((datetime)hpSignal.p2_time));
      | Cell(row,TS((datetime)hpSignal.p3_time));
1159 | Cell(row,TS((datetime)hpSignal.p1_confirmation_time));
      | Cell(row,TS((datetime)hpSignal.p2_confirmation_time));
      | Cell(row,TS((datetime)hpSignal.p3_confirmation_time));
1160 | Cell(row,TS((datetime)hpSignal.p1_available_time));
      | Cell(row,TS((datetime)hpSignal.p2_available_time));
      | Cell(row,TS((datetime)hpSignal.p3_available_time));
1161 | Cell(row,TS((datetime)hpSignal.pattern_available_time)); Cell(row,I(hpProcessedThisBar));
1162 | Cell(row,I(hpState.bars_processed)); Cell(row,TS((datetime)hpState.last_time));
      | Cell(row,TS((datetime)hpState.expected_open));
1163 | Row(barsFile,row); barRows++;
1164 | }
1165 | if(barsSeen%24==0 && barsFile!=INVALID_HANDLE) FileFlush(barsFile);
1166 | if(MQLInfoInteger(MQL_VISUAL_MODE)) Comment("HERMES PIVOS 2.00 | TESTADOR | ",ProfileName(InpCase),
1167 | "\n",TFName(InpCase)," | Lote base ",DoubleToString(InpFixedLot,2)," | Alvo ",TargetLabel()," |
      | Gestao por perfil",
1168 | "\nCiclos ",ArraySize(cycles)," | ",EnumToString(gate)," | Origem ",origin,
1169 | "\nIntegridade ",integrity ? "OK" : "ERRO: ver events.csv");
1170 | }
1171 | void ObserveOpenPath()
1172 | {
1173 |   if(active<0) return; double volume=0; ulong ticket=0;
1174 |   if(OwnPositions(volume,ticket)<1) return;
1175 |   MqlTick q; if(SymbolInfoTick(_Symbol,q) && !PEObserve(cycles[active].path,cycles[active].side==1 ?
      | q.bid : q.ask)) Invalid(102,"Invalid quote for excursion measurement.");
1176 | }
1177 | // Closing ownership follows POSITION_IDENTIFIER, including broker/tester exits with magic=0.
1178 | // At tester end history contains executed transactions only, not future market bars.
1179 | bool AggregateCycle(const int index)
1180 | {
1181 |   datetime finish=finalizing ? TimeCurrent()+86400 : TimeCurrent();
1182 |   if(!HistorySelect(cycles[index].start,finish)) return false;
1183 |   cycles[index].gross=0; cycles[index].swap=0; cycles[index].costs=0;
1184 |   cycles[index].inVolume=0; cycles[index].outVolume=0; cycles[index].finish=0;
1185 |   for(int i=0;i<HistoryDealsTotal();i++) {
1186 |     ulong d=HistoryDealGetTicket(i);
1187 |     if(d==0 || HistoryDealGetString(d,DEAL_SYMBOL)!=_Symbol ||
      | CycleByPid((ulong)HistoryDealGetInteger(d,DEAL_POSITION_ID))!=index) continue;
1188 |     long entry=HistoryDealGetInteger(d,DEAL_ENTRY),side=HistoryDealGetInteger(d,DEAL_TYPE);
1189 |     if(side!=DEAL_TYPE_BUY && side!=DEAL_TYPE_SELL) continue;
1190 |     cycles[index].gross+=HistoryDealGetDouble(d,DEAL_PROFIT);
      | cycles[index].swap+=HistoryDealGetDouble(d,DEAL_SWAP);
1191 |     cycles[index].costs+=HistoryDealGetDouble(d,DEAL_COMMISSION)+HistoryDealGetDouble(d,DEAL_FEE);
1192 |     double v=HistoryDealGetDouble(d,DEAL_VOLUME);
1193 |     long openingType=cycles[index].side==1 ? DEAL_TYPE_BUY : DEAL_TYPE_SELL;
1194 |     if(entry==DEAL_ENTRY_IN && side==openingType) cycles[index].inVolume+=v;
1195 |     else if((entry==DEAL_ENTRY_OUT || entry==DEAL_ENTRY_OUT_BY) && side!=openingType) {
1196 |       cycles[index].outVolume+=v; cycles[index].finish=(datetime)HistoryDealGetInteger(d,DEAL_TIME);
1197 |       cycles[index].finalPrice=HistoryDealGetDouble(d,DEAL_PRICE);
      | cycles[index].finalReason=HistoryDealGetInteger(d,DEAL_REASON);
1198 |     } else return false;
1199 |   }
1200 |   cycles[index].net=cycles[index].gross+cycles[index].swap+cycles[index].costs;
1201 |   return cycles[index].inVolume>0;
1202 | }
1203 | void FinishCycle(const int n)
1204 | {
1205 |   if(cycles[n].closed) return;
1206 |   cycles[n].closed=true;
1207 |   // Include the real final fill, never an OHLC high after the position was closed.
1208 |   if(!cycles[n].finalObserved && cycles[n].finalPrice>0) {
1209 |     if(!PEObserve(cycles[n].path,cycles[n].finalPrice)) Invalid(103,"Invalid final excursion
      | observation.");
1210 |     cycles[n].finalObserved=true;
1211 |   }
1212 |   int mi=MonthIndex(cycles[n].finish); months[mi].closed++; months[mi].cycleNet+=cycles[n].net;

```

```

1213 |   Event("CYCLE_CLOSED",n,"net="+N(cycles[n].net)+"; MFE_R="+N(cycles[n].path.maxR));
1214 | }
1215 | void SyncCycle()
1216 | {
1217 |   if(active<0) return; double volume=0; ulong ticket=0; int count=OwnPositions(volume,ticket);
1218 |   maxLots=MathMax(maxLots,volume); maxMargin=MathMax(maxMargin,AccountInfoDouble(ACCOUNT_MARGIN));
1219 |   if(count>0 || OwnPendingOrder()) return;
1220 |   int n=active;
1221 |   if(!AggregateCycle(n) || MathAbs(cycles[n].inVolume-cycles[n].outVolume)>1e-8) { Invalid(104,"Closed
cycle cannot be reconciled by position identifier."); return; }
1222 |   FinishCycle(n); active=-1;
1223 | }
1224 |
1225 | // Execution is ticket-based. Volume changes are never blindly retried after
1226 | // an ambiguous reply. Baseline broker SL/TP remain while an amendment is deferred.
1227 | void LoadSessions()
1228 | {
1229 |   sessionCount=0; sessionKnown=false;
1230 |   for(int d=0;d<7;d++) for(uint i=0;i<18;i++) {
1231 |     datetime from=0,to=0; if(!SymbolInfoSessionTrade(_Symbol,(ENUM_DAY_OF_WEEK)d,i,from,to)) break;
1232 |     if(sessionCount>=128) break;
1233 |     sessionKnown=true; sessionDay[sessionCount]=d; sessionFrom[sessionCount]=(int)((long)from%86400);
1234 |     sessionTo[sessionCount]=(int)((long)to%86400); sessionCount++;
1235 |   }
1236 | }
1237 | bool SessionOpen()
1238 | {
1239 |   if(!sessionKnown) return true; // server remains authoritative when schedule is unavailable
1240 |   MqlDateTime dt; TimeToStruct(TimeCurrent(),dt); int sec=dt.hour*3600+dt.min*60+dt.sec;
1241 |   for(int i=0;i<sessionCount;i++) {
1242 |     int a=sessionFrom[i],b=sessionTo[i],day=sessionDay[i];
1243 |     if(a==b && day==dt.day_of_week) return true;
1244 |     if(b>a && day==dt.day_of_week && sec>=a && sec<b) return true;
1245 |     if(b<a && ((day==dt.day_of_week && sec>=a) || ((day+1)%7==dt.day_of_week && sec<b))) return true;
1246 |   }
1247 |   return false;
1248 | }
1249 | bool CanRequest()
1250 | { return SessionOpen() && MQLInfoInteger(MQL_TRADE_ALLOWED) &&
TerminalInfoInteger(TERMINAL_TRADE_ALLOWED) &&
1251 |   AccountInfoInteger(ACCOUNT_TRADE_ALLOWED) && AccountInfoInteger(ACCOUNT_TRADE_EXPERT); }
1252 | ENUM_ORDER_TYPE_FILLING Filling()
1253 | {
1254 |   long flags=SymbolInfoInteger(_Symbol,SYMBOL_FILLING_MODE);
1255 |   if((flags&SYMBOL_FILLING_FOK)!=0) return ORDER_FILLING_FOK;
1256 |   if((flags&SYMBOL_FILLING_IOC)!=0) return ORDER_FILLING_IOC;
1257 |   return ORDER_FILLING_RETURN;
1258 | }
1259 | bool PositionForLeg(const int n,const int leg,ulong &ticket)
1260 | {
1261 |   ticket=0;
1262 |   for(int i=PositionsTotal()-1;i>=0;i--) {
1263 |     ulong t=PositionGetTicket(i);
1264 |     if(t>0 && PositionGetString(POSITION_SYMBOL)==_Symbol &&
1265 |       (ulong)PositionGetInteger(POSITION_IDENTIFIER)==cycles[n].pids[leg]) { ticket=t; return true; }
1266 |   }
1267 |   return false;
1268 | }
1269 | // Only narrows a configured fixed-distance TP. Never widens target or SL.
1270 | // The broker SL/TP stays attached during a rejected amendment. Failures halt
1271 | // entries and request a ticket-based protected close, recorded as invalid run.
1272 | void EnforceTargetCap()
1273 | {
1274 |   if(active<0 || dc.targetMode==0 || closeEmergency) return;
1275 |   int n=active; ulong ticket=0; if(!PositionForLeg(n,0,ticket)) return;
1276 |   double tick=SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_SIZE);
1277 |   double liveTarget=PositionGetDouble(POSITION_TP),sl=PositionGetDouble(POSITION_SL);
1278 |   double distance=cycles[n].side*(liveTarget-cycles[n].entry);
1279 |   double tolerance=MathMax(1e-10,tick*1e-7);

```

```

1280 | if(liveTarget<=0 || distance<=0 || sl<=0) {
1281 |     targetFailures++; Invalid(330,"Invalid fixed-target protection after fill.",true); return;
1282 | }
1283 | if(distance<=cycles[n].targetDistance+tolerance) return;
1284 | double allowed=0;
1285 | if(!HermesTargetPrice(cycles[n].side,cycles[n].entry,cycles[n].targetDistance,tick,allowed)) {
1286 |     targetFailures++; Invalid(331,"Actual fill cannot represent requested target cap.",true); return;
1287 | }
1288 | MqlTick q;
1289 | if(!CanRequest() || !SymbolInfoTick(_Symbol,q) || q.bid<=0 || q.ask<q.bid) {
1290 |     targetFailures++; Invalid(332,"Target cap cannot be confirmed; protected close requested.",true);
return;
1291 | }
1292 | double cp=cycles[n].side==1 ? q.bid : q.ask;
1293 | double minDistance=MathMax(SymbolInfoInteger(_Symbol,SYMBOL_TRADE_STOPS_LEVEL),SymbolInfoInteger(_Sy
mbol,SYMBOL_TRADE_FREEZE_LEVEL))*_Point;
1294 | if(cycles[n].side*(allowed-cp)<minDistance || cycles[n].side*(cp-sl)<minDistance) {
1295 |     targetFailures++; Invalid(333,"Target cap amendment blocked by stops/freeze; protected close
requested.",true); return;
1296 | }
1297 | MqlTradeRequest req={}; MqlTradeResult res={}; req.action=TRADE_ACTION_SLTP; req.position=ticket;
1298 | req.symbol=_Symbol; req.magic=InpMagic; req.sl=sl; req.tp=NormalizeDouble(allowed,_Digits);
1299 | bool sent=OrderSend(req,res);
1300 | if(!PositionSelectByTicket(ticket)) { SyncCycle(); return; }
1301 | double confirmed=PositionGetDouble(POSITION_TP);
1302 | double confirmedDistance=cycles[n].side*(confirmed-cycles[n].entry);
1303 | if(confirmed>0 && confirmedDistance>0 && confirmedDistance<=cycles[n].targetDistance+tolerance &&
MathAbs(confirmed-req.tp)<=tolerance && MathAbs(PositionGetDouble(POSITION_SL)-sl)<=tolerance) {
1304 |     cycles[n].target=confirmed; cycles[n].targetAdjustments++; targetAdjustments++;
1305 |     Event("TARGET_CAP_CONFIRMED",n,"quoted_entry="+N(cycles[n].quotedEntry)+";
fill="+N(cycles[n].entry)+"; TP="+N(confirmed),res.retcode);
1306 |     return;
1307 | }
1308 | }
1309 | lastRetcode=(int)res.retcode; lastErrorTime=TimeCurrent(); targetFailures++;
1310 | Event("TARGET_CAP_FAILED",n,"No second volume entry; SL remains; sent="+I(sent),res.retcode);
1311 | Invalid(334,"TP amendment unconfirmed after fill; protected close requested.",true);
1312 | }
1313 | double ActiveRisk()
1314 | {
1315 |     if(active<0) return 0;
1316 |     double risk=0;
1317 |     for(int i=0;i<cycles[active].legs;i++) {
1318 |         ulong t=0; if(!PositionForLeg(active,i,t)) continue;
1319 |         double p=0,sl=PositionGetDouble(POSITION_SL);
1320 |         ENUM_ORDER_TYPE type=cycles[active].side==1 ? ORDER_TYPE_BUY : ORDER_TYPE_SELL;
1321 |         if(sl<=0 ||
!OrderCalcProfit(type,_Symbol,PositionGetDouble(POSITION_VOLUME),cycles[active].fills[i],sl,p)) return
-1;
1322 |         risk+=MathMax(0,-p); // BE is valid zero risk; locked gains do not finance extra risk here.
1323 |     }
1324 |     return risk;
1325 | }
1326 | void CheckProtection()
1327 | {
1328 |     if(active<0) return; int n=active; double tick=SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_SIZE);
1329 |     EnforceTargetCap(); if(closeEmergency || active<0) return;
1330 |     for(int i=PositionsTotal()-1;i>=0;i--) {
1331 |         ulong ticket=PositionGetTicket(i); if(ticket==0 || !OwnSelected()) continue;
1332 |         int leg=LegByPid(n,(ulong)PositionGetInteger(POSITION_IDENTIFIER));
1333 |         if(leg<0) { Invalid(301,"Untracked position in active campaign.",true); return; }
1334 |         double sl=PositionGetDouble(POSITION_SL),tp=PositionGetDouble(POSITION_TP),v=PositionGetDouble(POSI
TION_VOLUME);
1335 |         if(PositionGetInteger(POSITION_TYPE)!=(cycles[n].side==1 ? POSITION_TYPE_BUY : POSITION_TYPE_SELL)
|| sl<=0 ||
!PENear(tp,cycles[n].target,tick) || cycles[n].side*(sl-cycles[n].expectedStops[leg])<-tick*.51)
1336 |         {
1337 |             Invalid(302,"Missing, widened or unexpected SL/TP.",true); return;
1338 |         }
1339 |         if(cycles[n].side*(sl-cycles[n].desiredStop)>tick*.51) { Invalid(303,"SL exceeds the requested

```

```

    protection level."); return; }
1340 |     cycles[n].expectedStops[leg]=sl;
1341 |     double wanted=cycles[n].volumes[leg];
1342 |     bool halfAllowed=leg==0 && cycles[n].partialEnabled && cycles[n].part.attempts>0;
1343 |     if(MathAbs(v-wanted)>1e-8 && !(halfAllowed && MathAbs(v-wanted*.5)<1e-8)) {
1344 |         Invalid(304,"Unexpected remaining volume; no further volume request allowed."); return;
1345 |     }
1346 | }
1347 | double risk=ActiveRisk();
1348 | if(risk<0) { Invalid(305,"Cannot value remaining stop risk.",true); return; }
1349 | cycles[n].peakRisk=MathMax(cycles[n].peakRisk,risk); maxRisk=MathMax(maxRisk,risk);
1350 | double balance=AccountInfoDouble(ACCOUNT_BALANCE); if(balance>0)
maxRiskPct=MathMax(maxRiskPct,100*risk/balance);
1351 | if(risk>cycles[n].initialRisk*profile.capRisk+.05) Invalid(306,"Stop-risk budget exceeded after
fill.",true);
1352 | }
1353 | void Defer(PERequest &r,const string kind,const long ret,const string detail)
1354 | {
1355 |     PERejected(r,NowMs(),ret); lastRetcode=(int)ret; lastErrorTime=TimeCurrent();
1356 |     if(kind=="PARTIAL_DEFERRED") partialDefers++; else beDefers++;
1357 |     if(ret==10018) sessionDefers++; if(ret==10016 || ret==10029) stopDefers++;
1358 |     Event(kind,active,detail,ret);
1359 | }
1360 | bool MoveStops(const double desired)
1361 | {
1362 |     if(active<0) return false; int n=active; long now=NowMs();
1363 |     if(cycles[n].side*(desired-cycles[n].desiredStop)>1e-8) {
1364 |         cycles[n].desiredStop=desired; cycles[n].be.done=false; cycles[n].be.uncertain=false;
PEArm(cycles[n].be,now);
1365 |     }
1366 |     bool all=true; double tick=SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_SIZE);
1367 |     for(int leg=0;leg<cycles[n].legs;leg++) {
1368 |         ulong ticket=0; if(!PositionForLeg(n,leg,ticket)) continue;
1369 |         double sl=PositionGetDouble(POSITION_SL);
1370 |         if(cycles[n].side*(sl-desired)>=-tick*.5) { cycles[n].expectedStops[leg]=sl; continue; }
1371 |         all=false;
1372 |         if(!PERequestDue(cycles[n].be,now)) continue;
1373 |         if(cycles[n].be.attempts>=24) { cycles[n].be.uncertain=true; Invalid(310,"SL amendment retry budget
exhausted."); return false; }
1374 |         if(!CanRequest()) { Defer(cycles[n].be,"BE_DEFERRED",10018,"Session or trading permission
unavailable."); continue; }
1375 |         MqlTick q; if(!SymbolInfoTick(_Symbol,q) || q.bid<=0 || q.ask<q.bid) continue;
1376 |         double cp=cycles[n].side==1 ? q.bid : q.ask;
1377 |         double distance=MathMax(SymbolInfoInteger(_Symbol,SYMBOL_TRADE_STOPS_LEVEL),SymbolInfoInteger(_Symb
ol,SYMBOL_TRADE_FREEZE_LEVEL))*_Point+tick;
1378 |         if(cycles[n].side*(cp-desired)<distance || cycles[n].side*(cycles[n].target-cp)<distance ||
(SymbolInfoInteger(_Symbol,SYMBOL_TRADE_FREEZE_LEVEL)>0 && cycles[n].side*(cp-sl)<distance)) {
1379 |             Defer(cycles[n].be,"BE_DEFERRED",10029,"Stop/target freeze distance; original SL remains.");
1380 |             continue;
1381 |         }
1382 |         MqlTradeRequest req={}; MqlTradeResult res={}; req.action=TRADE_ACTION_SLTP; req.position=ticket;
1383 |         req.symbol=_Symbol; req.magic=InpMagic; req.sl=NormalizeDouble(desired,_Digits);
req.tp=cycles[n].target;
1384 |         PEAttempt(cycles[n].be,now); bool sent=OrderSend(req,res); cycles[n].be.lastRet=res.retcode;
1385 |         bool exists=PositionSelectByTicket(ticket);
1386 |         if(exists && cycles[n].side*(PositionGetDouble(POSITION_SL)-desired)>=-tick*.5) {
1387 |             cycles[n].expectedStops[leg]=PositionGetDouble(POSITION_SL); cycles[n].be.nextTryMs=now;
1388 |             Event("SL_CONFIRMED",n,"ticket="+U(ticket)+" ; sl="+N(desired),res.retcode);
1389 |         } else if(!exists) continue;
1390 |         else {
1391 |             int cls=PERetcodeClass(res.retcode);
1392 |             // An SLTP retry is idempotent and changes no volume; re-read before every retry.
1393 |             if(cls==3) { cycles[n].be.uncertain=true; Invalid(311,"Permanent/unknown SL amendment
rejection."); }
1394 |             Defer(cycles[n].be,"BE_DEFERRED",res.retcode,"SL not yet confirmed; sent="+I(sent));
1395 |         }
1396 |     }
1397 |     all=true; int stillOpen=0;
1398 |     for(int leg=0;leg<cycles[n].legs;leg++) { ulong t=0; if(PositionForLeg(n,leg,t)) {

```

```

1399 |     stillOpen++; if(cycles[n].side*(PositionGetDouble(POSITION_SL)-desired)<=-tick*.5) all=false;
1400 | } }
1401 | if(stillOpen==0) return false;
1402 | if(all) {
1403 |     cycles[n].be.done=true;
1404 |     if(cycles[n].beTime==0 && cycles[n].side*(desired-cycles[n].entry)>=-tick*.5) {
1405 |         cycles[n].beTime=TimeCurrent(); months[MonthIndex(TimeCurrent())].bes++;
1406 |         Event("BE_CONFIRMED",n,"SL anchored to actual primary entry or better.");
1407 |     }
1408 |     return all;
1409 | }
1410 | bool ReconcilePartial()
1411 | {
1412 |     if(active<0) return false; int n=active; if(cycles[n].part.done) return true;
1413 |     ulong ticket=0; if(!PositionForLeg(n,0,ticket)) return false;
1414 |     double live=PositionGetDouble(POSITION_VOLUME);
1415 |     if(PEVolumeState(cycles[n].volume,live)!=1 || cycles[n].part.attempts==0) return false;
1416 |     if(!HistorySelect(cycles[n].start,TimeCurrent())) return false;
1417 |     double volume=0,value=0; datetime when=0;
1418 |     for(int i=0;i<HistoryDealsTotal();i++) {
1419 |         ulong d=HistoryDealGetTicket(i);
1420 |         if((ulong)HistoryDealGetInteger(d,DEAL_POSITION_ID)!=cycles[n].pid ||
1421 | HistoryDealGetInteger(d,DEAL_ENTRY)!=DEAL_ENTRY_OUT ||
1422 | HistoryDealGetInteger(d,DEAL_REASON)!=DEAL_REASON_EXPERT ||
1423 | (ulong)HistoryDealGetInteger(d,DEAL_MAGIC)!=InpMagic) continue;
1424 |         double v=HistoryDealGetDouble(d,DEAL_VOLUME); volume+=v;
1425 |         value+=v*HistoryDealGetDouble(d,DEAL_PRICE);
1426 |         when=(datetime)HistoryDealGetInteger(d,DEAL_TIME);
1427 |     }
1428 |     if(MathAbs(volume-cycles[n].volume*.5)>1e-8) { Invalid(312,"Partial remainder has no matching
1429 | executed history."); return false; }
1430 |     cycles[n].part.done=true; cycles[n].part.uncertain=false; cycles[n].partialTime=when;
1431 |     cycles[n].partialVolume=volume; cycles[n].partialPrice=value/volume;
1432 |     months[MonthIndex(when)].partials++;
1433 |     PEArm(cycles[n].be,NowMs()); Event("PARTIAL_CONFIRMED",n,"volume="+N(volume)+"
1434 | price="+N(value/volume));
1435 |     return true;
1436 | }
1437 | void TryPartial()
1438 | {
1439 |     if(active<0) return; int n=active; long now=NowMs();
1440 |     if(ReconcilePartial()) { MoveStops(cycles[n].entry); return; }
1441 |     if(cycles[n].part.uncertain) {
1442 |         if(now-cycles[n].part.lastTryMs>60000) Invalid(313,"Unresolved partial reply; no duplicate
1443 | reduction. Broker SL remains.");
1444 |         return;
1445 |     }
1446 |     if(!PERequestDue(cycles[n].part,now)) return;
1447 |     if(cycles[n].part.attempts>=24) { cycles[n].part.uncertain=true; Invalid(314,"Partial retry budget
1448 | exhausted."); return; }
1449 |     MqlTick q; if(!SymbolInfoTick(_Symbol,q)) return;
1450 |     if(!PEPartialPriceReady(q.bid,cycles[n].entry,cycles[n].stop,cycles[n].target)) return;
1451 |     if(!CanRequest()) { Defer(cycles[n].part,"PARTIAL_DEFERRED",10018,"Session/permission
1452 | unavailable."); return; }
1453 |     ulong ticket=0; if(!PositionForLeg(n,0,ticket) ||
1454 | PEVolumeState(cycles[n].volume,PositionGetDouble(POSITION_VOLUME))!=0) return;
1455 |     MqlTradeRequest req={}; MqlTradeResult res={}; req.action=TRADE_ACTION_DEAL; req.position=ticket;
1456 |     req.symbol=_Symbol; req.magic=InpMagic; req.type=ORDER_TYPE_SELL; req.price=q.bid;
1457 |     req.volume=cycles[n].volume*.5; req.type_filling=Filling(); req.deviation=InpDeviationPoints;
1458 |     req.comment="EV160-PARTIAL";
1459 |     PEAttempt(cycles[n].part,now); bool sent=OrderSend(req,res); cycles[n].partialDeal=res.deal;
1460 |     cycles[n].partialOrder=res.order;
1461 |     cycles[n].part.lastRet=res.retcode;
1462 |     if(ReconcilePartial()) { MoveStops(cycles[n].entry); return; }
1463 |     if(!PositionSelectByTicket(ticket)) return; // SL/TP may have closed the entire position meanwhile
1464 |     int cls=PERetcodeClass(res.retcode);
1465 |     if(cls==1 && res.deal==0) { Defer(cycles[n].part,"PARTIAL_DEFERRED",res.retcode,"Definitive
1466 | rejection; sent="+I(sent)); return; }

```

```

1454 |   cycles[n].part.uncertain=true; ambiguousPartials++; lastRetcode=(int)res.retcode;
1455 |   lastErrorTime=TimeCurrent();
1456 |   Event("AMBIGUOUS_PARTIAL",n,"Await history/remaining-volume confirmation; never resend
1457 |   blindly.",res.retcode);
1458 |   if(cIs==3) Invalid(315,"Permanent/unknown partial response.");
1459 | }
1460 | void ManageProtection()
1461 | {
1462 |   if(active<0) return; CheckProtection(); if(closeEmergency || active<0) return;
1463 |   int n=active; MqlTick q; if(!SymbolInfoTick(_Symbol,q)) return;
1464 |   if(cycles[n].partialEnabled) {
1465 |     if(PEPartialPriceReady(q.bid,cycles[n].entry,cycles[n].stop,cycles[n].target))
1466 |     PEArm(cycles[n].part,NowMs());
1467 |     TryPartial();
1468 |   } else if(cycles[n].beTrigger>0) {
1469 |     double r=(q.bid-cycles[n].entry)/(cycles[n].entry-cycles[n].stop);
1470 |     if(r>=cycles[n].beTrigger) PEArm(cycles[n].be,NowMs());
1471 |     if(cycles[n].be.armed && !cycles[n].be.done) MoveStops(cycles[n].entry);
1472 |   }
1473 |   if(profile.arm==8 && cycles[n].be.armed && !cycles[n].be.done) MoveStops(cycles[n].desiredStop);
1474 | }
1475 | void ManageAdd(const bool newBar,const bool recovery)
1476 | {
1477 |   if(active<0 || haltEntries || closeEmergency || profile.addFraction<=0) return;
1478 |   int n=active; if(cycles[n].addPending || NowMs()<cycles[n].addNextMs) return;
1479 |   if(profile.arm!=8 && !newBar) return;
1480 |   MqlTick q; if(!SymbolInfoTick(_Symbol,q) || q.bid<=0 || q.ask<q.bid) return;
1481 |   double distance=cycles[n].entry-cycles[n].stop,r=(q.bid-cycles[n].entry)/distance;
1482 |   if(!PEAddWindow(profile.arm,cycles[n].adds,r,recovery)) return;
1483 |   cycles[n].addNextMs=NowMs()+5000;
1484 |   if(!CanRequest() || OwnPendingOrder()) return;
1485 |   if(cycles[n].addAttempts>=24) { cycles[n].addPending=true; Event("ADD_CANCELLED",n,"24
1486 |   rejected/submitted attempts reached."); return; }
1487 |   double v=0; ulong t=0; if(OwnPositions(v,t)!=cycles[n].legs) return;
1488 |   if(InpMaxSpreadPoints>0 && (q.ask-q.bid)/_Point>InpMaxSpreadPoints) return;
1489 |   double sl=cycles[n].stop;
1490 |   if(profile.arm==8) {
1491 |     sl=cycles[n].entry+cycles[n].adds*distance;
1492 |     sl=MathFloor(sl/SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_SIZE)+1e-9)*SymbolInfoDouble(_Symbol,SYM
1493 |     BOL_TRADE_TICK_SIZE);
1494 |     if(!MoveStops(sl)) return; // lock primary/all existing legs before adding
1495 |   }
1496 |   double volume=cycles[n].volume*profile.addFraction;
1497 |   double stops=SymbolInfoInteger(_Symbol,SYMBOL_TRADE_STOPS_LEVEL)*_Point;
1498 |   if(q.bid-sl<stops || cycles[n].target-q.bid<stops) return;
1499 |   double loss=0,margin=0,existing=ActiveRisk();
1500 |   if(!OrderCalcProfit(ORDER_TYPE_BUY,_Symbol,volume,q.ask,sl,loss) || existing<0) {
1501 |     Invalid(320,"Cannot value proposed addition."); return; }
1502 |   if(!PERiskWithin(existing,MathMax(0,-loss),cycles[n].initialRisk,profile.capRisk)) { riskBlocks++;
1503 |   Event("ADD_RISK_BLOCK",n,"Nominal stop budget."); return; }
1504 |   if(!OrderCalcMargin(ORDER_TYPE_BUY,_Symbol,volume,q.ask,margin) ||
1505 |   margin>AccountInfoDouble(ACCOUNT_MARGIN_FREE)) { Event("ADD_MARGIN_BLOCK",n,"Insufficient free
1506 |   margin."); return; }
1507 |   cycles[n].addAttempts++;
1508 |   bool sent=trade.Buy(volume,_Symbol,0,NormalizeDouble(sl,_Digits),cycles[n].target,"EV160-ADD-N"+I(cy
1509 |   cles[n].number));
1510 |   uint ret=trade.ResultRetcode();
1511 |   if(!sent || ret!=TRADE_RETCODE_DONE) {
1512 |     addRejects++; lastRetcode=(int)ret; lastErrorTime=TimeCurrent();
1513 |     Event("ADD_REJECTED",n,trade.ResultRetcodeDescription(),ret);
1514 |     cycles[n].addNextMs=NowMs()+PERetryDelayMs(ret);
1515 |     if(PERetcodeClass(ret)==2 || ret==TRADE_RETCODE_DONE) { cycles[n].addPending=true;
1516 |     Invalid(321,"Uncertain addition; no duplicate request.",true); }
1517 |     else if(PERetcodeClass(ret)==3) { cycles[n].addPending=true; Invalid(322,"Permanent addition
1518 |     rejection."); }
1519 |     return;
1520 |   }
1521 |   ulong d=trade.ResultDeal();
1522 |   if(d==0 || !HistoryDealSelect(d) || HistoryDealGetInteger(d,DEAL_ENTRY)!=DEAL_ENTRY_IN ||

```

```

1510 |     HistoryDealGetInteger(d,DEAL_TYPE)!=DEAL_TYPE_BUY ||
|     (ulong)HistoryDealGetInteger(d,DEAL_MAGIC)!=InpMagic ||
1511 |     HistoryDealGetString(d,DEAL_SYMBOL)!=_Symbol ||
|     MathAbs(HistoryDealGetDouble(d,DEAL_VOLUME)-volume)>1e-8) {
1512 |         Invalid(323,"Addition fill cannot be reconciled.",true); return;
1513 |     }
1514 |     ulong pid=(ulong)HistoryDealGetInteger(d,DEAL_POSITION_ID); int j=cycles[n].legs;
1515 |     if(j>=3 || pid==0 || CycleByPid(pid)>=0) { Invalid(324,"Addition requires a distinct hedging
|     position.",true); return; }
1516 |     cycles[n].pids[j]=pid; cycles[n].fills[j]=HistoryDealGetDouble(d,DEAL_PRICE);
|     cycles[n].volumes[j]=volume;
1517 |     cycles[n].expectedStops[j]=sl; cycles[n].legs++; cycles[n].adds++;
1518 |     Event("ADD_FILLED",n,"lot="+N(volume)+" ; price="+N(cycles[n].fills[j])+" ; sl="+N(sl),ret);
1519 |     CheckProtection(); SyncCycle();
1520 | }
1521 | void EmergencyClose()
1522 | {
1523 |     if(!closeEmergency || !CanRequest() || NowMs()-lastEmergencyMs<10000) return;
1524 |     lastEmergencyMs=NowMs();
1525 |     for(int i=PositionsTotal()-1;i>=0;i--) {
1526 |         ulong t=PositionGetTicket(i); if(t==0 || !OwnSelected()) continue;
1527 |         bool sent=trade.PositionClose(t); uint ret=trade.ResultRetcode();
1528 |         Event("EMERGENCY_CLOSE",active,"ticket="+U(t)+" ; sent="+I(sent),ret);
1529 |     }
1530 |     SyncCycle();
1531 | }
1532 |
1533 |
1534 | PE_GATE OpenCycle(const int origin,const int side,const H1Signal &s,const MqlTick &q,const datetime
|     signalTime,
1535 |         double &sl,double &tp,double &risk,string &detail)
1536 | {
1537 |     double tick=SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_SIZE),entry=side==1 ? q.ask : q.bid;
1538 |     H1Signal setup=s; if(evo.entryPolicy==8) { setup.lowest=pivotSwing; setup.highest=pivotSwing; }
1539 |     if(!H1Levels(side,entry,setup,.20,1.0,2.5,5.0,tick,sl,tp)) return G_STOP;
1540 |     // Original structural stop is retained in every case.
1541 |     if(dc.minimumStopATR>1.0 && !DCStructuralStopAccepted(entry,sl,s.atr,dc.minimumStopATR)) {
1542 |         detail="Structural stop below required ATR distance; not widened."; return G_STOP;
1543 |     }
1544 |     if(dc.targetMode>0) {
1545 |         double distance=HermesTargetDistance(dc.targetMode,_Point);
1546 |         if(!HermesTargetPrice(side,entry,distance,tick,tp)) {
1547 |             targetRejects++; detail="Target below one tick, invalid units or non-representable target; no
|             widening."; return G_TARGET;
1548 |         }
1549 |     }
1550 |     plannedTargetDistance=side*(tp-entry);
|     plannedTargetRiskRatio=plannedTargetDistance/MathAbs(entry-sl);
1551 |     sl=NormalizeDouble(sl,_Digits); tp=NormalizeDouble(tp,_Digits);
1552 |     double minstop=SymbolInfoInteger(_Symbol,SYMBOL_TRADE_STOPS_LEVEL)*_Point;
1553 |     double closePrice=side==1 ? q.bid : q.ask;
1554 |     if(side*(closePrice-sl)<minstop || side*(tp-closePrice)<minstop || sl<=0 || tp<=0) {
1555 |         detail="Broker stop distance not met; target and structural stop were not enlarged.";
1556 |         if(dc.targetMode>0) targetRejects++;
1557 |         return G_BROKER_STOPS;
1558 |     }
1559 |     double mn=SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_MIN),mx=SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_MAX)
|     ,step=SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_STEP);
1560 |     if(!MathIsValidNumber(mn) || !MathIsValidNumber(mx) || !MathIsValidNumber(step) || mn<=0 || mx<mn ||
|     step<=0) return G_RISK;
1561 |     double volume=InpFixedLot,profit=0,margin=0;
1562 |     ENUM_ORDER_TYPE type=side==1 ? ORDER_TYPE_BUY : ORDER_TYPE_SELL;
1563 |     if(dc.fixedLot) {
1564 |         sizingMarginBudget=AccountInfoDouble(ACCOUNT_MARGIN_FREE);
1565 |         if(!HermesExactLot(volume,mn,mx,step)) {
1566 |             fixedVolumeRejects++; detail="Exact fixed lot incompatible with symbol min/max/step; NOT
|             reduced."; return G_RISK;
1567 |         }
1568 |     } else {

```

```

1569 | double referenceLoss=0,referenceMargin=0;
1570 | if(!OrderCalcProfit(type,_Symbol,mn,entry,sl,referenceLoss) || referenceLoss>=0) return G_RISK;
1571 | if(!OrderCalcMargin(type,_Symbol,mn,entry,referenceMargin)) return G_MARGIN_ERROR;
1572 | double eq=AccountInfoDouble(ACCOUNT_EQUITY),free=AccountInfoDouble(ACCOUNT_MARGIN_FREE),used=Account
1573 | tInfoDouble(ACCOUNT_MARGIN);
1574 | minimumLotRiskMoney=-referenceLoss; minimumLotMargin=referenceMargin;
1575 | minimumLotRiskPercent=eq>0 ? 100*(-referenceLoss)/eq : 0;
1576 | minimumEquityForLot=MathMax((-referenceLoss)*100.0/dc.riskPercent,(used+referenceMargin)*100.0/InpM
1577 | axMarginPct);
1578 | sizingRiskBudget=eq*dc.riskPercent/100.0;
1579 | sizingMarginBudget=MathMin(free,MathMax(0,eq*InpMaxMarginPct/100.0-used));
1580 | volume=DCRiskLot(eq,free,used,dc.riskPercent,InpMaxMarginPct,-referenceLoss/mn,referenceMargin/mn,m
1581 | n,mx,step,InpMaxLot);
1582 | if(volume<=0) { detail="Volume below broker minimum within risk/margin budgets; no forced
1583 | minimum."; riskBlocks++; return G_RISK; }
1584 | // Contract margin may be tiered. Revalue the proposed volume and step down using native values.
1585 | for(int tries=0;tries<8;tries++) {
1586 |     if(!OrderCalcProfit(type,_Symbol,volume,entry,sl,profit) || profit>=0) return G_RISK;
1587 |     if(!OrderCalcMargin(type,_Symbol,volume,entry,margin)) return G_MARGIN_ERROR;
1588 |     if(-profit<=sizingRiskBudget+1e-7 && margin<=sizingMarginBudget+1e-7) break;
1589 |     double ratio=MathMin(sizingRiskBudget/(-profit),margin>0 ? sizingMarginBudget/margin : 1.0);
1590 |     volume=MathFloor(MathMin(volume-step,volume*ratio)/step+1e-9)*step;
1591 |     if(volume<mn-1e-9) { riskBlocks++; return G_RISK; }
1592 | }
1593 | sizedLot=volume;
1594 | if(HlVolume(FIXED_LOT,volume,1,1,mn,mx,step)<=0) return G_RISK;
1595 | if(!OrderCalcProfit(type,_Symbol,volume,entry,sl,profit) || profit>=0) return G_RISK;
1596 | risk=-profit;
1597 | if(!OrderCalcMargin(type,_Symbol,volume,entry,margin)) return G_MARGIN_ERROR;
1598 | sizingMargin=margin;
1599 | if(!dc.fixedLot && risk>sizingRiskBudget+1e-7) { riskBlocks++; return G_RISK; }
1600 | if(!HermesMarginAvailable(margin,AccountInfoDouble(ACCOUNT_MARGIN_FREE)) || (!dc.fixedLot &&
1601 | margin>sizingMarginBudget+1e-7)) {
1602 |     detail="Required margin="+N(margin)+"; free="+N(AccountInfoDouble(ACCOUNT_MARGIN_FREE))+"; fixed
1603 | lot is not reduced.";
1604 |     return G_MARGIN_BLOCK;
1605 | }
1606 | if(!OrderCalcProfit(type,_Symbol,volume,entry,tp,plannedTargetProfit)) { detail="Cannot value target
1607 | profit."; return G_TARGET; }
1608 | double balance=AccountInfoDouble(ACCOUNT_BALANCE); int number=ArraySize(cycles)+1;
1609 | string comment="HP200-0"+I(origin)+"-N"+I(number);
1610 | bool sent=side==1 ? trade.Buy(volume,_Symbol,0,sl,tp,comment) :
1611 | trade.Sell(volume,_Symbol,0,sl,tp,comment);
1612 | uint ret=trade.ResultRetcode(); detail="ret="+I(ret)+"; deal="+U(trade.ResultDeal())+";
1613 | "+trade.ResultRetcodeDescription();
1614 | if(!sent || ret!=TRADE_RETCODE_DONE) {
1615 |     lastRetcode=(int)ret; lastErrorTime=TimeCurrent(); Event("ENTRY_REJECTED",-1,detail,ret);
1616 |     if(PERetcodeClass(ret)==2 || ret==TRADE_RETCODE_DONE) Invalid(201,"Initial order uncertain; no
1617 | duplicate entry allowed.",true);
1618 |     return G_REJECTED;
1619 | }
1620 | ulong deal=trade.ResultDeal();
1621 | if(deal==0 || !HistoryDealSelect(deal) || HistoryDealGetString(deal,DEAL_SYMBOL)!=_Symbol ||
1622 | (ulong)HistoryDealGetInteger(deal,DEAL_MAGIC)!=InpMagic ||
1623 | HistoryDealGetInteger(deal,DEAL_ENTRY)!=DEAL_ENTRY_IN ||
1624 | HistoryDealGetInteger(deal,DEAL_TYPE)!=(side==1 ? DEAL_TYPE_BUY : DEAL_TYPE_SELL)) {
1625 |     Invalid(202,"Entry fill identity missing.",true); return G_HALTED; }
1626 | int n=ArraySize(cycles); ArrayResize(cycles,n+1); ZeroMemory(cycles[n]); active=n;
1627 | cycles[n].entryPath=hpRoute.path;
1628 | if(hpRoute.path==HP_PIVOT) cycles[n].entryPivot=hpSignal;
1629 | cycles[n].number=number; cycles[n].origin=origin; cycles[n].side=side; cycles[n].legs=1;
1630 | cycles[n].pid=(ulong)HistoryDealGetInteger(deal,DEAL_POSITION_ID); cycles[n].pids[0]=cycles[n].pid;
1631 | cycles[n].start=(datetime)HistoryDealGetInteger(deal,DEAL_TIME); cycles[n].signalTime=signalTime;
1632 | cycles[n].quotedEntry=entry; cycles[n].quotedTarget=tp;
1633 | cycles[n].targetDistance=HermesTargetDistance(dc.targetMode,_Point);
1634 | cycles[n].entry=HistoryDealGetDouble(deal,DEAL_PRICE);
1635 | cycles[n].volume=HistoryDealGetDouble(deal,DEAL_VOLUME);
1636 | cycles[n].fills[0]=cycles[n].entry; cycles[n].volumes[0]=cycles[n].volume;

```

```

cycles[n].expectedStops[0]=sl;
1624 |   cycles[n].stop=sl; cycles[n].liveStop=sl; cycles[n].desiredStop=sl; cycles[n].target=tp;
cycles[n].balanceBefore=balance;
1625 |   cycles[n].partialEnabled=profile.partial || (origin==14 && profile.protect14);
1626 |   cycles[n].beTrigger=cycles[n].partialEnabled ? 1.0 : ((profile.mode==0 && origin!=15) ? 0 :
profile.be15);
1627 |   cycles[n].path.entry=cycles[n].entry; cycles[n].path.stop=sl; cycles[n].path.side=side;
1628 |   cycles[n].entryPolicy=evo.entryPolicy; cycles[n].entryFeatures=features;
1629 |   cycles[n].featureADXChange=s.adx-s.oldADX; cycles[n].featureADX=s.adx; cycles[n].featureATR=s.atr;
1630 |   cycles[n].featureDistance=(entry-s.ema)/s.atr; cycles[n].featureSlope=(s.sma200-oldSMA200)/s.atr;
1631 |   if(cycles[n].pid==0 || side*(cycles[n].entry-sl)<=0 || MathAbs(cycles[n].volume-volume)>1e-8)
1632 |     Invalid(203,"Initial fill does not match planned volume/stop.",true);
1633 |   if(!OrderCalcProfit(type,_Symbol,volume,cycles[n].entry,sl,profit) || profit>=0) Invalid(204,"Cannot
value initial risk.",true);
1634 |   else { cycles[n].initialRisk=-profit; cycles[n].peakRisk=-profit; maxRisk=MathMax(maxRisk,-profit);
if(balance>0) maxRiskPct=MathMax(maxRiskPct,-profit/balance*100); }
1635 |   maxLots=MathMax(maxLots,volume); MonthIndex(cycles[n].start); months[liveMonth].opened++;
1636 |   if(evo.entryPolicy==8) lastPivotUsed=pivotTime;
1637 |   detail+="; entry_path="; detail+=(hpRoute.path==HP_PIVOT ? "PIVOT" : "BASE");
1638 |   detail+="; original_gate="+I(hpRoute.original_gate)+"; extra_gate="+I(hpRoute.extra_gate);
1639 |   if(hpRoute.path==HP_PIVOT) {
1640 |     hpPivotFills++;
1641 |     detail+="; pivot_signal_time="+I(hpSignal.signal_time)+"; p1_time="+I(hpSignal.p1_time)+";
p2_time="+I(hpSignal.p2_time)+"; p3_time="+I(hpSignal.p3_time);
1642 |   }
1643 |   Event("ENTRY_FILLED",n,detail); ObserveOpenPath(); EnforceTargetCap();
1644 |   if(!closeEmergency && PositionForLeg(n,0,deal)) {
1645 |     double estimate=0;
1646 |     if(OrderCalcProfit(type,_Symbol,cycles[n].volume,cycles[n].entry,cycles[n].target,estimate))
cycles[n].targetProfitEstimate=estimate;
1647 |   }
1648 |   CheckProtection(); SyncCycle(); return G_FILLED;
1649 | }
1650 | void OnTick()
1651 | {
1652 |   if(!initialized) return;
1653 |   TrackEquity(); ObserveOpenPath(); SyncCycle();
1654 |   if(closeEmergency) { EmergencyClose(); TrackEquity(); return; }
1655 |   ManageProtection(); TrackEquity(); SyncCycle();
1656 |   datetime bar=iTime(_Symbol,signalTF,0); bool newBar=bar>0 && bar!=lastBar;
1657 |   if(!newBar) { if(profile.arm==8) ManageAdd(false,false); return; }
1658 |   lastBar=bar; lastEval=TimeCurrent(); if(firstEval==0) firstEval=lastEval;
1659 |   barsSeen++; int mi=MonthIndex(TimeCurrent()); months[mi].bars++;
1660 |   hpRoute.original_gate=-1; hpRoute.extra_gate=HP_DATA_PENDING; hpRoute.setup=-1; hpRoute.side=0;
hpRoute.path=HP_NONE;
1661 |   hpDataReady=HPSyncClosedBars(bar);
1662 |   if(!hpDataReady) hpDataFailures++;
1663 |   if(hpSignal.buy) hpCandidates++;
1664 |   H1Signal s; ZeroMemory(s); MqlTick q; ZeroMemory(q); datetime signalTime=0;
1665 |   sizedLot=0; sizingRiskBudget=0; sizingMarginBudget=0; sizingMargin=0;
1666 |   plannedTargetDistance=0; plannedTargetRiskRatio=0; plannedTargetProfit=0;
1667 |   minimumLotRiskMoney=0; minimumLotRiskPercent=0; minimumEquityForLot=0; minimumLotMargin=0;
1668 |   channelReady=false; channelBars=0; channelUpper=0; channelLower=0; channelOldest=0; channelNewest=0;
channelClosedAt=0; channelStatus="SIGNAL_DATA_PENDING";
1669 |   if(!SymbolInfoTick(_Symbol,q) || q.bid<=0 || q.ask<q.bid || !ReadSignal(s,signalTime)) {
Record(G_NO_DATA,-1,0,s,q,signalTime); return; }
1670 |   if(dc.channelMode>0 && !channelReady) {
Record(G_NO_DATA,120,evo.family,s,q,signalTime,0,0,0,channelStatus); return; }
1671 |   if(firstReady==0 && (dc.channelMode==0 || channelReady)) firstReady=TimeCurrent();
1672 |   int side=0;
1673 |   double minimumEntryDistance=InpMinEntryATR;
1674 |   HPRouteEntry(InpCase,evo,s,features,q.ask,oldSMA200,minimumEntryDistance,
1675 |     hpDataReady,hpSignal,hpRoute);
1676 |   int setup=hpRoute.setup; side=hpRoute.side;
1677 |   if(hpRoute.extra_gate==0) hpExtraEligible++;
1678 |   if(hpRoute.path==HP_BASE) hpBaseSelected++;
1679 |   if(hpRoute.path==HP_PIVOT) hpPivotSelected++;
1680 |   bool pivotOK=pivotSide!=0 && pivotTime!=lastPivotUsed && s.adx>=20 && s.adx>s.oldADX &&
1681 |     (pivotSide==1 ? (s.plusDI>s.minusDI && s.close>s.open && s.close>s.ema) : (s.minusDI>s.plusDI &&

```

```

s.close<s.open && s.close<s.ema));
1682 | if(pivotOK) { if(pivotSide==1) patternBuys++; else patternSells++; }
1683 | if(setup==0) { signals++; months[mi].signals++; }
1684 | if(haltEntries) { Record(G_HALTED,setup,0,s,q,signalTime); return; }
1685 | bool recovery=s.close>s.open && s.close>previousClose && s.adx>=20 && s.plusDI>s.minusDI &&
s.close>s.sma200 &&
1686 | s.ema>s.sma50 && s.sma50>s.oldSma50 && (cyclesSizeOrigin15() ? s.sma200>oldSMA200 : true);
1687 | if(active>=0) { ManageAdd(true,recovery); Record(G_POSITION,setup,active>=0 ? cycles[active].origin
: 0,s,q,signalTime); return; }
1688 | if(SymbolExposure()) { Record(G_POSITION,setup,0,s,q,signalTime); return; }
1689 | if(InpMaxSpreadPoints>0 && (q.ask-q.bid)/_Point>InpMaxSpreadPoints) {
Record(G_SPREAD,setup,0,s,q,signalTime); return; }
1690 | int origin=evo.family;
1691 | if(setup!=0) {
1692 | PE_GATE gate=setup==120 ? G_NO_DATA : setup==110 ? G_DISTANCE : (setup==111 ? G_REGIME :
(setup==109 ? G_DIRECTION : (setup>=100 ? G_ENTRY_FILTER : G_BASE)));
1693 | Record(gate,setup,origin,s,q,signalTime); return;
1694 | }
1695 | double sl=0,tp=0,risk=0; string detail=""; PE_GATE
result=OpenCycle(origin,side,s,q,signalTime,sl,tp,risk,detail);
1696 | Record(result,setup,origin,s,q,signalTime,sl,tp,risk,detail); if(closeEmergency) EmergencyClose();
TrackEquity();
1697 | }
1698 | bool cyclesSizeOrigin15() { return active>=0 && cycles[active].origin==15; }
1699 | bool WriteParameters()
1700 | {
1701 | int f=OpenText(folder+"\\parameters.txt"); if(f==INVALID_HANDLE) return false;
1702 | KV(f,"EA","Hermes_Pivos_Lab_200"); KV(f,"version","2.00"); KV(f,"case",I(InpCase));
KV(f,"profile",ProfileName(InpCase));
1703 | KV(f,"run_tag",InpRunTag); KV(f,"symbol",_Symbol);
KV(f,"currency",AccountInfoString(ACCOUNT_CURRENCY));
1704 | KV(f,"signal_tf",TFName(InpCase)); KV(f,"source_control170","28");
KV(f,"source_control180",I(dc.sourceControl180)); KV(f,"matched_control190",I(dc.matchedControl));
1705 | KV(f,"risk_percent",N(dc.riskPercent)); KV(f,"margin_cap_percent",dc.fixedLot ?
"no_extra_percent_cap_free_margin_only" : N(InpMaxMarginPct));
1706 | KV(f,"max_lot",dc.fixedLot ? N(InpFixedLot) : N(InpMaxLot));
KV(f,"initial_lot_reference",N(InpFixedLot));
1707 | KV(f,"fixed_lot_mode",I(dc.fixedLot)); KV(f,"capital_mode",dc.fixedLot ? "EXACT_FIXED_LOT" :
"EQUITY_RISK_2PCT");
1708 | KV(f,"target_unit",TargetUnitName()); KV(f,"target_unit_code",I(dc.targetMode));
KV(f,"target_value",dc.targetMode==0 ? "5" : "20");
1709 | KV(f,"resolved_target_distance_price",N(HermesTargetDistance(dc.targetMode,_Point)));
1710 | KV(f,"target_R",dc.targetMode==0 ? "5" : "variable_target_divided_by_original_stop");
KV(f,"partial_enabled","0"); KV(f,"BE_trigger_R","0"); KV(f,"max_adds","0");
1711 | KV(f,"case_protocol","1 HERMES_REFERENCIA;2 HERMES_PIVO_CONTINUIDADE;3 HERMES_PIVO_INICIO. All M30,
exact fixed lot, target5R, no partial/BE/add/reinvestment.");
1712 | KV(f,"reference_source_sha256","0ddbee937fc4b5af16510987f84d72e126012e0a080b61ef18530a80275d8e35");
1713 | KV(f,"reference_EA","0limpo_Consistencia_Lab_190_case2");
1714 | KV(f,"native_compilation_or_backtest_by_assistant","false; native user validation pending");
1715 | KV(f,"M30_entry","0Original Hermes rules: EMA21>SMA50; rising SMA50/SMA200; close>SMA200; ADX14>=20;
+DI>-DI; prior EMA touch; bullish close above EMA21 and prior high; ask-EMA>=minimum_distance_ATR.
Always original first.");
1716 | KV(f,"extra_entry_common","Fresh causal confirmed LONG 1-2-3 neckline cross; bullish closed candle
above EMA21; EMA21>SMA50 and SMA50>SMA50[6]; ADX14>=20; +DI>-DI; valid SMA3-SMA10 oscillator greater
than its previous value; ask-EMA>=minimum_distance_ATR.");
1717 | KV(f,"extra_entry_case2","Also close>SMA200 and SMA200>SMA200[6].");
1718 | KV(f,"extra_entry_case3","Only the two SMA200 conditions are omitted for the extra entry; original
entry rules unchanged.");
1719 | KV(f,"entry_priority","Extra only when original EVOEvaluate fails. An original signal later rejected
by stop/margin/spread does not fall back to another entry. One position; pivot event consumed even
while blocked.");
1720 | KV(f,"pivot_algorithm","HERMES_PIVOS_2x2_V1; strict 2-left/2-right extrema; tied extrema excluded;
double-extreme candle resets alternating sequence; L1-H-L2 with L1<L2<H; closed crossing of H; known
before trigger open; invalidated by low<=L1. Geometric L1 is NOT execution stop.");
1721 | KV(f,"pivot_time_rule","Pivots available at next observed opening after second right candle;
breakout evaluated only using references known at breakout-bar open; no open-candle OHLC used.");
1722 | KV(f,"pivot_seed","210 closed bars seeded chronologically once; only latest signal may be traded;
subsequent closed bars caught up continuously, including occupied periods; historical catch-up signals
discarded. Seed boundary may differ from a catalog started in 2022.");

```

```

1723 | KV(f,"pivot_data_failure","Extra entries blocked only; original reference remains eligible. State
      | not advanced on incomplete/invalid copy; next bar retries chronological catch-up. No synthetic
      | history.");
1724 | KV(f,"minimum_structural_stop_ATR",N(dc.minimumStopATR));
1725 | KV(f,"stop","Original: minimum low of closed signal plus 2 prior bars minus .20ATR14; entry-stop
      | must be<1.25ATR or reject. No widening and no replacing stop with geometric L1.");
1726 | KV(f,"sizing","Every case exact InpFixedLot=1.00 default; native free-margin and volume constraints;
      | unavailable lot rejected, never silently reduced. InpMaxMarginPct/InpMaxLot are retained
      | compatibility inputs and unused in these fixed-lot cases.");
1727 | KV(f,"target_execution","Original HlLevels5R quoted-entry target with attached server SL/TP. Tick
      | rounding and actual slippage retained from reference; fills/costs can change realized R. No partial,
      | no BE, no averaging or pyramiding.");
1728 | KV(f,"minimum_distance_ATR",N(InpMinEntryATR)); KV(f,"all_bars",I(InpExportAllBars));
1729 | KV(f,"channel_mode","0 not used");
1730 | KV(f,"warmup","Original minimum209 bars unchanged; extra pivot seed needs210 closed bars plus
      | current opening witness. Signal indicator failure remains G_NO_DATA.");
1731 | KV(f,"entry_frequency","One decision per new M30 bar, only closed inputs. Extra candidates are
      | hypotheses, not guaranteed fills or profits.");
1732 | KV(f,"account_margin_mode",I(AccountInfoInteger(ACCOUNT_MARGIN_MODE)));
      | KV(f,"leverage",I(AccountInfoInteger(ACCOUNT_LEVERAGE)));
1733 | KV(f,"account_stopout_mode",I(AccountInfoInteger(ACCOUNT_MARGIN_SO_MODE)));
1734 | KV(f,"account_margin_call_level",N(AccountInfoDouble(ACCOUNT_MARGIN_SO_CALL)));
1735 | KV(f,"account_stopout_level",N(AccountInfoDouble(ACCOUNT_MARGIN_SO_SO)));
1736 | KV(f,"initial_deposit",N(initialDeposit)); KV(f,"point",N(_Point));
      | KV(f,"tick_size",N(SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_SIZE)));
1737 | KV(f,"contract_size",N(SymbolInfoDouble(_Symbol,SYMBOL_TRADE_CONTRACT_SIZE)));
1738 | KV(f,"tick_value_profit",N(SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_VALUE_PROFIT)));
      | KV(f,"tick_value_loss",N(SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_VALUE_LOSS)));
1739 | KV(f,"volume_min",N(SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_MIN)));
      | KV(f,"volume_max",N(SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_MAX)));
      | KV(f,"volume_step",N(SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_STEP)));
1740 | KV(f,"margin_initial",N(SymbolInfoDouble(_Symbol,SYMBOL_MARGIN_INITIAL)));
      | KV(f,"margin_maintenance",N(SymbolInfoDouble(_Symbol,SYMBOL_MARGIN_MAINTENANCE)));
      | KV(f,"trade_calc_mode",I(SymbolInfoInteger(_Symbol,SYMBOL_TRADE_CALC_MODE)));
1741 | KV(f,"magic",U(InpMagic)); KV(f,"deviation_points",U(InpDeviationPoints));
      | KV(f,"max_spread_points",N(InpMaxSpreadPoints));
1742 | KV(f,"scope","Strategy Tester only; predeclared hypotheses, no fitted future outcomes. Profit
      | unknown; native report needed for dates/tick quality/model/latency.");
1743 | FileClose(f); return !ioFailure;
1744 | }
1745 | int OnInit()
1746 | {
1747 |     if(!MQLInfoInteger(MQL_TESTER)) { Print("Use Ctrl+R. EA de pesquisa, exclusivo do Testador de
      | Estrategias."); return INIT_FAILED; }
1748 |     if(!HPSelectProfile(InpCase,dc,evo,profile) || !ValidRunTag()) return INIT_PARAMETERS_INCORRECT;
1749 |     signalTF=PERIOD_M30; HPCoreReset(hpState); HPClearSignal(hpSignal);
1750 |     if(_Period!=signalTF) { Print("Caso ",InpCase," exige ",TFName(InpCase)," no testador."); return
      | INIT_PARAMETERS_INCORRECT; }
1751 |     if(!MathIsValidNumber(InpMaxMarginPct) || InpMaxMarginPct<=0 || InpMaxMarginPct>50 ||
1752 |        !MathIsValidNumber(InpMaxLot) || InpMaxLot<=0) return INIT_PARAMETERS_INCORRECT;
1753 |     string symbol=_Symbol; StringToUpper(symbol); if(StringFind(symbol,"XAU")<0 &&
      | StringFind(symbol,"GOLD")<0) return INIT_PARAMETERS_INCORRECT;
1754 |     if((profile.reinvest && evo.lotCap<InpFixedLot) || !MathIsValidNumber(InpFixedLot) || InpFixedLot<=0
      | || !MathIsValidNumber(InpMinEntryATR) || InpMinEntryATR<0 || InpMinEntryATR>3 ||
1755 |        !MathIsValidNumber(InpMaxSpreadPoints) || InpMaxSpreadPoints<0) return INIT_PARAMETERS_INCORRECT;
1756 |     double mn=SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_MIN),mx=SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_MAX)
      | ,step=SymbolInfoDouble(_Symbol,SYMBOL_VOLUME_STEP);
1757 |     if(!MathIsValidNumber(mn) || !MathIsValidNumber(mx) || !MathIsValidNumber(step) || mn<=0 || mx<mn ||
      | step<=0) return INIT_PARAMETERS_INCORRECT;
1758 |     if(dc.fixedLot && !HermesExactLot(InpFixedLot,mn,mx,step))
1759 |         Print("Lote fixo incompatível; sinais serão registrados como recusados, sem reduzir volume.");
1760 |     if(SymbolInfoDouble(_Symbol,SYMBOL_TRADE_TICK_SIZE)<=0) return INIT_PARAMETERS_INCORRECT;
1761 |     if(profile.addFraction>0 &&
      | AccountInfoInteger(ACCOUNT_MARGIN_MODE)!=ACCOUNT_MARGIN_MODE_RETAIL_HEDGING) {
1762 |         Print("Perfis com adições exigem conta HEDGE no testador; não executados em NETTING."); return
      | INIT_PARAMETERS_INCORRECT;
1763 |     }
1764 |     initialDeposit=AccountInfoDouble(ACCOUNT_BALANCE); LoadSessions();
1765 |     TesterHideIndicators(true); hEMA=iMA(_Symbol,signalTF,21,0,MODE_EMA,PRICE_CLOSE);

```

```

1766 | if(!dc.weekly) { h50=iMA(_Symbol,signalTF,50,0,MODE_SMA,PRICE_CLOSE);
      | h200=iMA(_Symbol,signalTF,200,0,MODE_SMA,PRICE_CLOSE); }
1767 | hADX=iADX(_Symbol,signalTF,14); hATR=iATR(_Symbol,signalTF,14);
1768 | if(hEMA==INVALID_HANDLE || (!dc.weekly && (h50==INVALID_HANDLE || h200==INVALID_HANDLE)) ||
      | hADX==INVALID_HANDLE || hATR==INVALID_HANDLE) return INIT_FAILED;
1769 | trade.SetExpertMagicNumber(InpMagic); trade.SetDeviationInPoints(InpDeviationPoints);
      | trade.SetAsyncMode(false); trade.SetTypeFillingBySymbol(_Symbol);
1770 | ArrayInitialize(counters,0); priorEquity=AccountInfoDouble(ACCOUNT_EQUITY);
      | priorBalance=AccountInfoDouble(ACCOUNT_BALANCE);
1771 | MqlRates warmup[]; int required=dc.weekly ? 55 : 209;
1772 | int count=CopyRates(_Symbol,signalTF,0,required,warmup);
1773 | if(count<required) Print("Aquecimento incompleto ",count,"/",required,". Verifique G_NO_DATA.");
1774 |
1775 | if(InpExportCSV && (!MQLInfoInteger(MQL_OPTIMIZATION) || InpExportOptimizationDetails)) {
1776 |     FolderCreate("Hermes_Pivos_Lab_200",FILE_COMMON); FolderCreate(RunRoot(),FILE_COMMON);
1777 |     folder=RunRoot()+"\\CASO_"+I(InpCase)+"_"+TFName(InpCase)+"_"+Stamp();
1778 |     if(!FolderCreate(folder,FILE_COMMON)) return INIT_FAILED;
1779 |     barsFile=OpenText(folder+"\\bars.csv"); eventsFile=OpenText(folder+"\\events.csv");
1780 |     if(barsFile==INVALID_HANDLE || eventsFile==INVALID_HANDLE || !WriteParameters()) return
      | INIT_FAILED;
1781 |     Row(eventsFile,"time_server;case;cycle;origin_case140;event;retcode;detail");
1782 |     Row(barsFile,"event_time;signal_time;case;origin_case140;gate;setup_gate;cycle;open;high;low;close;
      | ema21;sma50;sma200;old_sma50;old_sma200;adx;plus_di;minus_di;atr;bid;ask;sl;tp;risk_money;balance;equi
      | ty;detail;pivot_side;pivot_break_level;pivot_stop_swing;pivot_time;distance_atr;sma200_slope_atr;previ
      | ous_adx;previous_close;pivot_pattern_eligible;original15_candidate;original14_candidate;spread_atr;bod
      | y_atr;entry_policy;oscillator_valid;fast310;previous_fast310;signal310;true_range_atr;close_location;s
      | ignal_tf;donchian_mode;donchian_ready;donchian_bars;donchian_upper;donchian_lower;donchian_oldest_open
      | ;donchian_newest_open;donchian_latest_close;donchian_status;risk_percent;margin_cap_percent;planned_lo
      | t;risk_budget;margin_budget;planned_margin;free_margin;used_margin;minimum_lot_risk_money;minimum_lot
      | risk_percent;minimum_equity_for_minimum_lot;minimum_lot_margin;source_controll180;matched_controll190;fi
      | xed_lot_mode;target_unit;target_value;planned_target_distance_price;planned_target_R;planned_target_gr
      | oss_profit;hp_data_ready;hp_status;hp_candidate;hp_original_gate;hp_extra_gate;selected_entry_path;hp_
      | signal_time;hp_available_time;hp_p1_price;hp_p2_price;hp_p3_price;hp_p1_time;hp_p2_time;hp_p3_time;hp_
      | p1_confirmation_time;hp_p2_confirmation_time;hp_p3_confirmation_time;hp_p1_available_time;hp_p2_availa
      | ble_time;hp_p3_available_time;hp_pattern_available_time;hp_processed_this_bar;hp_total_processed;hp_la
      | st_processed_open;hp_expected_next_open");
1783 |     Print("RELATORIOS: ",TerminalInfoString(TERMINAL_COMMONDATA_PATH),"\\Files\\",folder);
1784 | }
1785 | if(InpShowIndicators && MQLInfoInteger(MQL_VISUAL_MODE)) {
1786 |     ChartIndicatorAdd(0,0,hEMA); if(!dc.weekly) { ChartIndicatorAdd(0,0,h50);
      | ChartIndicatorAdd(0,0,h200); }
1787 |     ChartIndicatorAdd(0,(int)ChartGetInteger(0,CHART_WINDOWS_TOTAL),hADX);
      | ChartIndicatorAdd(0,(int)ChartGetInteger(0,CHART_WINDOWS_TOTAL),hATR);
1788 | }
1789 | if(ioFailure) return INIT_FAILED;
1790 | initialized=true; Print("Hermes Pivos 2.00: ",ProfileName(InpCase)," . Hipotese fixa; detalhes por
      | caso."); return INIT_SUCCEEDED;
1791 | }
1792 |
1793 | bool IsTimeStat(const int i)
1794 | { return i==S_FIRST_EVAL || i==S_LAST_EVAL || i==S_FIRST_ENTRY || i==S_LAST_EXIT || i==S_FIRST_READY
      | || i==S_LAST_ERROR_TIME; }
1795 | int StatNames(string &names[])
1796 | {
1797 |     return StringSplit("valid_run;profit;deposit;final_balance;mt5_trades;mt5_profit_factor;equity_dd_re
      | lative_percent;equity_dd_max_money;equity_percent_at_max_money_dd;cycles_opened;cycles_closed;cycle_wi
      | ns;cycle_losses;cycle_zero;cycle_win_percent;cycle_profit_factor;cycle_net;average_net_R_initial;max_c
      | onsecutive_losses;average_hold_hours;max_hold_hours;swap;commission_and_fees;max_open_lots;max_stop_ri
      | sk_money;max_stop_risk_percent;max_margin;first_evaluation;last_evaluation;first_entry;last_exit;bars_
      | seen;signals_before_filters;first_ready;origin15_cycles;origin14_cycles;origin15_net;origin14_net;BE_a
      | rmed;BE_confirmed;BE_stop_exits;partial_armed;partial_done;partial_deferrals;BE_deferrals;session_defe
      | rrals;stop_freeze_deferrals;BE_requests;partial_requests;ambiguouspartials;error_code;last_retcode;la
      | st_error_time;reconcile_difference;monthly_equity_reconcile_difference;monthly_booked_reconcile_differ
      | ence;unmatched_deals;months_observed;positive_equity_months;negative_equity_months;positive_booked_mon
      | ths;negative_booked_months;mean_MFE_R;mean_winner_MFE_R;mean_loser_MFE_R;mean_MAE_R;path_quote_observa
      | tions;shadow_original_control;fixed_lot;BE_trigger_R;origin_mode;protect14;BE_unfulfilled;partial_unfu
      | lfilled;empty_paths;maximum_monthly_DD_percent;add_fills;cycles_with_add;net_cycles_with_add;add_rejec
      | tions;risk_blocks;initial_stop_factor;reinvestment_enabled;management_arm;initial_lots_sum;add_lots_su
      | m;pivot_buy_candidates;pivot_sell_candidates;entry_policy;reinvestment_fraction;reinvestment_lot_cap;c

```

```

    apital_mode;matched_control190;detail_export_enabled;exported_bar_rows;risk_percent;margin_cap_percent
    ;donchian_mode;source_control180;fixed_lot_mode;target_unit_code;target_value;fixed_target_distance_pr
    ice;nominal_target_R;target_rejections;target_cap_adjustments;target_cap_failures;fixed_volume_rejecti
    ons;hp_candidates;hp_extra_eligible;hp_base_selected;hp_pivot_selected;hp_pivot_fills;hp_data_failure_
    bars;hp_extra_position_blocks;hp_historical_catchup_candidates_discarded",'',names);
1798 }
1799 string MonthHeader()
1800 { return "month_server;equity_start;equity_end;equity_change;balance_start;balance_end;booked_net;cyc
    le_net_by_exit;equity_dd_relative_percent;equity_dd_money;evaluated_bars;signals;opened_cycles;closed_
    cycles;BE_confirmed;partial_exits;first_tick;last_tick"; }
1801 void MonthValues(const int i,double &v[])
1802 {
1803     v[0]=months[i].key; v[1]=months[i].path.startEquity; v[2]=months[i].path.endEquity; v[3]=v[2]-v[1];
1804     v[4]=months[i].path.startBalance; v[5]=months[i].path.endBalance; v[6]=months[i].bookedNet;
    v[7]=months[i].cycleNet;
1805     v[8]=months[i].path.maxRelativeDD; v[9]=months[i].path.maxMoneyDD; v[10]=months[i].bars;
    v[11]=months[i].signals;
1806     v[12]=months[i].opened; v[13]=months[i].closed; v[14]=months[i].bes; v[15]=months[i].partials;
1807     v[16]=(double)months[i].firstTick; v[17]=(double)months[i].lastTick;
1808 }
1809 string MonthCSV(const double &data[],const int start)
1810 {
1811     string s=""; for(int j=0;j<MONTH_FIELDS;j++) {
1812         double v=data[start+j];
1813         if(j==0) { int k=(int)v; Cell(s,I(k/100)+"-"+((k%100)<10 ? "0" : "")+I(k%100)); }
1814         else Cell(s,j>=16 ? TS((datetime)v) : N(v));
1815     } return s;
1816 }
1817 // Eight fields per threshold: threshold,closed,reached,returned,loser_returned,
1818 // winner_returned,zero_returned,sum_first_return_quote_R. Observations, NOT BE fills.
1819 void ShadowValues(const int t,double &v[])
1820 {
1821     ArrayInitialize(v,0); v[0]=PETHreshold(t);
1822     if(InpCase!=1) return;
1823     for(int i=0;i<ArraySize(cycles);i++) if(cycles[i].closed) {
1824         v[1]++; if(cycles[i].path.reached[t]) v[2]++;
1825         if(!cycles[i].path.returned[t]) continue;
1826         v[3]++; if(cycles[i].net<0) v[4]++; else if(cycles[i].net>0) v[5]++; else v[6]++;
1827         v[7]+=cycles[i].path.returnQuoteR[t];
1828     }
1829 }
1830 string ShadowHeader()
1831 { return "threshold_R;control_cycles;threshold_reached;returned_to_entry;losers_returned;winners_retu
    rned;zero_returned;sum_first_return_quote_R"; }
1832 void ExportDeals()
1833 {
1834     if(!HistorySelect(0,TimeCurrent()+86400)) { Invalid(401,"Cannot read final deal history."); return;
    }
1835     for(int i=0;i<ArraySize(months);i++) months[i].bookedNet=0;
1836     int f=folder!=" " ? OpenText(folder+"\\deals.csv") : INVALID_HANDLE;
1837     if(folder!=" " && f==INVALID_HANDLE) Invalid(402,"Cannot write deals.csv.");
1838     Row(f,"time_server;cycle;origin_case140;deal;order;position_id;entry_type;side;reason;volume;price;p
    rofit;commission;swap;fee;net_deal;magic;comment");
1839     for(int i=0;i<HistoryDealsTotal();i++) {
1840         ulong d=HistoryDealGetTicket(i); if(d==0 || HistoryDealGetString(d,DEAL_SYMBOL)!=_Symbol) continue;
1841         int n=CycleByPid((ulong)HistoryDealGetInteger(d,DEAL_POSITION_ID));
1842         if(n<0) { if((ulong)HistoryDealGetInteger(d,DEAL_MAGIC)==InpMagic) unmatchedDeals++; continue; }
1843         double profit=HistoryDealGetDouble(d,DEAL_PROFIT),swap=HistoryDealGetDouble(d,DEAL_SWAP);
1844         double fee=HistoryDealGetDouble(d,DEAL_FEE),commission=HistoryDealGetDouble(d,DEAL_COMMISSION);
1845         datetime time=(datetime)HistoryDealGetInteger(d,DEAL_TIME);
1846         months[MonthIndex(time)].bookedNet+=profit+swap+fee+commission;
1847         string s=""; Cell(s,TS(time)); Cell(s,I(cycles[n].number)); Cell(s,I(cycles[n].origin));
        Cell(s,U(d));
1848         Cell(s,I(HistoryDealGetInteger(d,DEAL_ORDER)));
        Cell(s,I(HistoryDealGetInteger(d,DEAL_POSITION_ID)));
1849         Cell(s,EnumToString((ENUM_DEAL_ENTRY)HistoryDealGetInteger(d,DEAL_ENTRY)));
1850         Cell(s,EnumToString((ENUM_DEAL_TYPE)HistoryDealGetInteger(d,DEAL_TYPE)));
1851         Cell(s,EnumToString((ENUM_DEAL_REASON)HistoryDealGetInteger(d,DEAL_REASON)));
        Cell(s,N(HistoryDealGetDouble(d,DEAL_VOLUME)); Cell(s,N(HistoryDealGetDouble(d,DEAL_PRICE)));
    }

```

```

1852 |     Cell(s,N(profit)); Cell(s,N(commission)); Cell(s,N(swap)); Cell(s,N(fee));
      |     Cell(s,N(profit+commission+swap+fee));
1853 |     Cell(s,I(HistoryDealGetInteger(d,DEAL_MAGIC)); Cell(s,HistoryDealGetString(d,DEAL_COMMENT));
      |     Row(f,s);
1854 | }
1855 | if(f!=INVALID_HANDLE) FileClose(f);
1856 | }
1857 | void ExportDetails()
1858 | {
1859 |     if(folder=="") return;
1860 |     int f=OpenText(folder+"\\cycles.csv"); if(f==INVALID_HANDLE) { Invalid(403,"Cannot write
      | cycles.csv."); return; }
1861 |     Row(f,"cycle;origin_casel40;side;entry_time;exit_time;signal_time;closed;initial_entry;initial_stop;
      | target;initial_lot;initial_risk;peak_stop_risk;balance_before;net;gross;swap;commission_fees;net_R_ini
      | tial;hold_hours;legs;adds;position0;position1;position2;add1_price;add1_lot;add2_price;add2_lot;BE_tri
      | gger_R;BE_armed;BE_confirmed_time;partial_armed;partial_time;partial_lot;partial_price;MFE_R;MAE_R;quo
      | te_observations;exit_reason;ADX_entry;ATR_entry;entry_distance_ATR;SMA200_slope_ATR;entry_policy;oscil
      | lator_valid;fast310_entry;previous_fast310_entry;signal310_entry;ADX_change_entry;TR_ATR_entry;body_AT
      | R_entry;close_location_entry;source_control180;matched_control190;fixed_lot_mode;target_unit;target_va
      | lue;requested_target_distance_price;quoted_entry;quoted_target;actual_target_distance_price;actual_tar
      | get_R;estimated_target_gross_profit;target_cap_adjustments;selected_entry_path;hp_signal_time;hp_avail
      | able_time;hp_p1_price;hp_p2_price;hp_p3_price;hp_p1_time;hp_p2_time;hp_p3_time;hp_pattern_available_ti
      | me");
1862 |     for(int i=0;i<ArraySize(cycles);i++) {
1863 |         PECycle c=cycles[i]; string s=""; Cell(s,I(c.number)); Cell(s,I(c.origin)); Cell(s,I(c.side));
1864 |         Cell(s,TS(c.start)); Cell(s,TS(c.finish)); Cell(s,TS(c.signalTime)); Cell(s,I(c.closed));
1865 |         Cell(s,N(c.entry)); Cell(s,N(c.stop)); Cell(s,N(c.target)); Cell(s,N(c.volume));
      | Cell(s,N(c.initialRisk)); Cell(s,N(c.peakRisk));
1866 |         Cell(s,N(c.balanceBefore)); Cell(s,N(c.net)); Cell(s,N(c.gross)); Cell(s,N(c.swap));
      | Cell(s,N(c.costs));
1867 |         Cell(s,c.initialRisk>0 ? N(c.net/c.initialRisk) : ""); Cell(s,c.finish>0 ?
      | N((c.finish-c.start)/3600.0) : "");
1868 |         Cell(s,I(c.legs)); Cell(s,I(c.adds)); for(int j=0;j<3;j++) Cell(s,U(c.pids[j]));
1869 |         for(int j=1;j<3;j++) { Cell(s,N(c.fill[j])); Cell(s,N(c.volumes[j])); }
1870 |         Cell(s,N(c.beTrigger)); Cell(s,I(c.be.armed)); Cell(s,TS(c.beTime)); Cell(s,I(c.part.armed));
1871 |         Cell(s,TS(c.partialTime)); Cell(s,N(c.partialVolume)); Cell(s,N(c.partialPrice));
1872 |         Cell(s,N(c.path.maxR)); Cell(s,N(c.path.minR)); Cell(s,I(c.path.observations));
      | Cell(s,EnumToString((ENUM_DEAL_REASON)c.finalReason));
1873 |         Cell(s,N(c.featureADX)); Cell(s,N(c.featureATR)); Cell(s,N(c.featureDistance));
      | Cell(s,N(c.featureSlope));
1874 |         Cell(s,I(c.entryPolicy)); Cell(s,I(c.entryFeatures.oscillatorValid));
      | Cell(s,N(c.entryFeatures.fast310));
1875 |         Cell(s,N(c.entryFeatures.previousFast310)); Cell(s,N(c.entryFeatures.signal310));
      | Cell(s,N(c.featureADXChange));
1876 |         Cell(s,N(c.entryFeatures.trueRangeATR)); Cell(s,N(c.entryFeatures.bodyATR));
      | Cell(s,N(c.entryFeatures.closeLocation));
1877 |         Cell(s,I(dc.sourceControl180)); Cell(s,I(dc.matchedControl)); Cell(s,I(dc.fixedLot));
      | Cell(s,TargetUnitName());
1878 |         Cell(s,N(dc.targetMode==0 ? 5 : 20)); Cell(s,N(c.targetDistance)); Cell(s,N(c.quotedEntry));
      | Cell(s,N(c.quotedTarget));
1879 |         Cell(s,N(c.side*(c.target-c.entry))); Cell(s,MathAbs(c.entry-c.stop)>0 ?
      | N(c.side*(c.target-c.entry)/MathAbs(c.entry-c.stop)) : "");
1880 |         Cell(s,N(c.targetProfitEstimate)); Cell(s,I(c.targetAdjustments));
1881 |         Cell(s,HPPPathName(c.entryPath)); Cell(s,TS((datetime)c.entryPivot.signal_time));
      | Cell(s,TS((datetime)c.entryPivot.available_time));
1882 |         Cell(s,N(c.entryPivot.stop_f1)); Cell(s,N(c.entryPivot.reference_price));
      | Cell(s,N(c.entryPivot.pullback_f2));
1883 |         Cell(s,TS((datetime)c.entryPivot.p1_time)); Cell(s,TS((datetime)c.entryPivot.p2_time));
      | Cell(s,TS((datetime)c.entryPivot.p3_time));
1884 |         Cell(s,TS((datetime)c.entryPivot.pattern_available_time)); Row(f,s);
1885 |     }
1886 |     FileClose(f);
1887 |     f=OpenText(folder+"\\months.csv"); if(f==INVALID_HANDLE) { Invalid(404,"Cannot write months.csv.");
      | return; }
1888 |     Row(f,MonthHeader()); for(int i=0;i<ArraySize(months);i++) { double v[18]; MonthValues(i,v);
      | Row(f,MonthCSV(v,0)); } FileClose(f);
1889 |     f=OpenText(folder+"\\breakeven_paths.csv"); if(f==INVALID_HANDLE) { Invalid(405,"Cannot write
      | breakeven_paths.csv."); return; }
1890 |     Row(f,"cycle;original_control;threshold_R;threshold_reached;returned_after_threshold;first_return_qu

```

```

    ote_R;net_original;MFE_R;MAE_R");
1891 |     for(int i=0;i<ArraySize(cycles);i++) for(int t=0;t<5;t++) {
1892 |         string s=""; Cell(s,I(cycles[i].number)); Cell(s,I(InpCase==1)); Cell(s,N(PETreshold(t)));
1893 |         Cell(s,I(cycles[i].path.reached[t])); Cell(s,I(cycles[i].path.returned[t]));
1894 |         Cell(s,cycles[i].path.returned[t] ? N(cycles[i].path.returnQuoteR[t]) : "");
1895 |         Cell(s,N(cycles[i].net)); Cell(s,N(cycles[i].path.maxR)); Cell(s,N(cycles[i].path.minR)); Row(f,s);
1896 |     } FileClose(f);
1897 | }
1898 | // Numeric frame carries statistics and ASCII relative-folder index, never raw future prices.
1899 | void BuildPayload(const double &stats[],double &payload[])
1900 | {
1901 |     int base=S_COUNT+G_COUNT,n=ArraySize(months),len=StringLen(folder);
1902 |     int tail=base+1+n*MONTH_FIELDS+5*SHADOW_FIELDS; ArrayResize(payload,tail+1+len);
1903 |     for(int i=0;i<base;i++) payload[i]=stats[i]; payload[base]=n;
1904 |     for(int m=0;m<n;m++) { double v[18]; MonthValues(m,v); for(int j=0;j<18;j++)
        payload[base+1+m*18+j]=v[j]; }
1905 |     int at=base+1+n*18;
1906 |     for(int t=0;t<5;t++) { double v[8]; ShadowValues(t,v); for(int j=0;j<8;j++) payload[at+t*8+j]=v[j];
    }
1907 |     payload[tail]=len; for(int j=0;j<len;j++) payload[tail+1+j]=StringGetCharacter(folder,j);
1908 | }
1909 | bool PayloadValid(const double &p[])
1910 | {
1911 |     int base=S_COUNT+G_COUNT; if(ArraySize(p)<base+1 || !MathIsValidNumber(p[base]) || p[base]<0 ||
        p[base]>1200) return false;
1912 |     int n=(int)p[base]; if(p[base]!=n) return false;
1913 |     int tail=base+1+n*18+40;
1914 |     if(ArraySize(p)<tail+1 || !MathIsValidNumber(p[tail]) || p[tail]<0 || p[tail]>512) return false;
1915 |     int len=(int)p[tail]; if(p[tail]!=len || ArraySize(p)!=tail+1+len) return false;
1916 |     for(int i=0;i<len;i++) if(!MathIsValidNumber(p[tail+1+i]) || p[tail+1+i]<32 || p[tail+1+i]>126 ||
        p[tail+1+i]!=(int)p[tail+1+i]) return false;
1917 |     return true;
1918 | }
1919 | string PayloadFolder(const double &p[])
1920 | {
1921 |     if(!PayloadValid(p)) return "";
1922 |     int tail=S_COUNT+G_COUNT+1+(int)p[S_COUNT+G_COUNT]*18+40; string name="";
1923 |     for(int i=0;i<(int)p[tail];i++) name+=ShortToString((ushort)p[tail+1+i]); return name;
1924 | }
1925 | double OnTester()
1926 | {
1927 |     if(!initialized) return -1e100;
1928 |     finalizing=true; SyncCycle(); TrackEquity();
1929 |     // Rebuild all P&L from the executed final history, including forced tester exits
1930 |     // that may arrive without another OnTick and may carry magic zero.
1931 |     for(int i=0;i<ArraySize(cycles);i++) {
1932 |         bool ok=AggregateCycle(i); cycles[i].closed=ok &&
            MathAbs(cycles[i].inVolume-cycles[i].outVolume)<1e-8 && cycles[i].finish>0;
1933 |         if(!cycles[i].closed) Invalid(406,"Open/unreconciled campaign at tester end.");
1934 |         if(cycles[i].closed && !cycles[i].finalObserved) { PEObserve(cycles[i].path,cycles[i].finalPrice);
            cycles[i].finalObserved=true; }
1935 |     }
1936 |     for(int m=0;m<ArraySize(months);m++) { months[m].closed=0; months[m].cycleNet=0; }
1937 |     for(int i=0;i<ArraySize(cycles);i++) if(cycles[i].closed) { int m=MonthIndex(cycles[i].finish);
        months[m].closed++; months[m].cycleNet+=cycles[i].net; }
1938 |     if(firstReady==0) Invalid(407,"No complete indicator snapshot available.");
1939 |     ExportDeals();
1940 |     double data[S_COUNT+G_COUNT]; ArrayInitialize(data,0);
1941 |     data[S_PROFIT]=TesterStatistics(STAT_PROFIT);
        data[S_DEPOSIT]=TesterStatistics(STAT_INITIAL_DEPOSIT);
1942 |     data[S_FINAL_BALANCE]=data[S_DEPOSIT]+data[S_PROFIT];
        data[S_MT5_TRADES]=TesterStatistics(STAT_TRADES);
1943 |     data[S_MT5_PF]=TesterStatistics(STAT_PROFIT_FACTOR);
        data[S_EQUITY_DD_REL]=TesterStatistics(STAT_EQUITY_DDREL_PERCENT);
1944 |     data[S_EQUITY_DD_MONEY]=TesterStatistics(STAT_EQUITY_DD);
        data[S_EQUITY_DD_AT_MONEY]=TesterStatistics(STAT_EQUITYDD_PERCENT);
1945 |     data[S_CYCLES_OPENED]=ArraySize(cycles); double winning=0,losing=0; int streak=0;
1946 |     for(int i=0;i<ArraySize(cycles);i++) {
1947 |         PECycle c=cycles[i]; data[S_BE_ARMED]+=c.be.armed; data[S_BE_CONFIRMED]+=c.beTime>0;

```

```

1948 | data[S_PARTIAL_ARMED]+=c.part.armed; data[S_PARTIAL_DONE]+=c.part.done;
1949 | data[S_BE_ATTEMPTS]+=c.be.attempts; data[S_PARTIAL_ATTEMPTS]+=c.part.attempts;
1950 | if(c.be.armed && !c.be.done) data[S_BE_UNFULFILLED]++;
1951 | if(c.part.armed && !c.part.done) data[S_PARTIAL_UNFULFILLED]++;
1952 | data[S_INITIAL_LOTS_SUM]+=c.volume; for(int j=1;j<c.legs;j++) data[S_ADD_LOTS_SUM]+=c.volumes[j];
1953 | data[S_ADD_FILLS]+=c.adds;
1954 | if(c.path.observations==0) data[S_EMPTY_PATHS]++;
1955 | if(!c.closed) continue;
1956 | data[S_CYCLES_CLOSED]++; data[S_CYCLE_NET]+=c.net; data[S_SWAP]+=c.swap; data[S_COSTS]+=c.costs;
1957 | if(c.origin==15) { data[S_ORIGIN15_CYCLES]++; data[S_ORIGIN15_NET]+=c.net; }
1958 | if(c.origin==14) { data[S_ORIGIN14_CYCLES]++; data[S_ORIGIN14_NET]+=c.net; }
1959 | if(c.adds>0) { data[S_ADD_CYCLES]++; data[S_ADD_NET]+=c.net; }
1960 | if(c.net>0) { data[S_WINS]++; winning+=c.net; streak=0; data[S_MFE_WIN_AVG]+=c.path.maxR; }
1961 | else if(c.net<0) { data[S_LOSSES]++; losing=-c.net; streak++; data[S_MFE_LOSS_AVG]+=c.path.maxR; }
1962 | else { data[S_ZERO]++; streak=0; }
1963 | data[S_MAX_LOSS_STREAK]=MathMax(data[S_MAX_LOSS_STREAK],streak);
1964 | if(c.initialRisk>0) data[S_AVG_NET_R]+=c.net/c.initialRisk;
1965 | double hold=(c.finish-c.start)/3600.0; data[S_AVG_HOLD]+=hold;
1966 | data[S_MAX_HOLD]=MathMax(data[S_MAX_HOLD],hold);
1967 | data[S_MFE_AVG]+=c.path.maxR; data[S_MAE_AVG]+=c.path.minR;
1968 | data[S_QUOTE_OBSERVATIONS]+=c.path.observations;
1969 | if(c.beTime>0 && c.finalReason==DEAL_REASON_SL) data[S_BE_STOP_EXITS]++;
1970 | if(data[S_FIRST_ENTRY]==0 || c.start<data[S_FIRST_ENTRY]) data[S_FIRST_ENTRY]=(double)c.start;
1971 | data[S_LAST_EXIT]=MathMax(data[S_LAST_EXIT],(double)c.finish);
1972 | }
1973 | double n=data[S_CYCLES_CLOSED];
1974 | if(n>0) { data[S_WIN_PCT]=100*data[S_WINS]/n; data[S_AVG_NET_R]/=n; data[S_AVG_HOLD]/=n;
1975 | data[S_MFE_AVG]/=n; data[S_MAE_AVG]/=n; }
1976 | if(data[S_WINS]>0) data[S_MFE_WIN_AVG]/=data[S_WINS]; if(data[S_LOSSES]>0)
1977 | data[S_MFE_LOSS_AVG]/=data[S_LOSSES];
1978 | data[S_CYCLE_PF]=losing>0 ? winning/losing : (winning>0 ? DBL_MAX : 0);
1979 | data[S_MAX_LOTS]=maxLots; data[S_MAX_RISK]=maxRisk; data[S_MAX_RISK_PCT]=maxRiskPct;
1980 | data[S_MAX_MARGIN]=maxMargin;
1981 | data[S_FIRST_EVAL]=(double)firstEval; data[S_LAST_EVAL]=(double)lastEval;
1982 | data[S_FIRST_READY]=(double)firstReady;
1983 | data[S_BARS]=barsSeen; data[S_SIGNALS]=signals; data[S_PARTIAL_DEFERRED]=partialDefers;
1984 | data[S_BE_DEFERRED]=beDefers;
1985 | data[S_SESSION_DEFERS]=sessionDefers; data[S_STOPS_DEFERS]=stopDefers;
1986 | data[S_AMBIGUOUS_PARTIALS]=ambiguousPartials;
1987 | data[S_UNMATCHED]=unmatchedDeals; data[S_RECONCILE]=data[S_PROFIT]-data[S_CYCLE_NET];
1988 | double eq=0,book=0;
1989 | for(int m=0;m<ArraySize(months);m++) {
1990 |     book+=months[m].bookedNet; if(!months[m].path.initialized) continue;
1991 |     data[S_MONTHS]++; double delta=months[m].path.endEquity-months[m].path.startEquity; eq+=delta;
1992 |     if(delta>.005) data[S_POS_EQ_MONTHS]++; if(delta<-.005) data[S_NEG_EQ_MONTHS]++;
1993 |     if(months[m].bookedNet>.005) data[S_POS_BOOK_MONTHS]++; if(months[m].bookedNet<-.005)
1994 |     data[S_NEG_BOOK_MONTHS]++;
1995 |     data[S_MONTH_DD_MAX]=MathMax(data[S_MONTH_DD_MAX],months[m].path.maxRelativeDD);
1996 | }
1997 | data[S_MONTH_EQ_RECONCILE]=data[S_PROFIT]-eq; data[S_MONTH_BOOK_RECONCILE]=data[S_PROFIT]-book;
1998 | if(MathAbs(data[S_RECONCILE])>.01 || MathAbs(data[S_MONTH_EQ_RECONCILE])>.01 ||
1999 | MathAbs(data[S_MONTH_BOOK_RECONCILE])>.01 ||
2000 | data[S_UNMATCHED]>0 || data[S_CYCLES_CLOSED]!=data[S_CYCLES_OPENED])
2001 | Invalid(408,"Native/cycle/month totals do not reconcile.");
2002 | data[S_SHADOW_CONTROL_ELIGIBLE]=(InpCase==1) ? 1 : 0;
2003 | data[S_FIXED_LOT]=dc.fixedLot ? InpFixedLot : 0; data[S_BE15_R]=profile.be15;
2004 | data[S_MODE]=profile.mode; data[S_PROTECT14]=profile.protect14;
2005 | data[S_ADD_REJECTS]=addRejects; data[S_RISK_BLOCKS]=riskBlocks;
2006 | data[S_STOP_FACTOR]=profile.stopFactor;
2007 | data[S_REINVEST]=profile.reinvest; data[S_ARM]=profile.arm; data[S_PATTERN_BUYs]=patternBuys;
2008 | data[S_PATTERN_SELLs]=patternSells;
2009 | data[S_ENTRY_POLICY]=evo.entryPolicy; data[S_REINVEST_FRACTION]=evo.reinvestFraction;
2010 | data[S_REINVEST_CAP]=dc.fixedLot ? InpFixedLot : InpMaxLot; data[S_CAPITAL_MODE]=evo.capitalMode;
2011 | data[S_RISK_PERCENT]=dc.riskPercent; data[S_MARGIN_CAP_PERCENT]=dc.fixedLot ? 0 : InpMaxMarginPct;
2012 | data[S_DONCHIAN_MODE]=dc.channelMode;
2013 | data[S_SOURCE_CONTROL180]=dc.sourceControl180; data[S_FIXED_MODE]=dc.fixedLot;
2014 | data[S_TARGET_MODE]=dc.targetMode;
2015 | data[S_TARGET_VALUE]=dc.targetMode==0 ? 5 : 20;
2016 | data[S_TARGET_DISTANCE]=HermesTargetDistance(dc.targetMode,_Point);

```

```

2000 | data[S_NOMINAL_TARGET_R]=dc.targetMode==0 ? 5 : 0; data[S_TARGET_REJECTS]=targetRejects;
    | data[S_TARGET_ADJUSTMENTS]=targetAdjustments;
2001 | data[S_TARGET_FAILURES]=targetFailures; data[S_FIXED_VOLUME_REJECTS]=fixedVolumeRejects;
2002 | data[S_HP_CANDIDATES]=hpCandidates; data[S_HP_EXTRA_ELIGIBLE]=hpExtraEligible;
2003 | data[S_HP_BASE_SELECTED]=hpBaseSelected; data[S_HP_PIVOT_SELECTED]=hpPivotSelected;
    | data[S_HP_PIVOT_FILLS]=hpPivotFills;
2004 | data[S_HP_DATA_FAILURES]=hpDataFailures; data[S_HP_POSITION_BLOCKS]=hpPositionBlocks;
    | data[S_HP_CATCHUP_IGNORED]=hpCatchupIgnored;
2005 | data[S_MATCHED_CONTROL]=dc.matchedControl; data[S_DETAIL_EXPORT]=folder!="";
    | data[S_BAR_ROWS]=barRows;
2006 | ExportDetails(); if(ioFailure) Invalid(410,"Detail CSV write failure.");
2007 | data[S_ERROR_CODE]=errorCode; data[S_LAST_RETCODE]=lastRetcode;
    | data[S_LAST_ERROR_TIME]=(double)lastErrorTime;
2008 | data[S_VALID]=integrity ? 1 : 0; for(int i=0;i<G_COUNT;i++) data[S_COUNT+i]=counters[i];
2009 | if(folder!="") {
2010 |     int f=OpenText(folder+"\\summary.csv");
2011 |     if(f==INVALID_HANDLE) { Invalid(409,"Cannot write summary.csv."); data[S_VALID]=0; }
2012 |     else { Row(f,"metric;value"); KV(f,"version","2.00"); KV(f,"case",I(InpCase));
    | KV(f,"name",ProfileName(InpCase));
    | KV(f,"signal_tf",TFName(InpCase)); string names[]; StatNames(names);
2013 |     for(int i=0;i<S_COUNT;i++) KV(f,names[i],IsTimeStat(i) ? TS((datetime)data[i]) : N(data[i]));
2014 |     for(int i=0;i<G_COUNT;i++) KV(f,EnumToString((PE_GATE)i),N(data[S_COUNT+i])); FileClose(f);
2015 | }
2016 | }
2017 | }
2018 | if(ioFailure) { Invalid(410,"CSV write failure."); data[S_VALID]=0; data[S_ERROR_CODE]=errorCode; }
2019 | if(MQInfoInteger(MQL_OPTIMIZATION)) { double payload[]; BuildPayload(data,payload);
    | if(!FrameAdd("HP200",InpCase,data[S_PROFIT],payload)) Print("FrameAdd failed: ",GetLastError()); }
2020 | Print("HP200 ",ProfileName(InpCase)," lucro=",data[S_PROFIT]," ciclos=",n," valid_run=",integrity);
2021 | return integrity ? data[S_PROFIT] : -1e100;
2022 | }
2023 | int OnTesterInit()
2024 | {
2025 |     string names[]; if(StatNames(names)!=S_COUNT) { Print("STAT SCHEMA ERROR"); return INIT_FAILED; }
2026 |     if(!ValidRunTag()) return INIT_PARAMETERS_INCORRECT;
2027 |     if(!InpExportCSV) return INIT_SUCCEEDED;
2028 |     FolderCreate("Hermes_Pivos_Lab_200",FILE_COMMON); FolderCreate(RunRoot(),FILE_COMMON);
    | optFolder=RunRoot()+"\\OTIMIZACAO_"+Stamp();
2029 |     if(!FolderCreate(optFolder,FILE_COMMON)) return INIT_FAILED;
2030 |     optFile=OpenText(optFolder+"\\comparacao.csv");
    | optMonths=OpenText(optFolder+"\\meses_comparacao.csv");
2031 |     optShadow=OpenText(optFolder+"\\breakeven_comparacao.csv");
    | optFiles=OpenText(optFolder+"\\arquivos_comparacao.csv");
2032 |     if(optFile==INVALID_HANDLE || optMonths==INVALID_HANDLE || optShadow==INVALID_HANDLE ||
    | optFiles==INVALID_HANDLE) return INIT_FAILED;
2033 |     string header="pass;case;name;signal_tf"; for(int i=0;i<S_COUNT;i++) header+=";"+names[i];
2034 |     for(int i=0;i<G_COUNT;i++) header+=";"+EnumToString((PE_GATE)i); Row(optFile,header);
2035 |     Row(optMonths,"pass;case;name;"+MonthHeader());
    | Row(optShadow,"pass;case;name;valid_run;"+ShadowHeader());
2036 |     Row(optFiles,"pass;case;name;relative_details_folder;details_exported");
2037 |     if(ioFailure) return INIT_FAILED;
2038 |     Print("COMPARACAO AUTOMATICA: ",TerminalInfoString(TERMINDATA_PATH),"\\Files\\",optFolder);
    | return INIT_SUCCEEDED;
2039 | }
2040 | void DrainFrames()
2041 | {
2042 |     ulong pass=0; long id=0; string name=""; double value=0,data[];
2043 |     while(FrameNext(pass,name,id,value,data)) {
2044 |         if(name!="HP200" || optFile==INVALID_HANDLE || !PayloadValid(data) || id<1 || id>3) continue;
2045 |         bool seen=false; for(int i=0;i<ArraySize(receivedPasses);i++) if(receivedPasses[i]==pass)
    | seen=true; if(seen) continue;
2046 |         int n=ArraySize(receivedPasses); ArrayResize(receivedPasses,n+1); receivedPasses[n]=pass;
2047 |         string prefix=""; Cell(prefix,U(pass)); Cell(prefix,I(id)); Cell(prefix,ProfileName((int)id));
2048 |         string row=prefix; Cell(row,PayloadFolder(data)); Cell(row,N(data[S_DETAIL_EXPORT]));
    | Row(optFiles,row);
2049 |         row=prefix; Cell(row,TFName((int)id));
2050 |         for(int i=0;i<S_COUNT+G_COUNT;i++) Cell(row,IsTimeStat(i) ? TS((datetime)data[i]) : N(data[i]));
    | Row(optFile,row);
2051 |         int base=S_COUNT+G_COUNT,mc=(int)data[base];
2052 |         for(int m=0;m<mc;m++) Row(optMonths,prefix+";"+MonthCSV(data,base+1+m*18));

```

```

2053 |     if(id==1) for(int t=0;t<5;t++) {
2054 |         row=prefix; Cell(row,N(data[S_VALID]));
2055 |         for(int j=0;j<8;j++) Cell(row,N(data[base+1+mc*18+t*8+j])); Row(optShadow,row);
2056 |     }
2057 |     FileFlush(optFile); FileFlush(optMonths); FileFlush(optShadow); FileFlush(optFiles);
2058 | }
2059 | }
2060 | void OnTesterPass() { DrainFrames(); }
2061 | void OnTesterDeinit()
2062 | {
2063 |     DrainFrames(); if(optFile!=INVALID_HANDLE) FileClose(optFile); if(optMonths!=INVALID_HANDLE)
2064 |     FileClose(optMonths);
2065 |     if(optFiles!=INVALID_HANDLE) FileClose(optFiles); optFiles=INVALID_HANDLE;
2066 |     if(optShadow!=INVALID_HANDLE) FileClose(optShadow); optFile=INVALID_HANDLE;
2067 |     optMonths=INVALID_HANDLE; optShadow=INVALID_HANDLE;
2068 |     Print("HP200 comparacoes recebidas: ",ArraySize(receivedPasses),". Confira com as passagens
2069 |     concluidas.");
2070 | }
2071 | void OnDeinit(const int reason)
2072 | {
2073 |     if(barsFile!=INVALID_HANDLE) FileClose(barsFile); if(eventsFile!=INVALID_HANDLE)
2074 |     FileClose(eventsFile);
2075 |     if(hEMA!=INVALID_HANDLE) IndicatorRelease(hEMA); if(h50!=INVALID_HANDLE) IndicatorRelease(h50);
2076 |     if(h200!=INVALID_HANDLE) IndicatorRelease(h200);
2077 |     if(hADX!=INVALID_HANDLE) IndicatorRelease(hADX); if(hATR!=INVALID_HANDLE) IndicatorRelease(hATR);
2078 |     Comment("");
2079 | }

```